Tempest Fauna Trail

Project Documentation — a weather-driven roguelike built with AI coding agents

Merlin Duty-Knez (Meduty) Adam Ebner (AdamZ0904)
https://github.com/Meduty/tempest-fauna-trail

July 2026

Contents

Chapter 1

Overview

Weather-driven roguelike where animal champions battle across real-world cities. Live OpenWeather data shapes every fight -- a Snow-affinity team dominates in snowy Tokyo but folds when the city turns Thunder.

Built with Flet (Python), cross-platform desktop + web. Designed to be developed collaboratively with AI coding agents -- see CLAUDE.md and SPEC.md.

1.1 Status

Playable end to end -- the full menu -> trail -> prep -> combat -> reward -> summary loop runs over the finished engine. Every functional task in SPEC.md §T is complete; the only open row is T.17 (this documentation) plus ongoing polish.

- [done] **Done** -- data models, tick combat engine (ability/passive/status framework, bosses, map effects), weather effects, 50-city route, champion/enemy/boss rosters + ability catalog, power scaling, encounter generation, economy & shop, item/augment/trait systems, Prep equip seam, OpenWeather client + cache + 3-stream refresher, theme/components, Flet views (menu, run-start, trail, prep, combat, augment, supply, reward, summary, settings), route-map + run-summary + affinity-clash Canvas visualizations, save/load, playtest CLI, power-simulation tooling.
- [planned] **Open** -- T.17 documentation (partial) + ongoing polish. See SPEC.md §T for per-task status.

For how the systems fit together and where to find them, read **ARCHITECTURE.md**.

1.2 Quickstart

1.2.1 Requirements

- Python >= 3.10
- OpenWeather API key (free tier -- openweathermap.org/api)

1.2.2 Setup

```
git clone <repo>
cd tempest-fauna-trail
uv sync                        # or: pip install -e . && pip install -r requirements.txt
cp .env.example .env
# edit .env, set OPENWEATHER_API_KEY=<your-key>
```

1.2.3 Weather API key (optional, but recommended)

The Trail shows live weather per city -- and that live weather **shapes combat**: each node's weather is frozen when you enter Prep, then drives Weather Favor for the whole fight (V.73). Without a key the game still plays on each city's deterministic default weather, so runs stay reproducible either way -- weather is locked *before* the fight resolves, so live-feed timing never breaks determinism (V.2). Configure the key **either** way:

1. `.env` **file** (preferred for local dev) -- `cp .env.example .env`, then edit the one line:

   ```
   OPENWEATHER_API_KEY=your_actual_key_here
   ```

   No quotes, no spaces around `=`. `.env` is git-ignored; the app/tests load it via `python-dotenv`.

2. **Environment variable** (CI / shells / one-off):

   ```
   export OPENWEATHER_API_KEY=your_actual_key_here
   uv run flet run
   ```

3. **In-app Settings menu** (no terminal needed) -- launch the app, open **Settings** from the main menu, paste your key, **Save**. It persists to a local config file (`~/<user-data>/tempest-fauna-trail/config.json`) and the Trail picks it up next time you open it. An `OPENWEATHER_API_KEY` env var, if set, overrides it.

Without a key the game still runs -- the weather refresher simply never starts, so every Trail node shows a `?` **"weather pending"** marker (the app **never** invents fake weather it hasn't fetched). Add a key to see live weather and the affinity favor/clash it drives. The key is read from the environment and **never logged** (V.3).

1.2.4 Run

```
uv run flet run                # desktop
uv run flet run --web          # browser
```

> **Note:** `uv run flet run` launches the real game (menu -> trail -> ... -> summary). For headless engine work -- no UI -- the **playtest CLI** drives the full engine straight from the terminal:
>
> ```
> uv run python -m tools.playtest.sim_fight --help     # resolve one fight
> uv run python -m tools.playtest.sim_run --help        # simulate a full run
> ```

1.2.5 Tests

```
uv run pytest                          # all tests
uv run pytest -m "not integration"     # unit only (no network)
uv run pytest -m integration           # live API tests (needs OPENWEATHER_API_KEY)
```

1.3 Project Map

Path	Purpose
SPEC.md	Canonical spec -- goals, invariants, tasks, bugs. **Source of truth.**
ARCHITECTURE.md	System map -- how the engine/content/weather/economy systems work and where each lives in the code
CLAUDE.md	AI agent onboarding -- project structure, conventions, workflow
docs/proposal.md	Pitch + design framing
docs/design/tasks/	Per-task plan docs (`tN_*_plan.md`)
docs/design/content/	Champion / enemy / boss / augment / item / trait rosters
docs/design/systems/	Combat, passive, effect, view system designs
docs/journal/	Chronological dev log
src/	Implementation
tests/	Unit + integration tests

1.4 AI-Driven Development

This repo is set up for collaboration with AI coding agents (Claude Code etc.):

- **Source of truth** lives in SPEC.md -- caveman-encoded sections §G/§C/§I/§V (invariants), §T (tasks), §B (bugs), §D (deferred).
- **Skills** wrap key workflows:
  - `/spec` -- sole mutator of SPEC.md (new spec, amend, distill from code, bug backprop)
  - `/build` -- plan-then-execute against §T tasks; auto-runs backprop on test failure
  - `/check` -- read-only drift audit, SPEC vs code
- **Per-path guardrails** in .claude/rules/ bound AI edits by scope (`api.md`, `game-logic.md`, `flet-ui.md`).
- **Journal** in docs/journal/ captures the "why" that SPEC compresses out -- append after non-trivial milestones.

1.5 Build (Flet packaging)

```
flet build apk -v            # Android
flet build ipa -v            # iOS
flet build macos -v
flet build linux -v
flet build windows -v
flet build web -v
```

See Flet build docs for signing + distribution details.

1.6 License

Source-available under the **PolyForm Noncommercial License 1.0.0** -- free to use, modify, and share for any **noncommercial** purpose. **Commercial use requires a separate license** from the authors. See LICENSE and COMMERCIAL.md.

Chapter 2

Architecture

How Tempest Fauna Trail is built, how the systems fit together, and **where to find each one in the code**. This is the map for understanding the game and finding your way around the repo.

> **How the docs relate.** SPEC.md is the *contract* (goals, invariants §V, task table §T, bug log §B) -- the source of truth for *what must hold*. CLAUDE.md is the *agent quick-start*. The journal is the *narrative* -- why decisions were made. **This file is the *system map*** -- what exists, how it interacts, and the file you open to touch it. When in doubt about a rule, SPEC wins; when lost in the tree, start here.

2.1 What the game is

A roguelike **auto-battler** (TFT-inspired) themed around animal spirits and weather. You pick **1 champion**, start with **10 Amber**, and travel a fixed **50-node route** of real-world cities across 6 continent stages. **Live OpenWeather data** at each city warps combat. Battles are **tick-based and auto-resolved** -- all player agency happens *between* fights.

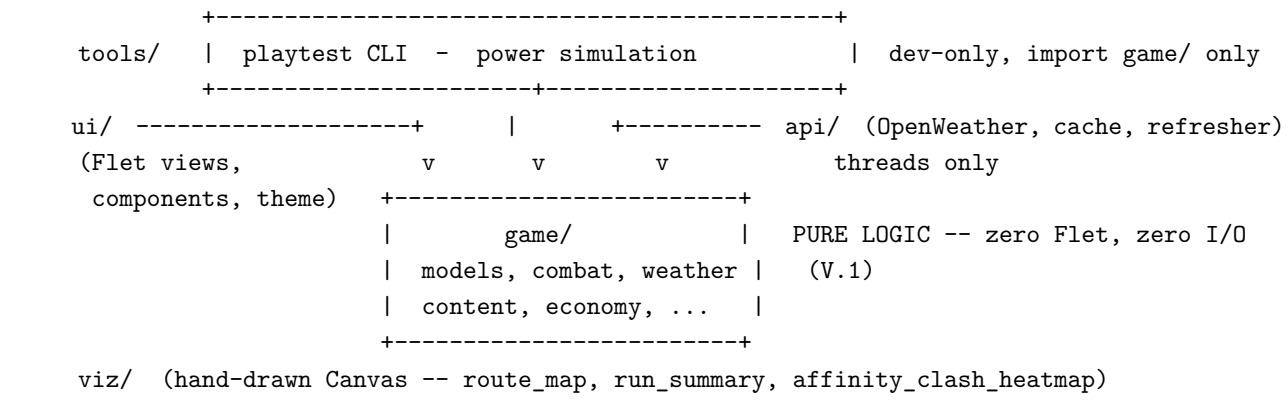
The per-node loop (SPEC §G):

```
Trail  -->  Prep  -->  Combat  -->  (rewards: Amber / items / Tempest XP)  -->  next node
view route   shop, board    auto-resolved
+ preview    placement,      tick battle
weather      level team
```

The player's decisions: **team composition + synergy traits**, **item optimization**, **weather-aware roster swaps**, **board positioning**. The engine then resolves the fight deterministically.

2.2 The layered architecture

Four source layers under `src/`, plus dev tooling under `tools/`. The **cardinal rule is dependency direction** -- everything points *inward* toward pure game logic:



Invariants that enforce this (SPEC §V):

Rule	Invariant
`game/` has **zero Flet imports** -- pure logic	V.1
`tools/simulation/` imports only from `src/game/` (no `ui/`, no `api/`)	V.14
All HTTP runs on a worker `threading.Thread`, never the main thread	V.4
API failure never crashes -- leaves a node `unknown`/`substitute`, keeps retrying	V.3

Because `game/` is pure and I/O-free, the **entire game is testable and simulatable without a UI or network** -- which is exactly what `tools/` exploits.

2.3 The combat engine -- the heart of the system

Combat is a **single pure function** (V.2):

```
resolve_combat(team, enemies, weather, *, node_id="", run_mods=None, positions=None) -> BattleResult
```

Identical inputs -> byte-identical output. No RNG, no clock, no globals. The optional `run_mods` (a `RunModifiers` from `game/augments.py`, T.31) threads active augments + quest state into the fight; it **defaults to** `None`, so every legacy caller and every sim stays byte-for-byte unchanged (V.2). It internally delegates through a fixed chain (the shared `build_combat` helper in `resolve.py`):

```
compile_loadout(team, enemies, weather, run_mods)   # content <-> combat boundary
        |  -> (pieces, EventBus, trait_activations)
        v
CombatContext(pieces, bus, weather, seed)    # the mutator API (board_state optional)
        |
        v
combat/engine.run(ctx)                        # the ONE tick loop (V.29)
        |  events --> EventBus
        v
BattleResultRecorder.build_result()          # rebuilds BattleResult from events
```

Boss fights insert one extra step -- `attach_map_effect(...)` after building the context and before `run()`. The canonical wiring is `src/game/combat/resolve.py::resolve_boss_combat` (T.12b/V.59

-- the single src-side boss entry, takes a `map_effect_id: str` so `combat/` stays content-import-free; `tools/playtest/_common` delegates to it). `CombatReplay/ inspect_at_tick` accept the same `map_effect_id` to replay a boss fight.

2.3.1 Where each piece lives

Concern	File
Public entry (`resolve_combat`) + the shared `build_combat` wiring helper (compile -> assign_spawns -> context, optional recorder) reused by resolve / boss / replay	`src/game/combat/resolve.py`
Package re-exports (`resolve_combat`, `CombatContext`, `run`, `inspect_at_tick`, `CombatReplay`)	`src/game/combat/__init__.py`
The tick loop -- the single `_step_combat` generator (energy meters, pathing, attacks, casts, statuses, map effects, sudden death) + tuning constants; `run` drains it, `CombatReplay` steps it forward (T.37c)	`src/game/combat/engine.py`
Mutator API -- the *only* way content touches the world	`src/game/combat/context.py`
Event -> `BattleResult` reconstruction (beats + `initial_pieces` board snapshot)	`src/game/combat/recorder.py`
Replay -- `CombatReplay` steps the engine **forward** for playback; `inspect_at_tick` re-runs to a tick (random seek) on a cloned `run_mods`; both return read-only `PieceViews`, record nothing (V.55); the live state is the view's resource truth, not the event stream (V.56/V.57, B.28)	`src/game/combat/replay.py`
Compile models -> combat `Pieces` + wire passives/weather/**traits**/augments; **uniquifies duplicate piece ids** so twins in a squad get distinct `ids` (`id`, `id#1`, ...) for stable combat-view/log references (B.65)	`src/game/loadout.py`
Synergy traits -- `TraitScope/TraitBreakpoint/DynamicThreshold`, `@register_trait`, `_resolve_traits` (unique-id count, affinity synthesis, apex/dynamic threshold) applied in `compile_loadout` step 3 (T.28a; primitives T.28b/c)	`src/game/traits/`

> **V.29 -- there is exactly one tick loop.** `engine.py` is it. The old `loop.py` was deleted after the T.26 unification; do not reintroduce a parallel engine (see the 2026-06-04 journal).

2.3.2 The tick model

Time is discretized into **10ms ticks**. A piece acts when its **action energy meter** overflows `ENERGY_THRESHOLD`; movement uses a separate meter. One action ~= **~600 ticks (~6s)** -- this scale matters: it's why DOT fires on a *per-status cadence*, not per tick (V.25). Meter overflow carries over, and triggered meters resolve in a **deterministic order** so replays stay identical.

2.3.3 The ability / passive / status framework (T.20)

Content (abilities, passives, items, traits, augments) never mutates combat state directly. It plugs in through three declarative primitives, then reacts through events:

- `EffectBundle` (`effects.py`) -- a bag of `Modifiers`, `Hooks`, granted statuses / abilities / traits. The unit of "what this thing does."
- `Modifier` (`effects.py`) -- a stat change with a `Lifetime` (PERMANENT / COMBAT / TIMED). Stats are computed via `compute_stat` layering base + modifiers.
- `Hook` (`effects.py`) -- a callback subscribed to the `EventBus` by `event` name, with a `HookScope` dedup (`PER_HIT` / `ONCE_PER_CAST` / `ONCE_PER_TARGET` / `ONCE_PER_COMBAT`). Real event names the loop emits include `on_attack_landed`, `on_attack_start`, `on_damage_pre` / `on_damage_dealt` / `on_damage_taken`, `on_cast` / `on_cast_complete`, `on_death`, `on_kill`, `on_heal`, `on_tick`, `on_status_applied` / `on_status_expired`, `on_spawn`, `on_combat_start` / `on_combat_end`. Typed payloads live in `events.py` (`AttackEvent`, `DamageEvent`, ...).
- **Registries** (`registries.py`) -- `ABILITY_REGISTRY` (144) + `PASSIVE_REGISTRY` (147)
  - `TRAIT_REGISTRY` (25, T.28+Multicaster) + `ITEM_REGISTRY` (50, T.29a-d)
  - `RUN_ACTION_REGISTRY` (5, T.29b) + `AUGMENT_REGISTRY` (54, T.31 -- populated by `game/augments.py`, alongside a `QUEST_TRACKER_REGISTRY` for quest-scoped augments). Content factories self-register via `@register_*` decorators; importing the content package triggers them. Lookups are by **string id**.
- **Presentation layer** (`registries.py` + `ability_text.py`, T.34/T.35) -- a parallel `ABILITY_META` (285 ids) gives every roster ability a tooltip. Numeric outlets flow through the **closed** `Magnitude` **family** (`ScalingTerm` linear / `PctResource` / `MaxOfTerm` / `SetByCaller`, GAS-modeled, V.46): the handler reads the number via `term.eval(...)` and `ability_text.render` renders the *same* object (source-of-truth B, V.38 -- tooltip can't drift from combat). Pure, no Flet (V.1). An AST guard (`test_no_orphan_stat_reads`) fails the build on any handler stat-read not backed by a `Magnitude`.
- **Description render-layer** (`game/describe.py` + `items/meta.py` + `traits/meta.py`, T.41) -- the same pattern for **champion-independent** content (items + traits): `RenderedEntry` / `RenderedTrait` (name + blurb + stat line) from a per-domain `*_META` (authored name/blurb, transcribed from the design catalogs; trait rung counts reconciled to code, V.79). The **stat line is derived from the live numbers** -- introspected from the item's `EffectBundle` (V.78) or from `traits/_packs.TRAIT_STAT_PACKS` (V.79), never re-typed, so it can't drift; no `Magnitude` machinery (these grant fixed %, not caster-scaled). Pure, no Flet, no mutation (V.80). Consumed by the Prep item chips + the `trait_synergies_panel` tooltips (Prep + Combat).

  V.15 / V.22 / V.17 / V.38 / V.46 / V.47 -- every id resolves + every scaler is visible. Any `ability_id`, `passive_id`, `trait` tag, or augment id referenced by content data must resolve in its registry; every ability id must have an `ABILITY_META`; every handler stat read must be a `Magnitude`; an `int/hybrid` unit must read INT in its kit. All CI-guarded.

The `CombatContext` (`context.py`) is the single mutation point: `ctx.deal_damage`, `ctx.apply_status`, `ctx.grant_barrier`, `ctx.heal`, `ctx.register_bundle`, etc. "Direct mutation architecture" -- no effect-as-data reducer.

2.3.4 Statuses, DOT, and barriers

- **Statuses** (`status.py`, `piece.py`) -- id-based, **one** `StatusInstance` **per** `status_id` **per piece** (V.26, non-stacking identity; intensity that should accumulate uses `StackBehaviour.STACK`, e.g. poison). Carry gates (stun -> can't act), DOT info, and stacks.
- **DOT cadence** (V.25) -- DOT damage + stack decay fire on a **per-status clock** (`dot_interval_ticks`, default 100t = 1s), *not* every engine tick. The clock free-runs (re-applying never resets the next tick). Stack decay is **percentage** (`decay_fraction`), giving an investment-scaling equilibrium with **no hard cap** -- see journal and the *no-hard-caps* balance philosophy.
- **Barriers** (V.28, `piece.py`) -- a temporary absorb pool **distinct from HP and from "shield"**. Soaked before HP inside `deal_damage`, FIFO, optional tick expiry, granted only via `ctx.grant_barrier`. "Shield" in content ids means an armor/resistance *buff*, a different mechanic -- do not conflate.

2.4 The two weather systems (T.2)

`src/game/weather_effects.py` -- **two decoupled systems**, never summed:

1. **Weather Favor** (`combat_modifier`) -- does the *node weather* suit my affinity? A 5-tier stat buff/debuff applied **once at combat init**, in the single application path `compile_loadout::_apply_weather_to_piece`.
2. **Affinity Clash** (`damage_modifier`) -- do *I* beat *this enemy*? A per-hit damage multiplier by attacker-affinity vs defender-affinity, resolved **per hit** in the loop (it depends on the defender, so it can't be pre-snapshotted).

Both run off the **single** `affinity: WeatherState` **field** every piece carries (V.6 -- there is no separate weakness field). `CLEAR` sits outside the predator/prey ring and is inert in both. The 6 weather states (V.5) map 1:1 to OpenWeather id groups.

2.5 Content layer -- champions, enemies, bosses, abilities

Concern	File
Champion / enemy rosters + base-stat template + level scaling	`src/game/content.py`
Champion ability/passive handlers (~60 champions)	`src/game/abilities/champions.py`
Enemy ability/passive handlers (~60 enemies)	`src/game/abilities/enemies.py`
Boss kits -- 6 two-phase bosses	`src/game/abilities/bosses.py`
Boss definitions, supporting cast, phase/death hooks	`src/game/bosses/data.py`
Boss **map effects** (decoupled board hazards)	`src/game/map_effects.py`, `src/game/board.py`
Ability tooltips -- `AbilityMeta` + `Magnitude` family + renderer (T.34/T.35)	`src/game/registries.py`, `src/game/ability_text.py`

Concern	File
Synergy traits (Kinship / Calling / Affinity)	`docs/design/content/trait_catalog.md` -> `game/traits/` (T.28 [done])
Items	`game/items/` (T.29a-d [done] -- components, combined, emblems, special run-actions, mana primitive, multi-slot)
Augments	`game/augments.py` (T.31 [done] -- model + ~50 catalog + offers/reroll; picked in-game at AUGMENT nodes via `ui/views/augment.py` + `economy.resolve_nonfight_node`, T.42a)

Content **vocabulary lives with content** (V.8): synergy tags are open-ended strings the engine treats as opaque labels. The roster has a history of drifting from the catalog docs (e.g. `CALLING_TAGS` once carried dead tags) -- always diff code vocabulary against `docs/design/content/*_catalog.md` and add a V-guard (see CLAUDE.md planning rules).

2.6 Power & scaling model (T.18)

`src/game/scaling.py` -- one formula governs all power:

```
P(T, L) = 2 ^ ((T-1)/3 + triplings(L))     triplings = {L1:0, L2:1, L3:3}
```

Stat multiplier is $\sqrt{\mathbf{P}}$ so that HP-DPS (~= combat value) grows *linearly* with P, keeping encounter budgets linear. Level-ups use a **tripling** mechanic (3 copies -> next level), giving an accelerating curve (L2 modest, L3 a spike). Total T1L1->T10L3 ~= **8x in stats**. Tier-10 "Primordials" are boss-only -- the buyable ceiling is T9.

2.7 Route & encounter generation

Concern	File
Fixed 50-node route, 6 stages, city coords + enemy pools	`src/game/route.py`
Seed-deterministic squads for FIGHT/REWARD/CHALLENGE/BOSS	`src/game/encounter.py`
Role-aware enemy board placement	`src/game/formation.py`

Determinism doctrine (V.19 + the T.19 contract): all "randomness" derives from `(run_seed, node_index, channel)` -- no clock, no global RNG. Same seed -> same route draws, same squads, same shop, same economy. This is what makes the simulation layer trustworthy. Enemy formation (`formation.py`) is fully deterministic and role-aware (tanks col 7, bruisers col 8, backline col 9, bosses authored positions).

2.8 Economy & shop (T.22)

Concern	File
Amber income, interest, Tempest progression; **node-result orchestrators** (`apply_node_result` for fights, `resolve_nonfight_node` for AUGMENT/SUPPLY, V.83) + CHALLENGE recruit	`src/game/economy.py`
Stage-gated tier rolls, buy / sell / reroll, SUPPLY free-recruit offer	`src/game/shop.py`
Item equip/unequip seam (`Run.inventory` <-> `Champion.items`, auto-combine on double-equip, V.63)	`src/game/inventory.py`
Augments -- `Augment`/`RunModifiers` model, ~54 catalog, offer/reroll, `apply_augment`	`src/game/augments.py`

Currency is **Amber**; team-size XP is **Tempest** (rank = deployable board cap, 1->10, **monotonic non-decreasing**, V.20). Shop offers are seed-deterministic `(run_seed, visit_index, reroll_count)` (V.19). These are the **headless economy substrate** the live Prep / Reward / Augment / Supply views drive: the view chooses, the game mutates (V.63), and the producer autosaves after (V.65). A non-fight node grants no income/Tempest and touches no Hearts -- its pick (augment or supply recruit) is applied by the view, then `resolve_nonfight_node` marks the node cleared + advances.

2.9 Data models & run state (T.1)

`src/game/models.py` -- all dataclasses (`slots=True`), JSON-serializable:

- `WeatherState`, `NodeType`, `NodeState`, `RunStatus`, `CombatOutcome` (enums)
- `Champion`, `Enemy` -- roster/source models (carry `affinity`, `traits`, stats, ability ids)
- `Node` -- one route stop
- `BattleEvent`, `BattleResult` -- combat output + event stream (the runtime combat entity is `Piece` in `piece.py`, built by the loadout compiler)
- `Run` -- the single object holding *all* game state (current node, roster, battle log, Amber, Tempest, augments). Per V: one `Run` is the whole game.

`src/game/save.py` (T.14) -- the **file-I/O layer** over that contract: `save_run` (atomic temp+`os.replace`), `load_run` (`schema_version` gate -> `Run.from_dict`), `default_save_dir`, `CURRENT_SCHEMA_VERSION`, and typed errors (`SaveError` / `CorruptSaveError` / `UnsupportedSchemaError`). No Flet import (V.1); the (de)serialization contract itself stays on the dataclasses. See docs/live/systems/save.md.

2.10 API layer (T.6 + T.7)

Concern	File
OpenWeather client -- fetch by lat/lon, parse -> `WeatherState`	`src/api/weather.py`
Stateless per-city cache (`unknown` / `live+fetched_at` / `substitute`)	`src/api/cache.py`

Concern	File
3-stream tick refresher (<=3 calls/min)	`src/api/refresher.py`

The refresher ticks 1/min and fires three deduped streams (A: full round-robin 50; B: window `[current+1..+6]`; C: uniform random) -> bounded API usage, staleness <= 50 min (V.11). On fetch failure a node falls back to the city's `default_weather` flagged `substitute` (V.3, V.13). Key via `OPENWEATHER_API_KEY`, **never logged** (V.3). All HTTP on a worker thread (V.4).

Persistence (T.39, V.73). The cache is the fetch-scheduling/freshness layer; the **persisted source of truth is the** `Run Node`: each node carries `weather` (the effective game weather -- `default_weather` placeholder until a live fetch overwrites it), `weather_state: NodeWeatherState` {`unknown`/`live`/`substitute`}, and `weather_locked`. The Trail writes fetched cache values onto the `Run` via the pure game-side mutators `Run.set_node_live_weather` / `Run.lock_node_weather` only (cache/refresher stay stateless re: game, V.10). The **current** node's weather **locks at the Trail->Prep transition** (frozen for the run; refresher skips locked nodes) -- so live weather reaches combat (Weather Favor, CHALLENGE rolls) while staying byte-identical across the fight + reward (load-bearing for V.70).

2.11 UI & visualization -- the full run loop is live

The player-facing Flet UI is built end-to-end: menu -> run-start -> the per-node loop (trail -> prep -> combat -> reward, plus augment / supply nodes) -> run summary.

Concern	File	Status
Design tokens (colors, type, spacing, animation)	`src/ui/theme.py`	[done] T.8
Shared components -- champion card, weather badge, meter bar, chips, **shared infocard** (Prep + Combat), **trait-synergy panel**, **hex board geometry**, **iconography glyphs**	`src/ui/components/`	[done] T.8/T.12d/T.23a
Pure Flet-free combat-playback model over the replay backend	`src/ui/combat_playback.py`	[done] T.12a
Flet entry point -- menu-rooted `page.views` shell + full run router	`src/main.py`	[done] T.9/T.42
Main menu (`/`) -- New Run / **Continue** (loads latest save into Trail, T.15b) / Playfight / Quit / Settings	`src/ui/views/menu.py`	[done] T.9
RunStart (`/run-start`) -- seed-deterministic 1-of-3 champion pick -> `Run`	`src/game/run_init.py`, `src/ui/views/run_start.py`	[done] T.10
Trail (`/trail`) -- route map + node focus + team summary + live weather	`src/ui/views/trail.py`	[done] T.11

Concern	File	Status
Prep (`/prep`) -- shop, hex-board placement, item equip, trait preview	`src/ui/views/prep.py`	[done] T.23
Combat view + dev/Playfight harness	`src/ui/views/combat.py`, `dev_harness.py`	[done] T.12
Reward (`/reward`) -- post-fight node-result panel + CHALLENGE recruit	`src/ui/views/reward.py`	[done] T.15a/T.38
Augment (`/augment`) -- 1-of-3 pick + reroll at AUGMENT nodes	`src/ui/views/augment.py`	[done] T.42a
Supply (`/supply`) -- 1-of-5 free-recruit at SUPPLY nodes	`src/ui/views/supply.py`	[done] T.42b
Run summary (`/summary`) -- run-end screen + damage chart	`src/ui/views/summary.py`	[done] T.13
Settings -- set OpenWeather API key in-app	`src/ui/views/settings.py`, `src/app_config.py`	[done]
Playtest admin panel (dev)	`src/ui/views/admin.py`	dev tool
Route map + run-summary chart + affinity-clash heatmap -- all **hand-drawn** `flet.canvas` (V.72)	`src/viz/route_map.py`, `run_summary.py`, `affinity_clash_heatmap.py`	[done] T.11/T.13

`main.py` is the menu-rooted `page.views` shell and the run router. **New Run** opens RunStart (`game/run_init.new_run`) -> champion pick -> `Run` -> Trail. **Continue** loads the most-recent save into the Trail (T.15b; a corrupt save is skipped). From the Trail, `_play_node` dispatches by `node.node_type`: fight nodes go Prep -> Combat -> Reward -> back to Trail; non-fight nodes go to the Augment or Supply view then `resolve_nonfight_node` -> Trail. **Playfight** opens the combat dev harness -> combat view. `TEMPEST_ADMIN=1` opens the admin panel and `TEMPEST_DEV=1` jumps straight to Playfight.

The **combat view is pure presentation over the replay backend** (V.56): it drives the Flet-free `combat_playback` model and the `CombatReplay` forward-stepper -- it records nothing and re-derives no formation, laying out from the recorder's `initial_pieces` snapshot. See SPEC §I Flet Routes and `views_spec.md`; Flet conventions live in .claude/rules/flet-ui.md and CLAUDE.md.

2.12 Dev tooling -- how to drive the game without a UI

Because `game/` is pure, two toolkits exercise it directly:

2.12.1 Playtest CLI (`tools/playtest/`, T.27) -- interactive inspection

```
uv run python -m tools.playtest.sim_fight --help     # one fight
uv run python -m tools.playtest.sim_node ...          # a node encounter
uv run python -m tools.playtest.sim_run ...           # a full run
uv run python -m tools.playtest.inspect ...           # roster inspection
uv run python -m tools.playtest.inspect_node ...
```

`_common.py` holds shared helpers; `resolve_boss_combat` now delegates to the src-side `combat/resolve.py` entry (V.59).

2.12.2 Power simulation (`tools/simulation/`, T.25) -- balance benchmarking

- `matchup.py` -- `run_matchup`, the pure unit of work (safe in worker processes)
- `tournament.py` -- battle generators (1v1, team2, sampled teams)
- `runner.py` -- CLI entry (`python -m tools.simulation.runner --mode 1v1 ...`)
- `mega.py` -- runs every sweep flavour in one parallelized go
- `ratings.py` -- win-rate + power-model metrics, incl. cross-weather `weather_metrics`
- `report.py` -- CSV / console output
- `stat_edge.py`, `weather_impact.py` -- focused single-axis sweeps (stat lead, weather swing)

Root-level dev scripts: `tools/export_roster.py` (dump the roster) and `tools/gen_role_matrix.py` (role-coverage matrix).

> **Sim metric invariants:** weather-affinity metrics are **cross-weather** (V.16) and **treat absent weather as NaN, never 0** (V.30) -- averaging missing-as-0 once fabricated a balance signal (B.12). Sim outputs land in `results/` (gitignored); published analyses live in `reviews/`.

2.13 Determinism doctrine (read this before touching combat or content)

Determinism is non-negotiable (V.2, V.14, V.19). The simulation and replay systems depend on it:

- **No RNG in mechanics.** Any "chance" / "every few hits" effect uses a **deterministic cadence counter** (like `crit_counter`), never `random`.
- **All procedural generation** derives from `(run_seed, node_index, channel)`.
- `resolve_combat` **is pure** -- same inputs, byte-identical `BattleResult`. Its T.31 `run_mods` arg defaults to `None`, leaving every existing caller (and every sim) byte-for-byte unchanged (V.2).
- **State for a view is recomputed, not recorded (V.55).** The `CombatReplay` forward stepper (playback) and `inspect_at_tick` (random seek) drive the same single `_step_combat` generator to read any piece's live state at any tick -- no per-tick keyframes in `BattleResult`. Same purity contract: they clone `run_mods` so the replay can't mutate the caller's quest state. This live state is the combat view's resource truth, **not** the event stream's partial `hp_after` (V.56/V.57, B.28).
- **Verify with:** fixed seed + `workers=1` -> identical output across runs.

2.14 "Where do I find X?" -- quick index

I want to...	Go to
Change how a fight resolves	`src/game/combat/engine.py`

I want to...	Go to
Add/modify what content can do to the world	`src/game/combat/context.py` (mutator API)
Add a champion/enemy ability	`src/game/abilities/{champions,enemies}.py` + register
Add a boss	`src/game/bosses/data.py` + `src/game/abilities/bosses.py` + `src/game/map_effects.py`
Tune stats / scaling	`src/game/content.py`, `src/game/scaling.py`
Change weather effects	`src/game/weather_effects.py`
Edit the route / cities	`src/game/route.py`
Change enemy generation	`src/game/encounter.py`, `src/game/formation.py`
Touch the economy / shop	`src/game/economy.py`, `src/game/shop.py`
Add / change an augment	`src/game/augments.py` (+ register), `src/ui/views/augment.py`
Add / change an item; equip logic	`src/game/items/`, `src/game/inventory.py`
Add a data field to game state	`src/game/models.py` (`Run`, `Champion`, ...)
Change save/load	`src/game/save.py` (run) - `src/app_config.py` (settings)
Change weather fetching / caching	`src/api/{weather,cache,refresher}.py`
Build / edit a UI screen	`src/ui/views/` (+ `theme.py`, `components/`)
Change the run flow / routing	`src/main.py` (menu shell + `_play_node` dispatch)
Balance-test a change	`tools/simulation/`
Manually try a fight/run	`tools/playtest/`
Understand *why* a decision was made	`docs/journal/`
Know what rule must hold	SPEC.md §V

2.15 End-to-end: what happens in one fight

1. Player advances to a node; `encounter.py` generates the enemy squad from the seed; `formation.py` places them; the node's weather is read from the persisted `Node` (`node.weather` -- live values written through from the cache while on the Trail, then **frozen by the Prep-entry lock**, T.39/V.73).
2. `resolve_combat(team, enemies, weather, run_mods=...)` is called (`run_mods` carries the run's active augments + quest state; `None` for sims/Playfight).
3. `compile_loadout` builds runtime `Pieces` (uniquifying duplicate ids, B.65), applies **Weather Favor** to base stats, resolves **synergy traits**, folds in augments, subscribes passive `Hooks` to a fresh `EventBus`, wires boss phase/death hooks.
4. `CombatContext` wraps the pieces + bus + board; bosses `attach_map_effect`.
5. `engine.run(ctx)` ticks: meters fill -> pieces move (BFS pathing) / auto-attack / cast abilities; every damage instance applies **Affinity Clash**; statuses tick on their cadence; barriers soak; map effects fire; sudden-death timeout guards stalls. Every action emits a typed event onto the bus.
6. `BattleResultRecorder` (subscribed to the bus) reconstructs a `BattleResult` -- outcome, survivors, the full beat stream (move/attack/cast/death + heal/dot/ status/spawn/despawn,

each beat one event with `hp_after`/`barrier_after` for HP-changing ones, T.37a) and an `initial_pieces` board snapshot + board dims so a combat view can lay out and animate the fight without re-deriving formation.

7. `economy.apply_node_result` folds the outcome into the `Run` -- battle log, Amber / Tempest income, Hearts on a loss -- the Reward view shows the panel (plus any CHALLENGE recruit), the producer autosaves (V.65), and the Trail reopens on the next node. (A non-fight node instead runs its Augment/Supply pick -> `resolve_nonfight_node`.)

2.16 Reading order for a newcomer

1. **This file** -- the map.
2. SPEC.md §G (goal), §V (invariants) -- the rules.
3. `src/game/models.py` -- the vocabulary.
4. `src/game/combat/engine.py` + `context.py` -- the engine.
5. `src/game/abilities/champions.py` -- see content plug into the framework.
6. Run `tools/playtest/sim_fight` -- watch a fight resolve.
7. docs/journal/ -- the "why" behind the non-obvious choices.

For per-system design depth, see the Documentation Map in CLAUDE.md and `docs/design/`. For the status of any task, see SPEC.md §T.

Chapter 3

Systems & Features

This chapter is the mid-level reference layer: how each subsystem and content area works *now*, and where it lives in the code. It is assembled from the project's **LIVING** documentation (`docs/live/`) -- docs that are reconciled to the code on every change that touches their subject and audited by the `/check` drift tool. Where the previous chapter mapped *what exists and how it interacts*, this one goes one level deeper into each system in turn.

The sections below cover, in order, the combat engine and its resolution pipeline, the effect/hook/status framework that all content plugs into, the two weather systems, enemy formation, encounter and board generation, stat scaling, the weather API layer, save/serialization, items, and the kit-authoring conventions -- followed by the content references (rosters, abilities, traits, augments, items) whose real data lives in code.

3.1 Combat -- engine & resolution

> **Status:** **LIVING** -- must match `src/game/combat/`, `src/game/loadout.py`, `src/game/piece.py`. Audited by `/check`. **Scope:** how one battle resolves end-to-end: entry -> compile -> tick loop -> result. **Reconciled:** 2026-06-05 @ refactor/combat-engine-single-source.
>
> Citations are by **symbol** (file + `function`), not line number, on purpose -- lines drift, symbols don't. Design rationale lives in the frozen `docs/design/systems/combat_system_proposal.md` + `docs/journal/`.

3.1.1 Entry point (the only one)

`resolve_combat(team, enemies, weather, *, node_id="", run_mods=None, positions=None) -> BattleResult` in `combat/resolve.py` is the **single public combat entry** (V.2). It is pure and deterministic: identical inputs -> byte-identical `BattleResult`. It delegates the setup to the shared `build_combat(...)` helper (the **one** wiring path) then runs the loop:

```
ctx, recorder = build_combat(team, enemies, weather, run_mods, node_id, seed)  # resolve.py
#   = compile_loadout (pieces, bus) -> assign_spawns -> recorder.register -> CombatContext
winner = run(ctx, recorder)                                                    # combat/engine.py
return recorder.build_result(winner)                                          # combat/recorder.py
```

`build_combat` is reused (no parallel setup, T.37b) by:

- `resolve_boss_combat` (`combat/resolve.py`, T.12b/V.59 -- the **single src-side boss entry**; `tools/playtest/_common` delegates to it) -- same `build_combat`, then `attach_map_effect(map_effect_id, ctx, seed)` before `run`. Takes a `map_effect_id: str` (never a `bosses/` content type => `combat/` stays content-import-free); byte-identical to the former `tools/` version (V.2). `CombatReplay`/`inspect_at_tick` accept the same `map_effect_id` to replay a boss fight; `CombatReplay.board_cells()` exposes the map-effect tiles as `(q,r,kind)` for the view overlay (V.1).
- `CombatReplay` / `inspect_at_tick` (`combat/replay.py`) -- `build_combat(..., with_recorder=False)`, then steps the `_step_combat` generator to read live state (see Replay).

3.1.2 The pieces

- `compile_loadout` (`loadout.py`) -- builds runtime `Pieces` from `Champion`/`Enemy` models, **uniquifies duplicate piece ids** (B.65 -- the 2nd+ occurrence of a def id gets a deterministic `#n` suffix, e.g. `wolf`, `wolf#1`, `wolf#2`; done in assembly order *before* any hook closes over the piece, so the fight is object-identity-based and unaffected while the recorded stream / per-id tallies / view control keys stay collision-free), applies **Weather Favor** to `base_stats` (see weather.md -- the single application path), applies item/trait/augment bundles, subscribes passive `Hooks` to a fresh `EventBus`, wires boss phase/death hooks, then seeds per-slot start mana. Returns `(pieces, bus)`.
- `Piece` (`piece.py`) -- the live combat entity (the *only* combat model; there is no separate snapshot type). Carries `base_stats`, modifiers, statuses, ability slots, position, meters, `crit_counter`. Effective stats via `piece.stat(name)` = `compute_stat` over base + modifiers.
- `CombatContext` (`combat/context.py`) -- the mutator API: the *only* way content touches the world (deal_damage, apply_status, cast_ability, ...). Holds the board state and `hex_distance`.
- `engine.run` (`combat/engine.py`) -- the tick loop (below). The **sole** tick engine (V.29); there is no second `run`.
- `BattleResultRecorder` (`combat/recorder.py`) -- subscribes to bus events and reconstructs `BattleResult` (outcome, survivors, damage, event stream, `piece_max_hp`, plus the `initial_pieces` board snapshot + board dims, T.37a). Observer-only: it never feeds combat math, so sims stay byte-identical (V.54).

3.1.3 Tick model

Time = **10 ms ticks** (`TICK_MS`); 1 round = 600 ticks (`ROUND_TICKS`, a presentation unit only). Each living piece accrues three meters per tick from its int stats: `action_energy += attack_speed`, `movement_energy += move_speed`, and mana via the **weighted-rank charge cycle** (T.29c, V.48): one slot is charged the full `mana_regen` per tick, cycle length `sum(slot.priority)`, each slot occupying `priority` positions (skip slots at `max_mana`) -> mana throughput = `mana_regen`/tick regardless of slot count. A meter fires at `ENERGY_THRESHOLD` (60 000) and **carries overflow** (it subtracts the threshold, not resets) so cadence is exact. Mana is per-slot: `mana_cost`/`max_mana` (default `2xcost`)/ `start_mana`/`priority` are authored on the ability def (`ABILITY_MANA`), not a piece stat; a cast deducts `-= mana_cost` so overflow banks toward the next.

Multi-slot pieces (T.29d, V.49): a piece carries `active_abilities: list[str]` -- one `ActiveSlot`

per id (single/null/multi are just list lengths; empty = a stat-stick with no mana bar). Roster ids are **discovered by convention** (`content.discover_abilities`: `{id}.active`, `{id}.active2`, ... sorted) unless a def sets `abilities=` explicitly (bosses' named kits, or `[]`). A multicaster's slots must differ in cost **or** priority (no lockstep simul-cast); the default is same cost + unique priorities (primary dominant), with high-tier **Ultimate** secondaries diverging by cost (2x) + priority proportional to cost.

Status durations are authored with `secs(x)` (seconds -> ticks, fractions OK, e.g. `secs(3.5)`) -- `SECS` (=100) for raw tick intervals. The stored value is always real ticks (no hidden runtime scaling; tick<->time stays honest). CC/DoT durations were re-tuned ~2x (1.5-3 s -> ~3-6 s) so effects don't expire between the slow ~5 s action cadence (`60000/attack_speed`).

- **Ordering** -- within a tick, triggered meters resolve in the canonical side-independent total order `_event_sort_key = (-round(attack_speedx1000), champion_id, load_order, kind)` (V.34, T.29-pre): faster attack-speed first (the quantized float key carries both whole + sub-integer speed in one term -- no separate `milli_AS`), then identity, then the seeded `load_order` (never team-then-enemy -> no side-A bias, B.14), then movement before action. Cadence reads `int(attack_speed)`. No RNG in the loop; `load_order` is a one-time seeded permutation (V.2/V.14). **Soft-CC:** `slow` stacks throttle action + movement meter *gain* by `_slow_factor` = `max(0.40, 1 - 0.15-stacks)` (V.53, B.25) -- not a gate, not the `stat`.
- **Movement** -- `_resolve_movement`: hold at threshold if in range or no enemies; else one BFS step toward the nearest in-range cell (`_next_step_toward`), carrying overflow; hold if no path. Gated by `BLOCKS_MOVEMENT` statuses.
- **Action** -- `_resolve_action`: cast an *unregistered* ability if mana full (fallback path), else auto-attack if an enemy is in range (`ctx.trigger_basic_attack`), else idle-hold. Registered abilities cast separately via `process_casts` -> `ctx.cast_ability` -- **at most one cast per window** (V.48 T4): among ready slots the highest `priority` casts (tie -> lowest slot index), the rest stay ready for later windows. Gated by `BLOCKS_ACTION`/`BLOCKS_ATTACK`/`BLOCKS_CAST`.
- **Per-tick upkeep** -- `_process_board_state` (slow tiles), `process_statuses` (DOT + decay on each status's cadence, then expiry), `expire_modifiers`, summon despawn.
- **Termination** -- ends when one side has no living piece. **Sudden death**: at `SUDDEN_DEATH_TICK_START` (= `MAX_TICKS`, 12 000) the loop applies a stacking DOT (+3 stacks/engine-tick) that **damages once per second** (`dot_interval_ticks=100`, like every DOT -- V.25), so it escalates but still leaves room for a few last actions; `HARD_CAP_TICKS` (= MAX_TICKS + 2 000) is the absolute ceiling. **Outcome is survivor-based (V.60):** WIN = team survivors & no enemy, LOSS = enemy survivors & no team, DRAW = **no survivors either side** (a true mutual wipe -- only via a simultaneous DOT/sudden-death pass). `timed_out` is an **independent flag**, not an outcome -- a sudden-death fight with a survivor is a real WIN/LOSS.

3.1.4 Damage pipeline

`_apply_hit` / `ctx.deal_damage` order:

```
raw = str_coeff-STR + int_coeff-INT                              # auto-attack path
# unregistered-ability fallback instead: raw = (ABILITY_STR_COEFF + ABILITY_INT_COEFF)-
max(STR,INT)
raw x= damage_modifier(attacker.affinity, target.affinity)   # Affinity Clash, weather.md
if crit:  raw x= CRIT_MULTIPLIER (1.5)                        # deterministic cadence
```

```
mitigated = raw x (1 - mit/(mit+100)) after penetration       # _mitigated_damage
final = max(1, round(mitigated))                             # true damage skips mitigation
```

- Coeffs (`combat/engine.py`): auto = `AUTO_STR_COEFF` 1.0-STR + `AUTO_INT_COEFF` 0.2-INT. Unregistered-ability fallback sums the two ability coeffs and applies them to the **primary** stat: `(ABILITY_STR_COEFF 0.2 + ABILITY_INT_COEFF 4.2)-max(STR,INT)` = 4.4-max(STR,INT) (T.30 §4 de-bias; `engine.py:496`), not a separate STR/INT split.
- **Crit is not random** -- `crit_counter` on the `Piece` increments per eligible hit and crits every `round(1/crit_chance)`-th, then resets. Shared autos/casts.
- **Mitigation** -- `magical`->resistance, `physical`->armor, `true`->unmitigated; `_effective_mitigation` applies `penetration_pct` then flat `penetration`, clamped >= 0. `damage_type` is a **closed vocabulary** `{physical, magical, true}` that `deal_damage` **validates** (raises on anything else, V.58) -- a typo can't silently fall into the armor branch (B.29).

3.1.5 Result

`BattleResultRecorder.build_result` emits `BattleResult` with: outcome (WIN/LOSS/DRAW), `rounds`/`turns`/`duration_ticks`, `team_damage_dealt`/`_taken`, survivor id lists, `events` (full tick-ordered `BattleEvent` stream), `piece_max_hp` (`{id: int(max_hp)}` captured from the engine's pieces), and the combat-view layout (`initial_pieces`: per-piece `PieceSnapshot` identity + spawn-time position + mana profile, with `spawn_tick=0` for starters and the spawn tick for mid-combat summons; `board_width`/`board_height`).

Beat taxonomy (V.54, T.37a; `ability` T.37 follow-up). Every visible-state- changing beat emits exactly one `BattleEvent` from a single producer path: `move`/`attack`/`cast`/`ability`/`death` plus `heal`/`dot`/`status`(applied)/ `status_expire`/`spawn`/`despawn`. `cast` **vs** `ability` **are distinct:** `cast` is the *activation* marker (`amount=0`, "a piece casts", `_on_cast`); `ability` is the resulting *damage* (one per target hit, `_on_damage_dealt` when `tag == ABILITY`) -- first a `cast`, then per-target `ability` beats. Beats that actually change HP carry `hp_after`/`barrier_after` = the engine's post-event truth (read after `deal_damage` applies, V.28-correct: the `amount` is the full pre-barrier figure for DPS accounting): on `attack`/`ability`/`dot`/`heal`. The `ability`/`attack` `amount` is the **final post-mitigation** figure with `is_crit`

- `damage_type` (`physical`/`magical`/`true`, on `DamageEvent`) -- the single `ctx.deal_damage` choke-point is the one producer (no separate ability handler). With `ability` added the stream is HP-complete; the view still reads bars from the live stepper (V.57) -- the beat's `amount`/`type` drives the floating *number*, not the bar. `turns` counts `attack`+`cast` only (`ability` excluded) => byte-identical. The `status` **(apply) beat fires once per *acquisition*** -- the recorder tracks a `(piece_id, status_id)` active set (cleared on `status_expire`) and skips re-applies/ refreshes (V.54), so sudden-death (re-applied every tick) + poison-restacks don't spam the stream (live stacks come from the stepper anyway, V.57). `expire_summon` fires `on_despawn` (distinct from `death`). The recorder is observer-only => sims byte-identical; only `combat_log` golden text re-baselines. `record_attack` (a dead parallel path) was removed -- `_on_attack_landed` is the sole attack producer.

Targeting footprints (T.12c, V.61) -- observer-only shape telemetry. Alongside beats, the recorder collects a `footprints: list[Footprint]` for the combat view's per-ability-shape VFX. The targeting helpers record the geometry they already compute: `enemies_in_radius`/`allies_in_radius`/`neighbors_of` -> a `circle` (center + radius), `line_targets` -> a `line` (origin + direction + length), via `ctx.note_footprint(kind, q, r, ...)` -> `on_footprint`

-> _on_footprint. Capture fires **only while a cast is in flight** (note_footprint gates on _current_cast_id), so idle/AI/passive target queries don't record. It is **observer-only**: the helper returns its target list unchanged and, on the sim / CombatReplay path (no recorder subscribed), the bus fire no-ops => **byte-identical** (V.2/V.14, extends V.54). The cast + ability beats now carry the engine's cast_id so the view joins a footprint to its ability for element colour.

3.1.6 Rendering

combat_log.py turns a BattleResult into text purely from the result (it does **not** recompute anything): the HP trace prefers each event's hp_after (barrier/DOT/heal-correct, T.37a) and falls back to piece_max_hp + damage subtraction for legacy events. Used by the playtest CLIs and golden-snapshot tests.

3.1.7 Replay / forward stepper / inspect-at-tick (T.37b, T.37c)

Continuous combat state for a view is **recomputed, not recorded** (V.55). The engine loop is the **single** _step_combat generator (combat/engine.py, V.29), driven two ways -- no parallel loop:

- run(ctx, recorder=None) *drains* it to completion (the resolve path) -- byte-identical to the pre-T.37c monolithic loop (V.2/V.14).
- CombatReplay(team, enemies, weather, *, run_mods=None) *steps* it **forward** (.step_to(tick), .pieces() -> list[PieceView], .tick, .winner) for sequential playback -- drives one instance once, O(total ticks). Forward-only; step_to to an earlier tick raises.

The generator yields once per fully-processed tick (yield 0 = after on_combat_start; yield N = after ticks 1..N), so a stepping consumer pauses mid-fight **before** the post-loop finalize (end_combat/on_combat_end never mutate the inspected state).

inspect_at_tick(team, enemies, weather, *, run_mods=None, tick) -> list[PieceView] is a random/backward-access wrapper (re-run from 0) **over** CombatReplay -- one driver, two shapes. Each reads hp, barriers, per-slot mana, **effective stats** via piece.stat() (STR/AS ramp included), statuses, position into frozen read-only PieceView/SlotView/StatusView structs. run_mods is **cloned** (deep-copy of mutable augment_state) so replay never mutates the caller. Raw Piece/Flet never escape src/game/ (V.1). No per-tick state is stored in BattleResult (avoids T.14 save bloat + stat-drift).

This live state is the combat view's resource-truth source (V.56/V.57), NOT the recorded event stream -- even though the stream is now HP-complete (the ability beat closed the B.28 gap), the view keeps **one** source of truth (the stepper) so bars can't drift from a dual pipeline; the stream's amount/type drives the floating *number*, the stepper drives the *bar*. The stream is **animation cues + action-queue projection** only. move/ spawn beats carry structured dest_q/dest_r int coords (T.37c), not a parsed note string.

3.1.8 Invariants this system owns

- **V.2** -- combat is a pure, seeded function; the ability/passive/status framework is invoked *through* resolve_combat, never alongside it.
- **V.29** -- exactly one tick loop (combat/engine.py); no parallel engine.
- **V.14** -- determinism: every "chance" mechanic uses a cadence counter (crit_counter), never RNG.

- **V.54** -- event-stream completeness: every visible-state-changing beat emits exactly one `BattleEvent` (T.37a).
- **V.55** -- view state is recomputed by replay, never recorded as per-tick keyframes; the forward `CombatReplay` stepper + `inspect_at_tick` are pure + clone `run_mods` (T.37b, forward stepper T.37c).
- **V.56/V.57** -- the combat view's resource truth (hp/mana/stats/position) is the live replay, **never** the event stream's partial `hp_after` fields (incomplete for ability burst, B.28); the stream is animation cues + queue projection (T.12).
- **V.58** -- `damage_type` is a closed vocabulary `{physical, magical, true}`; `deal_damage` validates + raises on anything else so a typo can't be mis-mitigated as physical (B.29).
- **V.61** -- targeting footprints are observer-only telemetry for the combat view: helpers record geometry via `ctx.note_footprint` -> `on_footprint` only while a cast is in flight; the recorder stamps `footprint` records on `BattleResult`; capture never changes targeting results or damage => byte-identical (T.12c).

3.1.9 File map

Concern	File
Public entry + shared `build_combat` wiring	`combat/resolve.py`
Single tick loop (`_step_combat` generator) + `run` drain, pathing, damage, spawns, constants	`combat/engine.py`
Forward stepper `CombatReplay` + `inspect_at_tick` (recompute state at a tick)	`combat/replay.py`
Mutator API (the only way to touch the world)	`combat/context.py`
Event -> `BattleResult` (incl. footprint telemetry)	`combat/recorder.py`
Targeting helpers + footprint capture	`targeting.py` (`ctx.note_footprint` in `combat/context.py`)
Model -> `Piece` compile + weather + passives	`loadout.py`
Runtime piece	`piece.py`
Boss wiring (map effect)	`combat/resolve.py::resolve_boss_combat` (src-side entry, V.59; `tools/playtest/_common.py` shim delegates to it)

3.2 Effects -- hooks, modifiers, statuses, registries

Status: LIVING -- must match `src/game/effects.py`, `events.py`, `status.py`, `registries.py`, `piece.py`, and `combat/context.py`. Audited by `/check`. **Scope:** the T.20 effect substrate -- how content (abilities/passives/traits) changes combat without touching the loop. **Reconciled:** 2026-06-05.

Citations by symbol, not line. Design rationale (frozen): `docs/design/systems/effect_systems_design.md`, `passive_system_proposal.md`. Catalog: see abilities.md, traits.md.

3.2.1 The shape

Content never calls the tick loop. It registers **hooks** on an `EventBus`; the loop fires events; hooks mutate the world **only** through the `CombatContext` API. Three data primitives carry the change:

- `Modifier` (`effects.py`, frozen) -- a stat delta `(stat, op, value)`, `op in {add, mul, set}`, applied by `compute_stat` as `(base + Σadd) - Πmul`, `set` overriding. Carries a `Lifetime` (`PERMANENT`/`COMBAT`/`TIMED`); `TIMED` uses `expires_at_tick`. Lives in `Piece.modifiers`; read via `piece.stat()`.
- `StatusInstance` (`status.py`) -- a stacking, expiring effect on `Piece.statuses`, defined by a `StatusDef` in `STATUS_DEFS` (DOT cadence, decay, `StackBehaviour`, gates). Tick upkeep is `engine.process_statuses`.
- `EffectBundle` (`effects.py`) -- a named group of hooks + modifiers a passive/item installs at once; `ctx.register_bundle(owner, bundle)`.

3.2.2 EventBus

`EventBus` (`effects.py`) is a synchronous, priority-ordered dispatcher:

- `subscribe(hook) -> hook_id` -- appends the `Hook`, then re-sorts that event's list by `-priority` (higher priority fires first; stable within a tie).
- `fire(name, event, *, cast_id=None, ctx=None)` -- notification dispatch. Every handler is called as `handler(ctx, event)`; return values are ignored.
- `fire_reducing(name, event, value, *, cast_id=None, ctx=None) -> float` -- the value-mutating chain (damage pre-mods). Each handler is `handler(ctx, event, value)` and **returns the new value**; a `None` return leaves `value` unchanged. Only `on_damage_pre` uses this path.
- `unsubscribe(hook_id)`, `reset_combat()` (clears the `ONCE_PER_COMBAT` ledger at combat start), `clear_cast(cast_id)` (drops a finished cast's per-cast / per-target ledger entries).

A `Hook` (`effects.py`, dataclass) has an `event` name, a `handler` callable, a `priority` int (default 0; the recorder subscribes at `-1000` so it observes after all game logic), a `HookScope` (default `PER_HIT`), and a `hook_id` auto-assigned at `subscribe` time (`hook_0`, `hook_1`, ...).

3.2.2.1 HookScope -- dedup within a cast/combat

`_should_dedup` skips a hook's invocation when its scope has already fired for the relevant key (tracked in `_dedup_ledger`):

Scope	Fires...	Ledger key
`PER_HIT` (default)	every time -- never deduped	--
`ONCE_PER_CAST`	once per `cast_id` (no-op when `cast_id is None`)	`("cast", cast_id, hook_id)`
`ONCE_PER_TARGET`	once per `(cast_id, target)` -- reads `event.target.id`	`("target", cast_id, hook_id, target_id)`
`ONCE_PER_COMBAT`	once per combat, cleared by `reset_combat()`	`("combat", hook_id)`

`cast_id` is threaded from `ctx.current_cast_id` at fire time; `ONCE_PER_CAST` / `ONCE_PER_TARGET` degrade to `PER_HIT` when there is no cast in flight.

3.2.2.2 Events fired by the engine/context

Payloads live in `events.py` (plain `@dataclass(slots=True)`, no methods). Verified against the `bus.fire/fire_reducing` call sites:

`on_combat_start`, `on_tick`, `on_attack_start`, `on_attack_landed`, `on_damage_pre` (reducing), `on_damage_dealt`, `on_damage_taken`, `on_ability_damage`, `on_cast`, `on_cast_complete`, `on_heal`, `on_status_applied`, `on_status_expired`, `on_kill`, `on_death`, `on_spawn`, `on_despawn`, `on_phase_change`, `on_footprint`, `on_combat_end`.

`ManaEvent` (`on_mana_full`) is defined in `events.py` but is **not** currently fired by the engine -- mana readiness is polled in the tick loop, not published.

3.2.2.3 Event payloads -- which event carries which fields

Filtering hooks read `event.<field>`, so the exact field set matters. Note the asymmetry between the two damage-adjacent events:

Event(s)	Payload	Key fields
`on_attack_start` / `on_attack_landed`	`AttackEvent`	`attacker`, `target`, `amount` -- **no** `tag`, **no** `damage_type`, **no crit**
`on_damage_pre` / `_dealt` / `_taken` / `_ability_damage`	`DamageEvent`	`attacker`, `target`, `amount`, `tag` (`SourceTag` value), `cast_id`, `hit_id`, `is_crit`, `damage_type` (`physical`/`magical`/`true`), `is_dot`
`on_cast` / `on_cast_complete`	`CastEvent`	`caster`, `ability_id`, `cast_id`, `slot_idx`, `mana_cost`, `mana_after`
`on_heal`	`HealEvent`	`source`, `target`, `amount`
`on_status_applied` / `on_status_expired`	`StatusEvent`	`target`, `status_id`, `duration_ticks`, `stacks`
`on_kill`	`KillEvent`	`killer`, `victim`
`on_death`	`DeathEvent`	`victim`, `killer` (may be `None`)
`on_spawn`	`SpawnEvent`	`piece`, `position`
`on_despawn`	`DespawnEvent`	`piece` (summon expiry -- **not** a death)
`on_phase_change`	`PhaseEvent`	`piece`, `new_phase`
`on_footprint`	`FootprintEvent`	`cast_id`, `kind` (`circle`/`line`), `center_q`, `center_r`, `radius`, `direction`, `length`
`on_tick`	`TickEvent`	`tick`
`on_combat_start` / `on_combat_end`	`CombatStartEvent` / `CombatEndEvent`	(start: empty) / `winner`

To react to auto-attacks *with* damage attribution, subscribe to `on_damage_dealt` and filter on `event.tag == SourceTag.BASIC_ATTACK` -- the `AttackEvent` alone can't tell you damage type or whether it crit.

3.2.3 StatusGate -- how statuses disable actions

`StatusGate` (`status.py`) values, checked by `piece.is_gated(gate)` in the loop:

Gate	Blocks
`BLOCKS_ACTION`	all meter advancement (stun, frozen, fear)
`BLOCKS_CAST`	ability casts (silence)
`BLOCKS_ATTACK`	auto-attacks (disarm)
`BLOCKS_MOVEMENT`	hex movement (root, frozen)
`HEXPROOF`	not a meter gate -- excludes the bearer from single-target acquisition (autos + targeted abilities); AoE still lands, and the piece can still act (T.28d, V.40)

`STATUS_DEFS` (`status.py`) currently registers: `stun`, `silence`, `disarm`, `root`, `burn`, `poison`, `slow`, `charged`, `focus_fire`, `grievous`, `hexproof`, `taunt`, `soaked`, `frozen`, `fear`, `sudden_death`, `grief`, `stone_charge`, `soul_charged`, `nerei_grudge`. Pure markers (`focus_fire`, `stone_charge`, `soul_charged`, `nerei_grudge`, `taunt`, ...) carry no gate/DOT -- a kit's hooks read their presence/`stacks`/`source_id` directly.

A `StatusDef` is frozen data: `stack_behaviour` (`REFRESH` resets duration on reapply / `STACK` accumulates), `gates`, and the DOT clock -- `dot_per_tick` (damage per **DOT tick**, not per engine tick), `dot_interval_ticks` (default 100 = 1 s, V.25), `dot_scales_with_stacks`, `decay_stacks_per_dot` / `decay_fraction` (poison's percentage shed -> self-limiting plateau, no hard cap), and `dot_true_damage` (bypasses mitigation, e.g. `sudden_death`). A live `StatusInstance` adds `remaining_ticks`, `stacks`, `source_id`, `potency` (a per-DOT-tick damage override; 0 -> fall back to the def's `dot_per_tick`), and `ticks_to_next_dot` (the free-running DOT clock). `ctx.apply_status` honours `piece.cc_immune` (drops gating statuses) and seeds the DOT clock.

3.2.4 CombatContext -- the mutator API

The **only** way content touches the world (`combat/context.py`).

- Damage / heal / shield: `deal_damage(attacker, target, amount, tag, *, crit=None, damage_type="magical", is_dot=False) -> float` (returns the final post-mitigation amount; `crit=None` defers to the deterministic `crit_counter` cadence; `is_dot` is presentation-only -- routes the recorder to a `dot` beat), `heal` (honours the `grievous` antiheal marker), `grant_barrier`.
- Statuses / mods: `apply_status`, `remove_status`, `apply_modifier`, `register_bundle`.
- Actions: `trigger_basic_attack(attacker, target, mult=1.0)`, `cast_ability(actor, slot_idx=0)`, `gain_mana`, `teleport`, `spawn`, `expire_summon`, `kill`, `revive(target, hp_frac=0.3)`, `end_combat`.
- Queries: `enemies_of`/`allies_of`, `all_pieces`/`living_pieces`, `both_sides_alive`, `current_tick`, `current_cast_id`, `weather`, `bus`, `board_state`, `rng` (seeded -- for non-combat-affecting choices only).
- Footprint telemetry: `note_footprint(kind, q, r, ...)` -- observer-only shape capture for the combat view, no-op outside a cast (see combat.md).

`SourceTag` (`basic_attack` / `ability` / `item_proc` / `dot` / `status` / `reflect` / `true`) tags every damage instance so hooks can filter what they react to.

compute_stat (effects.py) folds base + Σadd then xΠmul, with set overriding, then clamps via _STAT_FLOORS (currently attack_range >= 1, V.43). stat_breakdown(piece) telescopes the same fold by source: prefix (item/augment/passive/trait/weather, in _SOURCE_ORDER) for the prep view's per-source stat attribution -- pure, no Flet (V.1/V.45).

3.2.5 Registries -- id -> handler

registries.py holds ABILITY_REGISTRY, PASSIVE_REGISTRY, ITEM_REGISTRY, TRAIT_REGISTRY, AUGMENT_REGISTRY. Content registers via @register_active, @register_passive, @register_item, @register_trait (and register_active_simple for declarative single-target spec abilities). Roster ability ids are prefixed (champ_x.active); every id must resolve (CI-guarded). compile_loadout imports the abilities package to trigger the decorators.

3.2.6 Invariants this system owns

- **V.14 / determinism** -- "chance" effects use a cadence counter (crit_counter, StatusDef.dot_interval_ticks), never RNG.
- **Boundary** -- combat/ never imports content at module scope; content reaches combat only through CombatContext + the registries.

3.2.7 File map

Concern	Symbol
Bus, Hook, HookScope, Modifier, EffectBundle, compute_stat, Lifetime, SourceTag	effects.py
Typed event payloads	events.py (DamageEvent, AttackEvent, DeathEvent, ...)
Status defs/instances, gates, STATUS_DEFS, StackBehaviour	status.py
Mutator API	combat/context.py (CombatContext)
Per-piece modifier/status/barrier state, crit_counter	piece.py
id -> handler registries + @register_*	registries.py
Status tick upkeep	combat/engine.py::process_statuses / expire_modifiers

3.3 Weather -- Favor & Affinity Clash

> **Status:** **LIVING** -- must match src/game/weather_effects.py + loadout._apply_weather_to_piece. Audited by /check. **Scope:** the two decoupled weather systems and the single place each is applied. **Reconciled:** 2026-07-01 @ refactor/combat-engine-single-source.

Six weather states (V.5): CLEAR, CLOUDY, MIST, RAIN, SNOW, THUNDER. Each piece carries exactly one affinity: WeatherState (V.6). CLEAR sits outside the predator/prey ring and is inert in both systems. The two systems are **never summed**:

3.3.0.1 The predator/prey ring (ring_relation)

The five active weathers form a directed cycle CYCLE_ORDER = (MIST, CLOUDY, RAIN, SNOW, THUNDER). For member i: i-1 is its **primary prey**, i-2 its **secondary prey**; i+1/i+2 are its primary/ secondary **predators** (mod 5). ring_relation(a, b) returns a's relation to b -- SELF / PRIMARY_PREDATOR / SECONDARY_PREDATOR / SECONDARY_PREY / PRIMARY_PREY, or NEUTRAL whenever either side is CLEAR. Both systems key off this one function.

The full ring (each member's prey/predators, CYCLE_ORDER order):

weather	primary prey	secondary prey	secondary predator	primary predator
Mist	Thunder	Snow	Rain	Cloudy
Cloudy	Mist	Thunder	Snow	Rain
Rain	Cloudy	Mist	Thunder	Snow
Snow	Rain	Cloudy	Mist	Thunder
Thunder	Snow	Rain	Cloudy	Mist

(Clear is off-ring -- NEUTRAL to everything.)

3.3.1 1. Weather Favor -- combat_modifier

"Does the node weather suit my affinity?" A 5-tier stat buff/debuff (combat_modifier(affinity, weather) -> CombatModifier) driven by the directional predator/prey ring. Applied **once at combat init**, in exactly one place: loadout._apply_weather_to_piece. As of **T.29-pre (V.42)** it no longer folds into base_stats -- it emits source="weather:<state>" Modifiers (*_mult!=1.0 -> ("<stat>","mul",mult), attack_range_delta -> ("attack_range","add",delta)) applied via apply_bundle, so weather composes through compute_stat (base+Σadds)xΠmuls like every other source -- uniformly attributable (stat_breakdown, V.45) and it scales item/augment adds. Values are now **unrounded floats** (not round(valuexmult)); attack_range underflow is held

> = 1 by the _STAT_FLOORS clamp in compute_stat (V.43); HP is reconciled (max_hp = hp = stat("hp")) afterwards since resources are never Modifier'd directly. There is no other application path -- the old weather_effects.apply_weather snapshot and the CombatPieceState model it built were removed (one source of truth).

Magnitude. WEATHER_FAVOR_MAGNITUDE = 0.3 sets the strong-tier primary deviation (±30%); combat_modifier scales that deviation from 1.0 by a per-relation TIER_SCALAR -- **SELF 1.0, PRIMARY 0.6, SECONDARY 0.3**. Buffs reach all three tiers; debuffs reach medium **and** weak (_DEBUFF_RELATIONS = primary/secondary prey -> TIER_SCALAR 0.6 and 0.3), so there is **no strong debuff** (SELF tier never debuffs). The strong-tier packs (WEATHER_BUFF_BASE / WEATHER_DEBUFF_BASE, CombatModifier fields):

weather	buff (self tier)	debuff (prey)
CLOUDY	+30% HP, +30% RES, +15% AS	−AS
MIST	+30% MS, +30% threat, +15% INT	−1 attack_range
SNOW	+30% armor, +30% RES, small STR	−MS
RAIN	+30% AS, +30% mana_regen	−STR
THUNDER	+30% STR, +30% AS	−INT, −mana_regen

weather	buff (self tier)	debuff (prey)
CLEAR	-- (inert)	--

attack_range_delta is an add (rounded): -1 survives the medium tier (round(-0.6) = -1) and vanishes at the weak tier (round(-0.3) = 0).

Favor matrix -- a piece's affinity (row) against the node weather (column). combat_modifier reads ring_relation(affinity, weather): buff +++ = SELF (scalar 1.0), buff ++ = PRIMARY_PREDATOR (0.6), buff + = SECONDARY_PREDATOR (0.3); deb -- = PRIMARY_PREY (0.6), deb - = SECONDARY_PREY (0.3); - = NEUTRAL/inert (_BUFF_RELATIONS = SELF + both predators; debuffs = both prey):

affinity \ weather	Clear	Mist	Cloudy	Rain	Snow	Thunder
Clear	-	-	-	-	-	-
Mist	-	buff +++	deb −−	deb −	buff +	buff ++
Cloudy	-	buff ++	buff +++	deb −−	deb −	buff +
Rain	-	buff +	buff ++	buff +++	deb −−	deb −
Snow	-	deb −	buff +	buff ++	buff +++	deb −−
Thunder	-	deb −−	deb −	buff +	buff ++	buff +++

3.3.2 2. Affinity Clash -- damage_modifier

"Do I beat this enemy?" A per-hit multiplier damage_modifier(attacker_affinity, defender_affinity) applied on **every damage instance** inside the damage pipeline (see combat.md). It depends on the defender, so it can't be pre-snapshotted -- it's resolved per hit, not at init. The multiplier by ring relation of attacker->defender (DAMAGE_MULT):

relation	mult
PRIMARY_PREDATOR	1.30
SECONDARY_PREDATOR	1.12
SELF / NEUTRAL	1.00
SECONDARY_PREY	0.88
PRIMARY_PREY	0.70

Clash matrix -- attacker affinity (row) vs defender affinity (column), the per-hit damage_modifier multiplier (DAMAGE_MULT[ring_relation(atk, def)]):

atk \ def	Clear	Mist	Cloudy	Rain	Snow	Thunder
Clear	1.00	1.00	1.00	1.00	1.00	1.00
Mist	1.00	1.00	0.70	0.88	1.12	1.30
Cloudy	1.00	1.30	1.00	0.70	0.88	1.12
Rain	1.00	1.12	1.30	1.00	0.70	0.88
Snow	1.00	0.88	1.12	1.30	1.00	0.70
Thunder	1.00	0.70	0.88	1.12	1.30	1.00

3.3.3 Why decoupled

Favor asks about the *node*; Clash asks about the *opponent*. Keeping them separate means weather tuning and matchup tuning don't entangle. Both read the same predator/prey ring (`ring_relation`) but apply at different times to different magnitudes (`combat_modifier` stat packs vs `damage_modifier` multiplier).

3.3.4 Also here

`shop_weight(affinity, weather)` -- prep-shop pull weight by ring relation (content economy, not combat). `SHOP_WEIGHT`: SELF 2.0, primary/secondary predator 1.5 / 1.2, NEUTRAL 1.0, secondary/primary prey 0.8 / 0.6 -- pieces that hunt the upcoming node weather surface more often.

3.3.5 File map

Concern	Symbol
Ring relation (predator/prey)	`weather_effects.ring_relation`
Weather Favor stat pack	`weather_effects.combat_modifier -> CombatModifier`
Weather Favor application (only path)	`loadout._apply_weather_to_piece`
Affinity Clash per-hit multiplier	`weather_effects.damage_modifier`
Shop pull weight	`weather_effects.shop_weight`

3.4 Formation -- enemy spawn placement

> **Status: LIVING** -- must match `src/game/formation.py` (called by `combat/engine.py::assign_spawns`). Audited by `/check`. **Scope:** how a generated enemy squad is placed on the board. **Reconciled:** 2026-06-05 @ refactor/combat-engine-single-source.

Deterministic, role-aware placement of the enemy squad on the right of the board (columns 7-9). Pure function -- no RNG, no I/O (V.1, V.2). Player champions are placed by the engine on the left (index-packed); this module handles **enemies only**.

3.4.1 Entry

`plan_enemy_formation(pieces, enemy_defs_by_id, *, boss_position=None, board_height=BOARD_HEIGHT) -> dict[int, tuple[int, int]]` -- maps each enemy's `formation_index -> (col, row)`. Called by `combat/engine.py::assign_spawns`, which builds a lightweight `_FormationEnemy` shim per enemy (not a full `Piece`), calls this, then writes `formation[enemy.formation_index]` back onto `piece.position_q/position_r`.

3.4.2 The input contract -- `FormationPiece`

The planner reads exactly three attributes, declared as a structural `Protocol FormationPiece` in `formation.py`:

- `piece_id: str` -- identity (sorted by, for determinism).
- `tier: int` -- boss/role weighting (tier 10 = boss).

- `formation_index: int` -- the board-slot key the returned `placements` dict is keyed by. **Not** the combat tiebreak: T.33a split the old `speed_tiebreaker` into `formation_index` (this placement key) and `load_order` (the `_event_sort_key` tiebreak, see combat.md). Formation only touches `formation_index`.

`combat/engine.py::_FormationEnemy` and the tests satisfy this structurally -- formation imports **no** piece model, keeping it pure (V.1).

3.4.3 Placement policy

`classify_role(enemy_def: EnemyDef) -> PlacementRole` buckets each enemy by `enemy_def.durability` and `enemy_def.reach`:

`PlacementRole`	Column	Rule (durability / reach)
`FRONTLINE`	`COL_FRONT` (7)	`durability in ("tanky_hp", "tanky_arm")`
`FLANK`	`COL_MID`/`COL_BACK` (8-9) edge rows	`reach == "melee" and durability == "squishy"`
`MIDLINE`	`COL_MID` (8)	`reach == "melee"` (else)
`BACKLINE`	`COL_BACK` (9)	ranged (fallthrough)

The planner runs a fixed, deterministic pass order (enemies pre-sorted by `piece_id` first): **frontline -> flankers -> midline -> backline -> boss**. Each band is placed by `_place_band`, which fills its primary column center-out (`_center_out_rows`), spills to `overflow_cols` when the column is full, and falls back to `_nearest_free` as a last resort. Flankers are placed by `_place_flankers` at the four board corners -- edge rows `(0, board_height-1)` of columns 8-9 -- so assassins slip around the frontline toward the backline.

Bosses (tier 10) are detected up front and placed **last** by `_place_boss` at the per-boss authored `boss_position` (default `(COL_BACK, board_height // 2)` = center-back when none is given); a boss **displaces** any occupant of that cell, relocating it via `_nearest_free`. An enemy whose `piece_id` has no matching `EnemyDef` falls back to the `MIDLINE` bucket.

3.4.4 File map

Concern	Symbol
Plan a squad	`formation.plan_enemy_formation`
Role classification	`formation.classify_role` -> `formation.PlacementRole`
Input contract	`formation.FormationPiece` (Protocol: `piece_id`/`tier`/`formation_index`)
Columns	`formation.COL_FRONT`/`COL_MID`/`COL_BACK`
Band placement (column + overflow)	`formation._place_band`
Flanker placement (corner edge rows)	`formation._place_flankers`
Row packing	`formation._center_out_rows`, `formation._nearest_free`
Boss slot (displaces occupant)	`formation._place_boss`

Concern	Symbol
Caller (builds `_FormationEnemy`, writes coords)	`combat/engine.py::assign_spawns`

3.5 Encounter & board -- squad generation, bosses, map effects

Status: **LIVING** -- must match `src/game/encounter.py`, `bosses/data.py`, `map_effects.py`, `board.py`. Audited by `/check`. **Scope:** seed-deterministic enemy/boss squad generation, difficulty, and the board-cell state combat reads. **Reconciled:** 2026-07-01.

Citations by symbol, not line. Placement of the generated squad is formation.md. Design rationale (frozen): `docs/design/tasks/t19_*`, `t21_*`.

3.5.1 Determinism -- every roll derives from the seed

All procedural choices come from `derive_seed(run_seed, node_index, channel) -> int` (V.2/V.14) -- same `(run_seed, node_index, channel)` -> identical result. The derivation is integer-only (no `hash()`, which is per-process salted):

```
(run_seed * 2654435761 + node_index * 40503 + channel * 97) & 0xFFFFFFFF
```

The RNG is Python `Random` seeded from that derived value; there is no clock, no global RNG, no external state. Each domain gets its own **channel** so rolls never collide (e.g. the REWARD loot roll is independent of the squad roll):

Channel const	#	Used by
`CH_ENEMIES`	0	`generate_fight` / `generate_reward` squad rolls
`CH_AUGMENT`	1	fresh augment offer (`augment_seed`, `reroll_count==0`)
`CH_SUPPLY`	2	supply-node champion offer (`supply_seed`)
`CH_REROLL`	3	first augment/supply reroll (`augment_seed` count `1`, strided for `>=2`)
`CH_CHALLENGE`	4	`generate_challenge` squad + reward
`CH_BOSS`	5	`generate_boss_encounter` variable adds
`CH_SHOP`	6	champion shop offers (`shop_seed`)
`CH_ECONOMY`	7	per-node Amber win bonus (`economy_seed`)
`CH_REWARD`	8	REWARD-node loot roll (`generate_reward_loot`)

Per-domain seed helpers wrap `derive_seed` with the right channel (and, for rerolls, a stride so successive rerolls stay distinct without colliding across nodes/visits):

- `augment_seed(run_seed, node_index, reroll_count=0)` -- `reroll_count 0` -> `CH_AUGMENT`; `1` -> `CH_REROLL` (both **byte-identical to the pre-reroll channels**, no determinism re-baseline); `>=2` folds `node_index * AUGMENT_REROLL_STRIDE + reroll_count` into the node arg on `CH_REROLL` for awarded/banked rerolls (T.42a, V.84).
- `supply_seed(run_seed, node_index, rerolled=False)` -- `CH_SUPPLY`, or `CH_REROLL` when `rerolled`.

- `shop_seed(run_seed, visit_index, reroll_count=0)` -- folds `visit_index * SHOP_REROLL_STRIDE + reroll_count` into the node arg on `CH_SHOP` so each manual reroll within a visit is deterministic + distinct.
- `economy_seed(run_seed, node_index)` -- `CH_ECONOMY`.

Both `*_REROLL_STRIDE` constants are `1000` -- far above any realistic per-node reroll count.

3.5.2 Difficulty

`next_dc(current_dc) -> float` advances the difficulty coefficient (`xDC_STEP`, `= 1.1`, rounded to 4 dp) from `DEFAULT_DC = 1.0`; `dc_name(dc)` labels it (`"DC +0"`, `"DC +1"`, ... from `round(log(dc)/log(1.1))`). The `dc` multiplier scales the per-node power **budget** (`STAGE_BASE[stage] * dc * TYPE_MULT[...]`), so a higher DC buys bigger/higher-level squads, not more of them.

Budget inputs (all `Final` dicts in `encounter.py`): `STAGE_BASE` (per-stage power floor `3.5 ... 65.0`), `TYPE_MULT` (`fight 1.0` / `reward 0.5` / `challenge 1.3`), `STAGE_MAX_SQUAD` (`4 ... 10` cap), `LEVEL_WEIGHTS` + `PREFERRED_TIERS` (stage->tier/level curve feeding `_tier_weight` / `_pick_level`).

3.5.3 Squad generation

- `roll_squad(...)` -- the core roller: weighted tier pick (`_tier_weight` by stage), level pick (`_pick_level`), affinity slotting (`_affinity_slots` by stage affinity), role-composition guard (`_check_composition` over `_is_tanky`/`_is_support`/`_is_dps`), via `_weighted_pick` over a `filter_pool` of `EnemyDefs`. Builds `Enemy` instances with `_instantiate_enemy`.
- `generate_fight(...)` -- a standard fight encounter.
- `generate_reward(...)` -- reward-node contents (enemy squad / supplies).
- `generate_reward_loot(run_seed, node_index) -> RewardLoot` -- seed-deterministic item drop for REWARD nodes; uses channel `CH_REWARD = 8`. 60% one component, 25% one core item, 15% two components. Added in T.29a. See items.md.
- `generate_challenge(...)` -- challenge nodes; can pull champion-derived enemies (`_champion_def_to_enemy`) and yields a `ChallengeReward`.
- `generate_boss_encounter(run_seed, node_index, stage) -> BossEncounterResult` -- the boss squad + its map effect.
- `generate_node_reward(run_seed, node) -> NodeReward | None` (T.38, V.70) -- the **single reward-payload source**, type-dispatched (mirrors `node_encounter`): **REWARD** -> `generate_reward_loot` items; **CHALLENGE** -> the `generate_challenge` reward (amber `2 x stage_index`, both components, `tempest_bonus`, `champion_offer`); **all other types** -> `None`. Uses `node.weather` (the node's `default_weather`, not live API weather) so the payload is byte-identical to the reward `node_encounter` *discards* (the fight-build path stays squad-only; the resolve path `economy.apply_node_result` owns the reward). `NodeReward` fields -> `Run.inventory` / `amber` / tempest / pending recruit. Applied **on win only** => a loss is structurally reward-zeroed.

3.5.4 Per-node dispatch -- one seam, two outputs

Two dataclass-returning entry points dispatch on `node.node_type` (`models.NodeType`) so the Trail *preview* and the later Prep *fight* read the same seed:

- `node_encounter(run_seed, node, weather=None, dc=DEFAULT_DC) -> NodeEncounter` -- the **squad** (and boss `map_effect_id`). FIGHT -> `generate_fight`, REWARD -> `generate_reward`,

CHALLENGE -> `generate_challenge` (squad only; `weather` defaults to `node.weather` and only steers CHALLENGE affinity), BOSS_FIGHT -> `generate_boss_encounter().all_enemies`. **AUGMENT / SUPPLY -> empty** `NodeEncounter([])` (no fight). Same `(run_seed, node, weather, dc)` => same squad (V.2/V.63) -- the previewed squad is byte-identical to the one fought.

- `generate_node_reward(run_seed, node) -> NodeReward | None` (see below).

Non-combat nodes (AUGMENT / SUPPLY) never call combat: the node's *pick* (augment / supply recruit) is applied by the view, then `economy.resolve_nonfight_node(run)` marks the node CLEARED + advances -- **no income, tempest, Hearts, or** `battle_log` (V.83). See economy.

3.5.5 Bosses

`bosses/data.py`: `BossDef` (kit, `BossCastEntry` list, `map_effect_id`, `spawn_position`), the `BOSS_DEFS` registry, and `BossEncounterResult` (`.all_enemies` property = boss + adds, `.map_effect_id`). Combat wiring for a boss fight (attach the map effect before the loop) is `combat/resolve.py::resolve_boss_combat` (the single src-side entry, V.59; the `tools/playtest/_common.py` function is a shim that delegates to it) -- see combat.md.

3.5.6 Board & map effects

`board.py BoardState` carries per-cell state the loop reads each tick: `slow_cells` (+ `is_slow(q, r)`, consumed by `engine._process_board_state`), `fog_range` (+ `is_in_fog_range(...)`, read by `targeting.py`), and `CellModifier`. `map_effects.py` defines `MapEffect` (base; `register(bus)` wires it) and the concrete effects -- `SunlitTilesEffect`, `FogEffect`, `HazardTilesEffect`, `DefensiveLeyEffect`, `FloodLanesEffect`, `SlowTilesEffect` -- keyed in `MAP_EFFECT_CLASSES`; `build_map_effect(effect_id, board, seed)` constructs one. `loadout.attach_map_effect` (see combat.md) builds

- registers it onto a `CombatContext`.

3.5.7 File map

Concern	Symbol
Seed derivation	`encounter.derive_seed` + `CH_*` channels; helpers `augment_seed(reroll_count)`/`supply_seed`/`shop_seed`/`economy_seed`; `AUGMENT_REROLL_STRIDE`/`SHOP_REROLL_STRIDE`
Difficulty	`encounter.next_dc` / `dc_name` (`DC_STEP`, `STAGE_BASE`, `TYPE_MULT`)
Squad roll	`encounter.roll_squad`, `generate_fight`/`generate_reward`/`generate_challenge`/`generate_boss_encounter`
Per-node dispatch	`encounter.node_encounter` -> `NodeEncounter`; `generate_node_reward` -> `NodeReward`
Loot roll	`encounter.generate_reward_loot` -> `RewardLoot` (`CH_REWARD`)
Boss data	`bosses/data.py` (`BossDef`, `BOSS_DEFS`, `BossEncounterResult`)
Board state	`board.py` (`BoardState`, `CellModifier`)
Map effects	`map_effects.py` (`MapEffect`, `MAP_EFFECT_CLASSES`, `build_map_effect`)

3.6 Stat scaling

Status: **LIVING** -- must match `src/game/scaling.py` + `content.py` stat curves. Audited by `/check`. **Scope:** the power curve P(tier,level), the three stat-scaling classes, baseline parity, and the combat tie-break. **Reconciled:** 2026-07-01 (T.33a/b).

3.6.1 Power curve

- `power(tier, level) = 2 ** ((tier-1)/3 + triplings[level])`, `triplings = {1:0, 2:1, 3:3}` (T.18). `tier in [1,10]`, `level in [1,3]` -- out of range raises `ValueError`.
- Level-ups are a **tripling** mechanic (mirrors TFT 3-to-1): 3 copies -> L2 (1 tripling), 9 -> L3 (3 triplings). Accelerating: L1->L2 is $\sqrt{2}$ in stats, L2->L3 is `x2`. `LEVEL_UP_MOD = 2.0`, `TIER_UP_MOD = 2**(1/3) ~= 1.26` (three tier-ups = one tripling of power); `LEVEL_STEP`/`TIER_STEP` are back-compat aliases.
- `stat_multiplier(tier, level, exponent=PRIMARY_EXPONENT) = power(tier, level) ** exponent`. At the default `0.5` this is `sqrt(power)`, so HP-DPS proportional to P and encounter budgets stay **linear** in P.
- `scale_stat(base, tier, level) -> int` -- one-shot `round(base * stat_multiplier(...))` for a single primary value (used where a full dict isn't in play).

3.6.2 Three scaling classes (T.33, V.34)

Every base stat is in exactly one class; both curves ride the same `power` curve at a different exponent:

class -- stats	exponent	per-tier	T1L1->T10L3
`PRIMARY_SCALABLE_STATS` -- `max_hp strength intelligence armor resistance`	`PRIMARY_EXPONENT=0.5` (`sqrt(power)`)	~= x1.122	x8
`SECONDARY_SCALABLE_STATS` -- `attack_speed move_speed mana_regen threat`	`SECONDARY_EXPONENT=0.0857`	~= x1.02	x1.428
`FLAT_STATS` -- `attack_range`	--	--	--

`crit_chance`/`penetration`/`penetration_pct` are ratios, off the scaling model. `SCALABLE_STATS` is a deprecated alias of the primary tuple. `level_scale_stats(stats, tier, level)` applies both curves in place -- the single source of truth for the four builders (`content._build_champion`/`_build_enemy`, `encounter._instantiate_enemy`/`_champion_def_to_enemy`). `threat`/`move_speed` stay **off** the HP-DPS power budget (V.33, B.6).

3.6.3 Quantities: int except attack_speed

Stored quantities (hp, damage, mana, `move_speed`/`mana_regen`/`threat`, costs, energy meters) are **int**; the ratio stats **and** `attack_speed` are float. `attack_speed` is a float (T.29-pre, amends V.34): cadence reads `int(attack_speed)`, and the sub-integer precision for same-tick ordering derives from the **same** value via `round(attack_speed x 1000)` -- there is no separate `milli_AS` field. Because weather now applies as `mul` modifiers (not a base fold), `attack_speed` stays float through level **and** combat.

3.6.4 Speed-stat baseline parity (#39, V.35)

_BASE_STATS attack_speed == move_speed == mana_regen == 100 so the three speed stats compare directly as power investments. The per-meter capacitor is deliberately unequal and non-player-facing: mana mana_cost baseline 300_000 » action/move ENERGY_THRESHOLD = 60_000 (a cast ~= 5 autos). mana_cost 300_000 (vs cadence-neutral 360_000) is a deliberate ~20% mage buff; move_speed 90->100 a ~11% movement buff. **T.29c (V.48):** cast cost is no longer a per-piece ability_cost FLAT stat -- it is authored **per-ability** on the ability def (ABILITY_MANA, default 300_000); bosses register their own (380k-520k). max_mana defaults to 2xmana_cost (overload headroom).

3.6.5 Speed axis -- 7 levels (T.33b)

_SPEED (content.py): moloch / leaden / heavy / hybrid / light / swift / blazing, slow->fast. Faster => ↑move_speed, ↓primary_stat (softer per-hit/cast), and a tempo gain that **routes by playstyle**: auto/hybrid pieces get the full attack_speed mult; ability casters get **half** the AS deviation applied to *both* attack_speed and mana_regen (more, softer casts) -- speed no longer touches resistance. Roughly power-neutral (cadence up, per-hit down). hybrid is the neutral centre (omitted from role_code). Widening 3->7 took the role-code space to 1512 combos (V.32); classify_role ignores speed, so role *titles* are unaffected. The full 60-champion + 60-enemy roster is assigned across the 7 levels by theme + within-tier AS spread (54/60 champions distinct-AS within their tier).

3.6.6 Combat tie-break (V.34, fixes B.14)

Same-tick action order is the canonical side-independent total order in _event_sort_key (combat/engine.py): (-round(attack_speed x 1000), champion_id, load_order, kind) -- the quantized AS key (T.29-pre) is monotonic in the float attack_speed, so it subsumes the old coarse -AS_int level and the separate milli_AS field in one term.

- load_order -- a seeded, side-independent permutation assigned in compile_loadout (never team-block-then-enemy), so equal-AS ties never systematically favour the player team (the old B.14 bug). Renamed from the overloaded speed_tiebreaker; the formation-position key is now formation_index.

3.6.7 Where it lives

- scaling.py -- power curve, the three class tuples + exponents, stat_multiplier, level_scale_stats, scale_stat.
- content.py -- base stat blocks, axis multipliers, compose_stats (attack_speed kept float). Cast cost is authored per-ability via ABILITY_MANA / DEFAULT_MANA_COST in registries.py (the former _ABILITY_COST baseline).
- combat/engine.py -- _event_sort_key. loadout.py -- load_order/formation_index assignment, weather application.
- tools/gen_role_matrix.py -- regenerates docs/design/tasks/t32_role_matrix.txt.

3.7 Weather API -- fetch, cache, refresher

Status: LIVING -- must match src/api/weather.py, api/cache.py, api/refresher.py. Audited by /check. **Scope:** OpenWeather client, per-city cache states, and the 3-stream

tick refresher (<=3 calls/min). **Reconciled:** 2026-07-01.

Citations by symbol, not line. Design (frozen): `docs/design/tasks/t6_*`, `t7_*`. This layer is the *only* I/O in the project -- `src/game/` never touches it (V.1).

3.7.1 Client -- `api/weather.py`

`WeatherClient(api_key=None)` reads `OPENWEATHER_API_KEY` from env (raises `ValueError` if neither arg nor env is set; the key is never logged, V.3). `fetch_weather(lat, lon, *, fallback=WeatherState.CLEAR)` hits OpenWeather (`/data/2.5/weather`, `units=metric`, 10 s timeout) and returns a **frozen** `WeatherResult(state, temperature, icon_code, description, weather_id, is_fallback=False, error=None)`. On **any** exception (network, HTTP status, parse) it logs a warning and returns `_fallback_result(fallback, error=str(exc))` -- a `WeatherResult` flagged `is_fallback=True` carrying the caller's `fallback` state (`fetch_and_cache` passes the city's `default_weather`) -- it **never raises** (V.3/V.4). The OpenWeather condition-id -> `WeatherState` mapping lives on `WeatherState.from_openweather_id` (`models.py`); each fallback state has a canned `(icon_code, weather_id)` in `_FALLBACK_WEATHER` (e.g. `RAIN -> ("10d", 500)`, `THUNDER -> ("11d", 200)`).

3.7.2 Cache -- `api/cache.py`

`WeatherCache` holds one `CacheEntry` per `city_id` with a `CacheState`:

`CacheState`	Meaning	`fetched_at`
`UNKNOWN`	never fetched	`None`
`LIVE`	real fetch succeeded	set
`SUBSTITUTE`	fetch failed -> city-default weather	set

`set_live` / `set_substitute` update an entry; `fetch_and_cache(cache, client, city_id, city_def)` does the round trip -- success -> `set_live`, `is_fallback` -> `set_substitute`. Advancing to an `UNKNOWN` city triggers a synchronous fetch.

3.7.3 Refresher -- `api/refresher.py`

`WeatherRefresher` ticks once per `tick_interval` (default 60 s) on a daemon thread (HTTP off the main thread, V.4). Each `tick()` picks **<=3 unique** cities across three streams and fetches each, returning the fetched `city_ids`:

- **A** -- full round-robin over all cities (`_a_pointer`).
- **B** -- round-robin within the window `[current+1 .. current+6]` (`_b_pointer`). `get_current_node_index()` returns the **1-based** `Run.current_node_index`; the refresher converts to a 0-based list offset internally.
- **C** -- uniform random (seedable via `rng_seed` for tests).

Dedupe across streams (a `seen` set; B and C only append if unseen) keeps it to <=3 API calls/min (V.11 -- <=50 min staleness via stream A). `start()` arms a daemon `threading.Timer` that re-arms itself each tick; `stop()` is idempotent; `running` reflects timer state. Each `tick()` fetches via `fetch_and_cache` (exceptions per city are logged, never propagated) and then fires the optional `on_tick(fetched_ids)` callback (T.11) on the worker thread so a UI can repaint when entries

flip LIVE/SUBSTITUTE -- a callback exception is logged, never allowed to kill the refresher. `on_tick=None` (default) keeps every existing caller byte-identical.

3.7.4 Node weather lifecycle -- persisted on the `Run` (T.39, V.73)

The cache/refresher are **fetch-scheduling + freshness** only; the **persisted source of truth is the `Run` `Node`**. Each `Node` (`game/models.py`) carries:

Field	Meaning
`weather: WeatherState`	the **effective** game weather all systems read -- `default_weather` placeholder until a live fetch overwrites it (T4 §2)
`weather_state: NodeWeatherState`	`UNKNOWN` (never fetched -> display `?`) / `LIVE` / `SUBSTITUTE` -- distinct from `api.cache.CacheState` (game-local enum; importing CacheState would cycle)
`weather_locked: bool`	frozen for the run; the refresher skips it

The Trail copies fetched cache values onto the `Run` via the **pure game-side mutators** `Run.set_node_live_weather(node_index, weather, *, is_substitute)` and `Run.lock_node_weather(node_index)` only -- the cache/refresher never touch game state (V.10). `set_node_live_weather` is a **no-op on a locked node**. The **current** node's weather **locks at the Trail->Prep transition** (`trail.py::_play_next -> lock_node_weather`, persisted by `main._push_prep`'s `save_run`); an `UNKNOWN` node freezes `default_weather` flagged `SUBSTITUTE`. Locking before the fight keeps the CHALLENGE squad roll + `generate_node_reward` byte-identical (load-bearing for V.70). Pre-T.39 saves lack the two new fields -> `UNKNOWN`/`False` on read (no `schema_version` bump).

3.7.5 Invariants this system owns

- **V.3** -- the API key is never logged.
- **V.4** -- all HTTP runs on a worker thread; a fetch failure never crashes (falls back to substitute weather).
- **V.1** -- game logic has zero dependence on this layer; it consumes only the resulting `WeatherState`.

3.7.6 File map

Concern	Symbol
HTTP client + fallback	`api/weather.py` (`WeatherClient.fetch_weather`, `WeatherResult`, `_fallback_result`)
Per-city cache	`api/cache.py` (`WeatherCache`, `CacheEntry`, `CacheState`, `fetch_and_cache`)
3-stream tick loop	`api/refresher.py` (`WeatherRefresher.tick`)
id -> state mapping	`models.py::WeatherState.from_openweather_id`

Concern	Symbol
persisted node lifecycle	`models.py` (`Node.weather`/`weather_state`/`weather_locked`, `NodeWeatherState`, `Run.set_node_live_weather`/`lock_node_weather`); write-through + lock in `ui/views/trail.py`

3.8 Save / serialization

Status: **LIVING** -- must match `Run`/`BattleResult` (de)serialization in `src/game/models.py` + the file-I/O layer in `src/game/save.py`. Audited by `/check`. **Scope:** how game state round-trips to/from JSON, the back-compat rules, and the file read/write contract. **Reconciled:** 2026-07-01.

Citations by symbol, not line. The (de)serialization contract lives on the **model dataclasses**; the **file I/O** (atomic write, schema gate, typed errors) lives in `game/save.py` (T.14).

3.8.1 The model is the schema

Every persisted dataclass has `to_dict() -> dict` and `@classmethod from_dict(payload)`: `Run`, `Champion`, `Enemy`, `Node`, `BattleEvent`, `BattleResult`. `Run` is the root -- it holds all game state and serializes its nested objects.

`Run` fields (root): `run_id`, `schema_version` (>= 1, validated), `seed`, the route `Nodes`, the champion roster, `amber` (economy), `battle_log` (`list[BattleResult]`), status. `Run.to_dict` nests each child's `to_dict`; `from_dict` rebuilds them.

3.8.2 Back-compat rules (the part that bites)

`from_dict` must tolerate older payloads. Current rules:

- `amber <- gold` -- old saves used `"gold"`; `Run.from_dict` reads the legacy `gold` key when `amber` is absent (B.4).
- `BattleResult.piece_max_hp` -- added later; `from_dict` defaults it to `{}` for pre-field saves (the combat-log HP trace is just empty for them). See combat.md.
- `Node.weather_state` / `Node.weather_locked` (T.39) -- `Node.from_dict` reads `payload.get("weather_state", "unknown")` (-> `NodeWeatherState.UNKNOWN`) and `payload.get("weather_locked", False)`, so pre-T.39 saves load with no `schema_version` bump. See weather_api.md.
- New optional fields follow the same pattern: `payload.get(key, default)`, never a hard `payload[key]`, so old saves still load. The *required* keys (`schema_version`, and whatever `Run.from_dict` reads unconditionally) are the only ones that can raise `CorruptSaveError`.

`schema_version` exists to gate breaking migrations; bump it when a change can't be handled by a `.get` default.

3.8.3 File I/O (`game/save.py`, T.14)

The disk layer over the model contract. Imports only `json`/`os`/`pathlib` -- no Flet (V.1).

- `CURRENT_SCHEMA_VERSION` (`= 1`) -- the single source for the version this build writes/reads (was hardcoded per-Run before T.14).
- `save_run(run, path)` -- `run.to_dict()` -> JSON, **atomic**: writes `<path>.tmp`, `flush+fsync`, then `os.replace` onto `path`; auto-creates parent dirs; removes the temp on any failure. Readers never see a partial file.
- `load_run(path)` -- reads + gates on `schema_version` (see errors below), then `Run.from_dict`. The migration hook sits between the gate and `from_dict`.
- `default_save_dir()` -- platform app-data dir (`%APPDATA%` / macOS `Application Support` / `$XDG_DATA_HOME`) `.../tempest-fauna-trail/saves`. Helper only; core fns take an explicit path. Does **not** create the dir (save does).
- **Errors:** `SaveError` (base) -> `CorruptSaveError` (bad JSON, missing/mistyped required keys incl. `schema_version`, or a payload that fails `Run` validation -- `from_dict`'s `ValueError/KeyError/TypeError` are wrapped), `UnsupportedSchemaError` (`schema_version` > `CURRENT_SCHEMA_VERSION`). `FileNotFoundError` is **not** wrapped -- callers branch on "no save yet".

3.8.4 Invariant

- Round-trip identity: `from_dict(x.to_dict()) == x` for current data, and old-shaped payloads load without error (back-compat). `/check`, `tests/game/test_models.py`, and `tests/game/test_save.py` (through disk) guard this.
- File safety: `save_run` is atomic (temp + `os.replace`); `load_run` never returns a `Run` from a future-schema or invalid file -- it raises a typed `SaveError`. (V.36)

3.8.5 File map

Concern	Symbol
Root state + nesting	`models.py::Run.to_dict` / `Run.from_dict`
Battle records	`models.py::BattleResult.to_dict` / `from_dict` (`piece_max_hp` optional)
Per-entity	`Champion` / `Enemy` / `Node` / `BattleEvent .to_dict`/`.from_dict`
Legacy key read	`Run.from_dict gold->amber` (B.4)
File read/write	`save.py::save_run` / `load_run` (atomic, schema-gated)
Save version constant	`save.py::CURRENT_SCHEMA_VERSION`
Default save location	`save.py::default_save_dir`
Typed errors	`save.py::SaveError` / `CorruptSaveError` / `UnsupportedSchemaError`

3.9 Items -- engine, components, combined, equip, reward drops

Status: LIVING -- must match `src/game/items/`, `src/game/loadout.py`, `src/game/models.py`, `src/game/encounter.py`, `src/game/registries.py`. Audited by `/check`. **Last reconciled: 2026-07-01** (T.29a-d complete -- components, combined, emblems, special run-actions, mana primitive, multi-slot).

Design rationale (frozen): `docs/design/tasks/t29_item_engine_plan.md`. Content roster (frozen): `docs/design/content/item_catalog.md`.

3.9.1 Package layout

```
src/game/items/
  __init__.py     -- re-exports BASE_COMPONENTS, SPIRIT_GEM, KINSHIP_OF, kinship_of,
                     RECIPE_MAP, combine; importing this module triggers all
                     @register_item / @register_run_action side-effects
  base.py         -- BASE_COMPONENTS: frozenset[str] (8 component IDs), SPIRIT_GEM,
                     KINSHIP_OF (6 component->Kinship), kinship_of()
  recipes.py      -- RECIPE_MAP: dict[frozenset[str], str] (36 entries), combine()
  combined.py     -- @register_item factories: 8 raw components + 16 core + 20
                     combined (T.29b) = 44 in ITEM_REGISTRY
  emblems.py      -- 6 Kinship emblems (T.29b; ITEM_REGISTRY total = 50)
  special.py      -- 5 RUN_ACTION_REGISTRY special items (T.29b, operate on Run) +
                     decompose() helper; Spirit Gem handled inline by combine()
  meta.py         -- ITEM_META: name + blurb for all 50 ids (T.41a; see content doc)
```

Two registries in `src/game/registries.py` (populated by the `@register_*` decorators when `src.game.items` is imported):

- `ITEM_REGISTRY` = **50** -- `item_id -> Callable[[Piece], EffectBundle]`. The combat-facing factories (`register_item`).
- `RUN_ACTION_REGISTRY` = **5** -- `item_id -> Callable[[Run, *args], None]`. Special run-actions (`register_run_action`) that mutate `Run` and **never enter combat** (V.24). Kept separate so `game/combat/` never sees them.

3.9.2 Data structures

3.9.2.1 Champion.items (models.py)

```python
@dataclass
class Champion:
    items: list[str] = field(default_factory=list)   # <=3 item IDs, validated
```

Invariants (V.23):

- Length <= 3 (validated in `Champion.__post_init__`).
- Every string is non-empty.
- Items persist across combats on the `Champion`; each combat rebuilds from scratch.

3.9.2.2 RECIPE_MAP (recipes.py)

A `dict[frozenset[str], str]` mapping a pair (or singleton, for same-component recipes) of component IDs to the combined item ID.

- 36 entries total (8x8 upper triangle + diagonal).
- Same-component recipes use a **single-element frozenset** because `frozenset({"fang", "fang"}) == frozenset({"fang"})`.
- `spirit_gem` is **not** in `BASE_COMPONENTS`; it is a separate special-item. `combine("spirit_gem", c)` returns `c`'s emblem (`base.KINSHIP_OF`) -- 6 mapped components craft emblems; `ward-pelt/keen_claw -> None` (T.29b).

```python
def combine(a: str, b: str) -> str | None:
```

Returns the combined item ID or `None` if no recipe exists.

3.9.3 Prep equip seam (`game/inventory.py`, T.23b)

The Prep view moves items between `Run.inventory` and `Champion.items` **only** through `game/inventory.py` (V.63 -- never inline):

- `equip_item(run, champion, item_id) -> bool` -- consumes one from inventory onto the champion. **Auto-combines on double-equip:** if the champion already holds a component that pairs with `item_id` into a recipe (`items.combine`), the two fuse into the combined item in a single slot (works even at the 3-item cap); otherwise the item takes a free slot (<=3, the `Champion` cap). The combine partner is the **first** held item that pairs -- deterministic, no RNG (V.2).
- `unequip_item(run, champion, item_id) -> bool` -- returns the item whole to inventory (no de-combine).

3.9.4 Equip pipeline (combat-side)

`compile_loadout` (`loadout.py`) applies items at **step 2.5** -- after weather modifiers (step 2) but before trait resolution (step 3). The full pass order (§10.1) is:

```
step 1   : build pieces from Champion/Enemy models
step 1b  : uniquify duplicate piece ids (B.65: e.g. enemy twins -> id, id#1, ...)
step 2   : weather-favor modifiers (source="weather:<state>", V.42)
step 2.5 : item bundles          <- T.29a  (emblem granted_traits must land here)
step 3   : trait breakpoint resolution (player team only, V.22)
step 3.5 : Built-Different active-synergy flag (augment filter, T.31)
step 6   : active augment bundles (TEAM/PIECE, T.31)
step 7   : champion/enemy passive bundles
step 8   : boss phase / on-death hooks
step 9   : quest trackers (Run-level bus subscribers)
```

Flow at step 2.5:

1. `piece_from_champion` copies `champion.items -> piece.items` (Piece.items is a `list[str]`; defaults to `[]`).
2. `compile_loadout` imports `src.game.items` (triggers registry population) then iterates `piece.items`; for each item ID it strips a `heartwood:` prefix if present, looks up `ITEM_REGISTRY[base_id](piece) -> EffectBundle` (`_heartwood_scaled` x1.5 if it was a Heartwood), then `apply_bundle(piece, bundle, bus)`.
3. `apply_bundle` (`loadout.py:122`) appends `granted_traits`, `modifiers`, and `statuses` to the piece and `bus.subscribes` each `Hook`. Item modifiers use `Lifetime.COMBAT` (pieces are rebuilt each combat -- no cross-combat accumulation).

Item `granted_traits` (emblems) land at 2.5 **before** trait resolution (step 3) so an emblem wearer counts toward that Kinship's breakpoints. Enemies carry **no items**; `piece_from_enemy` is unchanged.

3.9.5 Item factories (@register_item) -- the modifier + hook pattern

Each factory in `combined.py` / `emblems.py` is a `Callable[[Piece], EffectBundle]` registered via `@register_item(item_id)`. A **fresh** `EffectBundle` (with fresh hook closures) is returned on every call, so per-combat state resets automatically. A factory returns an `EffectBundle` containing any of:

- `modifiers`: stat deltas via `Modifier(stat, op, value, Lifetime.COMBAT, source)` -- `op="mul"` (e.g. `1.12` = x1.12 the composed stat) or `op="add"` (flat).
- `hooks`: `list[Hook]` -- each `Hook(event_name, fn, scope)` is subscribed to the EventBus; `fn(ctx, ev)` closures capture `owner: Piece` for per-instance state (one-shot flags, **deterministic** cadence counters -- never RNG, V.2/V.14).
- `granted_traits`: Kinship strings (emblems only) appended to `piece.traits`.

A pure stat-stick returns just `modifiers`; an active-proc item pairs a `Modifier` with a `Hook`. Concrete example (`combined.py`):

```
@register_item("apex_fang")                # fang + fang
def apex_fang(owner: Any) -> EffectBundle:
    def on_kill(ctx, ev):                   # +5% current STR on every kill (compounds)
        if ev.killer is not owner:
            return
        bonus = owner.stat("strength") * 0.05
        ctx.apply_modifier(owner, Modifier("strength", "add", bonus, Lifetime.COMBAT,
↪  "item:apex_fang"))
    return EffectBundle(
        modifiers=[Modifier("strength", "mul", 1.24, Lifetime.COMBAT, "item:apex_fang")],
        hooks=[Hook("on_kill", on_kill, scope=HookScope.PER_HIT)],
    )
```

Hooks fire on engine events: `on_combat_start`, `on_tick`, `on_attack_landed` (an `AttackEvent`, no `tag` field -- real basics only), `on_damage_dealt`, `on_damage_taken`, `on_cast`, `on_kill`. `ctx` is the `CombatContext` mutator API (`deal_damage`, `heal`, `apply_status`, `apply_modifier`, `grant_barrier`, `gain_mana`, `cast_ability`, `enemies_of`, ...). Every `source`/`source_id` is tagged `"item:<id>"` for `stat_breakdown` attribution (V.45).

3.9.5.1 Raw components (8)

ID	Grants
`fang`	+12% STR (multiply)
`keen_claw`	+15% crit chance (additive)
`talon`	+12% AS (single float `attack_speed` Modifier)
`wardpelt`	+14% RES (`resistance`)
`stoneplate`	+14% Armor
`old_hide`	+12% HP
`heartseed`	+12% INT
`springtear`	x1.15 `mana_regen` (Modifier) + `on_combat_start` hook: +100_000 flat starting mana (~=1/3 of default cost, V.48)

`springtear` grants `mana_regen` (the cast-rate knob) plus `start_mana` via an `on_combat_start` hook

(mana is per-ActiveSlot, not in Piece.base_stats). **Mana items NEVER reduce** mana_cost (V.48, T.29c -- kills negative-cost stacking, B.21); they grant mana_regen or start_mana only.

All modifiers use "mul" (multiplicative) or "add" (additive) operation via Modifier(stat, op, delta, Lifetime.COMBAT, source). The multipliers in the factories represent the scale factor (e.g. 1.12 = x1.12 the base stat).

3.9.5.2 Combined items (16-core cut)

ID	Components	Key effects
apex_fang	fang + fang	x1.24 STR; on-kill grants +5% of current STR (compounding add)
tempest_talons	talon + talon	x1.24 AS; each auto-hit adds +0.5% of current AS (compounding ramp)
worldroot_bloom	heartseed + heartseed	x1.30 INT (pure stat stick)
deepwell	springtear + springtear	x1.30 mana_regen + +200_000 flat starting mana; combat-start barrier (15% holder max HP) on lowest-HP ally (support); after first cast, refunds 50% mana_cost (clamp max_mana) per cast (V.48)
mammoth_hide	old_hide + old_hide	x1.24 HP; every 2 s heals holder + adjacent allies 2% max HP (team regen aura ~=1%/s, ungated)
bramble_carapace	stoneplate + stoneplate	x1.28 Armor; flat 80 magic retaliate + grievous (halved healing, 2 s) to melee attackers (thorns, flat by design)
mistward_shroud	wardpelt + wardpelt	x1.28 RES; regenerates 1% max HP every second (self only)
perfect_predator	keen_claw + keen_claw	+30% crit chance; critical hits deal +25% bonus damage (ITEM_PROC)
bloodthorn_briar	fang + heartseed	x1.12 STR, x1.12 INT; heals holder for 18% of all damage dealt
wildfury_lash	talon + heartseed	x1.12 AS, x1.12 INT; each auto adds +1% current AS; every 5th auto triggers a free cast
everbloom_staff	heartseed + springtear	x1.12 INT, x1.15 mana_regen, +100_000 flat starting mana; INT grows +1% per 2 s while alive (V.48)
witherbloom_censer	heartseed + old_hide	x1.12 INT, x1.12 HP; autos apply burn (3 s) + RES x0.80 sunder + grievous (halved healing) (single refreshing instances)
stormglass_totem	heartseed + wardpelt	x1.12 INT, x1.14 RES; when a nearby enemy casts (radius 5), zaps them for INTx0.50 magic damage
spellfang_crown	heartseed + keen_claw	x1.12 INT, +15% crit chance; abilities can crit (ability_can_crit set on on_combat_start)
living_bulwark	old_hide + stoneplate	x1.12 HP, x1.14 Armor; combat-start +18% Armor aura to adjacent allies (support anchor)
splitwind_talons	talon + wardpelt	x1.12 AS, x1.14 RES; each auto strikes nearest second enemy within range 2 for 50% dmg + applies Slow (soft CC) to both

Hook guards: `on_damage_dealt` hooks check `ev.tag == SourceTag.ITEM_PROC` to avoid triggering on bonus damage from the same item (preventing infinite loops).

`ability_can_crit` is set in an `on_combat_start` hook (matching the `ability_crit()` idiom in `traits/mechanics.py:287-292`).

`attack_speed` **sort order:** items grant a single float `attack_speed` Modifier -- the old separate int `milli_AS` field is **gone** (removed in T.29-pre, B.18). Sub-integer tiebreak order now **derives** from the float `attack_speed` itself (V.34), so there is no paired field to keep in sync.

3.9.6 REWARD-node drops (encounter.py)

```python
CH_REWARD: Final[int] = 8   # seed channel for reward loot

@dataclass
class RewardLoot:
    item_ids: list[str]

def generate_reward_loot(run_seed: int, node_index: int) -> RewardLoot:
```

Deterministic drop table (derives from `(run_seed, node_index, CH_REWARD)`):

Roll	Probability	Outcome
< 0.60	60%	1 base component
< 0.85	25%	1 core combined item
else	15%	2 base components

All item IDs returned are guaranteed members of `BASE_COMPONENTS U ITEM_REGISTRY`.

3.9.7 Combined items (remaining 20 -- T.29b)

`combined.py` (T.29b block). Hook items use the closure/cadence/`secs()` patterns; all magnitudes first-pass, judged against combat scale (autos ~5 s, HP 600-1500).

ID	Components	Key effects
`huntress_talon`	fang + talon	+12% STR/AS; autos apply stacking poison (3 s)
`relentless_spear`	fang + springtear	+12% STR, +15% MR; +30k mana per auto
`titanbone_charm`	fang + old_hide	+12% STR/HP; +0.4% STR per attack/hit, barrier (15% HP) at 12 stacks
`beastheart_gauntlet`	fang + stoneplate	+12% STR, +14% Armor; barrier (25% HP) first time below 35%
`twinclaw_pact`	fang + wardpelt	+12% STR, +14% RES; alternates +50% bonus hit / 30% heal
`giantsbane`	fang + keen_claw	+12% STR, +15% Crit; autos +4% target max HP magic (anti-tank)
`stormscale_quiver`	talon + springtear	+12% AS, +15% MR; every 4th auto chains lightning to <=3 enemies
`quickpelt_harness`	talon + old_hide	+12% AS/HP; first hard-CC -> cleanse + 3 s CC-immune

ID	Components	Key effects
sundertalon	talon + stoneplate	+12% AS, +14% Armor; autos shred target Armor x0.82 (3 s)
stalkerclaw	talon + keen_claw	+14% AS, +15% Crit (clean crit stat stick)
stoneward_idol	heartseed + stoneplate	+14% INT, +16% Armor (durable backline caster)
sapwood_aegis	springtear + old_hide	+15% MR, +12% HP; start shield (20% HP) -> INT burst on break
wardens_dewstone	springtear + stoneplate	+15% MR, +14% Armor; +15% MR aura to adjacent allies
seasonward_charm	springtear + wardpelt	+15% MR, +14% RES; adaptive +20% Armor/RES vs recent damage type
dewclaw_fetish	springtear + keen_claw	+15% MR, +15% Crit (cast-cycling crit carry)
spiritbark_hide	old_hide + wardpelt	+12% HP, +16% RES (anti-magic brick)
gorehide_wrap	old_hide + keen_claw	+14% HP, +15% Crit (fragile crit carry survivability)
greatward_carapace	stoneplate + wardpelt	+14% Armor/RES; +4% Armor & RES per living enemy at start
edge_of_stone	stoneplate + keen_claw	+16% Armor, +15% Crit (bruiser-carry hybrid)
hexward_claw	wardpelt + keen_claw	+16% RES, +15% Crit (crit that survives magic burst)

3.9.8 Emblems (6 -- emblems.py, T.29b)

Each carries `granted_traits=["<Kinship>"]` + an 8% flavour stat; applied at the item step (§10.1 step 2.5) **before** `_resolve_traits`, so the wearer counts toward that Kinship breakpoint. Crafted via `combine(spirit_gem, component)`.

Emblem	Kinship	Stat	Crafted from
beast_emblem	Beast	+8% STR	spirit_gem + fang
skyborn_emblem	Skyborn	+8% AS	spirit_gem + talon
scaled_emblem	Scaled	+8% Armor	spirit_gem + stoneplate
tidekin_emblem	Tidekin	+8% MR	spirit_gem + springtear
swarm_emblem	Swarm	+8% HP	spirit_gem + old_hide
spirit_emblem	Spirit	+8% INT	spirit_gem + heartseed

3.9.9 Special items -- run-actions (special.py, T.29b)

Never enter combat (V.24) -- `RUN_ACTION_REGISTRY`, operate on `Run` only. Spirit Gem is the 6th special, handled inline by `combine()` (crafting, not a run-action).

Run-action	id	Effect on `Run`
Wildwood Reforging Stone	`reforger`	Swap one component of a combined item (deterministic next) + recombine
Unbinding Totem	`unbinding_totem`	Strip a champ's items -> bench inventory, decomposed to components
Echo Acorn	`echo_acorn`	Add a bench copy of a champion (+`champion_copies`, T.22 levelling)
Glimmerdust	`glimmerdust`	Upgrade item -> `heartwood:` version (D.21 MVP: x1.5 modifiers at equip)
Reclaimer's Cache	`reclaimers_cache`	Salvage base components -> 10 Amber each

Heartwood (`heartwood:<id>`) is scaled at equip by `loadout._heartwood_scale` (mul/add modifiers x1.5; hooks/procs untouched -- D.21 MVP). `decompose()` reverses any item to base components (used by unbind/reforge).

CLI: `sim_run --interactive` opens a prep shell (`combine/equip/reforge/ unbind/echo/glimmer/salvage`) over a Run before the route walk.

> **Status: LIVING** -- the definitive "how to author a champion / enemy / boss kit" reference *plus* the balance/identity faults to watch. Must match the primitives in `src/game/registries.py`, `src/game/effects.py`, `src/game/events.py`, `src/game/status.py`, `src/game/piece.py`, and the mutator surface in `src/game/combat/context.py`; and the formulas in `src/game/scaling.py` / `src/game/content.py`. Audited by `/check` (every cited symbol/number must resolve; the prose rules are review-enforced). **Reconciled:** 2026-07-01 @ authoring-reference expansion (verified against T.36 roster).

3.10 Kit design conventions

This doc has two halves. **Part A -- How a kit is authored** is the mechanical reference: the decorators, the `EffectBundle` shape, the closure-per-combat pattern, the `ctx` mutator API, the event->payload map, dedup semantics, determinism, and the description layer. **Part B -- Faults to watch** is the hard-won balance/identity checklist, each rule paid for by a real misfit caught mid-design (mostly the T.36 Primordial rework + the 12-piece distribution re-axis).

Read Part A before writing *any* handler; read Part B before authoring or re-axising a kit.

3.11 Part A -- How a kit is authored

3.11.1 The content <-> combat boundary

`game/` is pure logic with zero Flet imports (V.1); combat is a pure function `resolve_combat(team, enemies, weather) -> BattleResult` (V.2). A kit is **declarative content** -- factories decorated into registries at import time -- that the engine looks up by string id and invokes through the single `CombatContext` mutator surface. Content never mutates combat state directly; it only calls `ctx` methods, which fire the event bus.

The compile path (src/game/loadout.py):

- compile_loadout(team, enemies, weather, seed=42, run_mods=None) (loadout.py:315) builds a Piece per model (piece_from_champion loadout.py:156, piece_from_enemy loadout.py:193), applies weather -> items -> traits -> passives -> augments as EffectBundles, and returns (pieces, bus, trait_activations).
- apply_bundle(target, bundle, bus, ctx=None) (loadout.py:122) is the *one* function that installs a bundle -- used both pre-combat by compile_loadout and mid-combat by ctx.register_bundle. It appends modifiers, applies statuses, appends granted_abilities (as ActiveSlots) and granted_traits, and bus.subscribe(hook)s every hook.
- Boss fights additionally call attach_map_effect(effect_id, ctx, seed) (loadout.py:228) **after** building the CombatContext and **before** engine.run(ctx) -- see tools/playtest/_common.py:105 resolve_boss_combat for the canonical wiring, and resolve.py:46.

3.11.2 Registering an ability

All decorators live in src/game/registries.py and populate module-level dicts that the engine reads by string id. **Unregistered ability ids gracefully no-op** -- cast_ability returns early if ABILITY_REGISTRY.get(id) is None (context.py:442-444).

Decorator	Registry	Factory signature	Returns
@register_active(id, *, mana_cost=None, max_mana=0, start_mana=0, priority=1)	ABILITY_REGISTRY	handler(ctx, actor, targets) -> None	--
@register_passive(id)	PASSIVE_REGISTRY	factory(owner) -> EffectBundle	EffectBundle
@register_item(id)	ITEM_REGISTRY	factory(owner) -> EffectBundle	EffectBundle
@register_trait(id)	TRAIT_REGISTRY	factory() -> list[TraitBreakpoint]	breakpoints
@register_run_action(id)	RUN_ACTION_REGISTRY	fn(run, *args) -> None (V.24, never enters combat)	--
register_active_simple(id, SimpleActive(...))	ABILITY_REGISTRY	synthesised handler	--

An active handler receives (ctx, actor, targets) where targets is the list of living enemies at cast time (context.py:462) -- most handlers ignore it and re-query via targeting helpers (primary_target, lowest_hp_ally, enemies_in_radius, furthest_enemy, ...).

3.11.2.1 Mana is per-ActiveSlot, not a Piece stat

Mana lives on each ActiveSlot (separate pool per slot -- piece.py:17), authored on the **ability def** via the mana kwargs on @register_active (T.29c, V.48):

- mana_cost -- cast threshold + amount spent per cast. Default DEFAULT_MANA_COST = 300_000 (registries.py:48).
- max_mana -- universal pool cap (regen/start/grant clamp to it). 0 normalizes to 2 * mana_cost (overload headroom, piece.py:39).
- start_mana -- seeds current_mana at combat start.

- `priority` -- unified rank (>=1): drives both the weighted-rank charge cycle and the <=1-cast-per-window pick.

```
@register_active("champ_ember_salamander.active", mana_cost=230_000, priority=2)
def ember_salamander_active(ctx, actor, targets):
    target = primary_target(actor, ctx)
    if not target:
        return
    ctx.deal_damage(actor, target, EMBER_SALAMANDER_DMG.eval(actor), SourceTag.ABILITY)
    ctx.apply_status(target, "burn", duration_ticks=secs(6), source_id=actor.id)
```

Omitting the kwargs uses the defaults (byte-identical for single-ability champs). You may also author a statline separately with `register_ability_mana(id, ...)` (`registries.py:72`).

3.11.2.2 `SimpleActive` shorthand

For "deal X + optional heal" abilities, skip the handler and register declaratively (`registries.py:171`): `SimpleActive(target, damage, scaling, tag, heal_amount, heal_scaling, heal_target)`. `scaling` is an `_eval_scaling` expression like `"strength*1.5"` (stat aliases `str/int/atk/spd/mr/arm/res/pen` are accepted). Example from `reference.py`: `register_active_simple("smash", SimpleActive(target="primary", damage=100.0, scaling="strength*1.5", tag=SourceTag.ABILITY))`.

3.11.3 The `EffectBundle` -- the registration payload

Passives/items/traits/augments/boss-phase-hooks all return an `EffectBundle` (`effects.py:180`). It is a descriptor handed to `apply_bundle` once, not a runtime value:

```
@dataclass
class EffectBundle:
    modifiers: list[Modifier]        = []   # static/timed stat deltas
    hooks: list[Hook]                = []   # event subscriptions
    statuses: list[tuple[str, int]]  = []   # (status_id, duration_ticks) applied at install
    granted_abilities: list[str]     = []   # extra ActiveSlots
    granted_traits: list[str]        = []   # extra trait ids
```

A pure static buff needs only `modifiers` (`goldcrest_lark.passive`):

```
@register_passive("champ_goldcrest_lark.passive")
def goldcrest_lark_passive(owner):
    return EffectBundle(modifiers=[
        Modifier("intelligence", "add", 10.0, Lifetime.COMBAT, "passive:champ_goldcrest_lark"),
    ])
```

3.11.3.1 `Modifier` -- declarative stat delta

`Modifier(stat, op, value, lifetime=Lifetime.COMBAT, source_id="", expires_at_tick=None)` (`effects.py:45`). `op` in `{"add", "mul", "set"}`. `compute_stat` folds `(base + Σadds) x Πmuls`, and a `set` overrides everything (last `set` wins) (`effects.py:86`). A per-stat floor clamp applies at the tail (`attack_range >= 1`, V.43).

- `Lifetime` (`effects.py:19`): `COMBAT` (default -- cleared at combat end), `TIMED` (removed when `current_tick >= expires_at_tick`, so `TIMED` **requires** `expires_at_tick=ctx.current_tick +`

N), PERMANENT (survives across combats -- reserve for augments; nothing else should persist, see Fault #11).

- **Stat keys** (the real base_stats vocabulary, loadout.py:160-173): strength, intelligence, attack_speed, move_speed, mana_regen, threat, armor, resistance, attack_range, crit_chance, penetration, penetration_pct, plus hp. The primary damage stats are strength and intelligence -- there is no attack_damage/ability_power/mana_max stat. max_hp is a Piece *attribute*, not a base_stats key, so piece.stat("max_hp") is 0 -- read owner.max_hp directly, or use a PctResource (below).

3.11.3.2 source_id tagging (V.45)

Every Modifier.source_id should carry a <prefix>: group so the stat-breakdown UI can attribute it. The canonical prefixes and order (effects.py:109) are: item:, augment:, passive:, trait:, weather: -- with ability:<id> used for a cast-applied buff. Unknown prefixes sort last. The breakdown telescopes so rows sum exactly to the effective total.

3.11.4 Hooks -- subscribing to the event bus

Hook(event, handler, priority=0, scope=HookScope.PER_HIT, hook_id="") (effects.py:164). Hooks fire synchronously inside the ctx mutator that produced the event; reentrancy is the Python call stack. Higher priority fires first (effects.py:217). Nearly every hook handler must **filter to its owner first** (if event.attacker is not owner: return) because the bus is global across all pieces.

Two handler shapes:

- **Normal hook** -- handler(ctx, event) -> None. Fired via bus.fire(...).
- **Reducing hook** -- only on_damage_pre, fired via fire_reducing(...). Signature handler(ctx, event, value) -> float; return the (possibly modified) numeric value. Used to scale/reduce a hit before mitigation (aegis_tortoise.passive returns value*0.8 for adjacent attackers; nerei.passive amplifies by grudge stacks).

3.11.4.1 Which event fires which payload

Payloads are plain dataclasses in src/game/events.py. The engine fires these exact strings (confirmed across combat/*.py + context.py):

Event string	Payload	Key fields
on_combat_start	CombatStartEvent	(fires once, after all bundles applied)
on_combat_end	CombatEndEvent	winner ("team"/"enemy"/"draw")
on_tick	TickEvent	tick -- *use sparingly*
on_cast, on_cast_complete	CastEvent	caster, ability_id, cast_id, slot_idx, mana_cost, mana_after
on_attack_start, on_attack_landed	AttackEvent	attacker, target, amount -- **NO** .tag
on_damage_pre (reducing), on_damage_dealt, on_damage_taken, on_ability_damage	DamageEvent	attacker, target, amount, tag (str), cast_id, hit_id, is_crit, damage_type, is_dot
on_heal	HealEvent	source, target, amount

Event string	Payload	Key fields
`on_status_applied`, `on_status_expired`	`StatusEvent`	`target`, `status_id`, `duration_ticks`, `stacks`
`on_kill`	`KillEvent`	`killer`, `victim`
`on_death`	`DeathEvent`	`victim`, `killer` (may be `None`)
`on_spawn`	`SpawnEvent`	`piece`, `position`
`on_despawn`	`DespawnEvent`	`piece` (summon expiry -- **not** a death, G6)
`on_footprint`	`FootprintEvent`	observer-only cast geometry telemetry (V.61)

The `AttackEvent` / `DamageEvent` split is a common trap. `on_attack_landed` carries an `AttackEvent` with only `attacker`/`target`/`amount` -- it has **no** `.tag`. If you need the `SourceTag`, the `damage_type`, `is_crit`, or `is_dot`, hook `on_damage_dealt`/ `on_damage_taken`/`on_damage_pre` which carry a `DamageEvent` (whose `.tag` is the *string* value, e.g. compare against `SourceTag.REFLECT.value`, as in `holloway.cinder_husk`).

Two payloads are **defined but not wired by the engine**: `ManaEvent`/`on_mana_full` (no engine `fire`), and `PhaseEvent`/`on_phase_change` -- the latter is fired *by content*, from a boss phase hook via `ctx.fire("on_phase_change", PhaseEvent(...))` and by `map_effects.py`, not by the engine loop. Do not subscribe expecting the engine to raise them.

3.11.4.2 `HookScope` dedup (`effects.py:152, 269-302`)

The bus dedups by scope so a single logical effect doesn't multi-fire:

- `PER_HIT` -- never deduped; fires on every matching event (the default).
- `ONCE_PER_CAST` -- one fire per `cast_id`; needs a cast in flight (no-op filter if `cast_id` is `None`). Cleared when the cast completes (`clear_cast`).
- `ONCE_PER_TARGET` -- one fire per `(cast_id, target)`.
- `ONCE_PER_COMBAT` -- one fire for the whole battle; ledger cleared at `reset_combat`.

Use `ONCE_PER_COMBAT` for on-combat-start buffs (`sunlit_vigor`) and once-ever phase gates. Use `ONCE_PER_CAST` for "react to my own heal without recursing" (`dawnwisp.passive`) or "once per damage-taken event" ramps (`sunmane_lion.passive`).

3.11.5 The closure-per-combat pattern

A passive factory is called **once per combat** (pieces rebuild every `resolve_combat`, V.2), so per-combat mutable state lives in a **closure dict** the hook closes over. This is how counters/flags/timers persist within a fight and reset between fights *by construction*:

```
@register_passive("champ_veldt_pronghorn.passive")
def veldt_pronghorn_passive(owner):
    state = {"count": 0}                    # per-combat state
    def hook(ctx, event):
        if event.attacker is not owner:
            return
        state["count"] += 1
        if state["count"] % 3 == 0:         # deterministic cadence -- every 3rd auto
```

```
            ctx.deal_damage(owner, event.target, event.amount * 0.5,
                            SourceTag.BASIC_ATTACK, damage_type="physical")
    return EffectBundle(hooks=[Hook("on_attack_landed", hook, scope=HookScope.PER_HIT)])
```

Common closure idioms: cadence counters (`% N`), "empower next auto" flags across the active->passive boundary (`mirage_caracal.passive`, or via a `soul_charged` marker status), and periodic-tick timers keyed on `ctx.current_tick - state["last_tick"] >= interval` (`springfrog.passive`).

3.11.6 The `ctx` mutator API (`src/game/combat/context.py`)

A handler/hook touches the world **only** through these methods. Damage/heal/status all fire their own downstream events (so a `deal_damage` can trigger another piece's `on_damage_taken`).

- `deal_damage(attacker, target, amount, tag, *, crit=None, damage_type="magical", is_dot=False) -> float` -- the core damage pipeline (`context.py:194`): raw x weather-affinity clash x crit -> `on_damage_pre` (reducing) -> mitigation -> apply (barriers soak first) -> `on_damage_dealt` -> `on_damage_taken` -> `on_ability_damage` (if `tag == ABILITY`) -> kill check. `damage_type` is a **closed vocabulary** `{"physical", "magical", "true"}` (V.58) -- an unknown string raises. `physical`->armor, `magical`->resistance, `true`/`SourceTag.TRUE`-> unmitigated. `crit=None` lets the deterministic crit cadence decide (see below). Returns the final amount dealt (use it to scale lifesteal/reflect -- `sunmane_lion`, `riptide_caiman`).
- `heal(source, target, amount) -> float` -- clamps to missing HP, honours `grievous` antiheal (`GRIEVOUS_HEAL_MULT = 0.5`), fires `on_heal`.
- `grant_barrier(target, amount, duration_ticks=0)` -- temp absorb pool, consumed before HP, not counted in hp/max_hp (distinct from armor/resistance "shield" buffs).
- `apply_status(target, status_id, duration_ticks, stacks=1, source_id="", potency=0.0)` -- one instance per status per piece; raises on an unknown `status_id`. `cc_immune` pieces ignore hard-CC (gate-bearing) statuses. On merge the **stronger** `potency` **wins**.
- `remove_status(target, status_id)`, `apply_modifier(target, modifier)`.
- `trigger_basic_attack(attacker, target, mult=1.0)` -- resolves an auto: fires `on_attack_start`, computes `(1.0-STR + 0.25-INT)-mult` as **physical** `BASIC_ATTACK` damage (`context.py:422`), fires `on_attack_landed`.
- `cast_ability(actor, slot_idx=0)`, `gain_mana(actor, amount)` (adds to **all** slots, clamped to `max_mana`), `teleport`, `spawn`, `expire_summon`, `kill`, `revive`, `end_combat`, `register_bundle(owner, bundle)` (mid-combat install), `fire(event, payload)`, `note_footprint(...)` (observer-only view telemetry, V.61).
- Read-only queries: `enemies_of`, `allies_of`, `living_pieces`, `is_enemy`, `is_alive`, `both_sides_alive`, and properties `current_tick`, `current_cast_id`, `weather`, `rng`, `bus`, `board_state`.

3.11.6.1 The free-auto subsidy is baked into the engine

Every STR- or hybrid-stat piece has *live* autos worth `1.0-STR + 0.25-INT` per swing (`context.py:422`) whether or not its kit references them. This is the mechanical basis of Fault #4: it is why a STR/`ability` piece's ability coeff sits below its INT peer's.

3.11.7 Determinism (V.2 / V.14) -- non-negotiable

Sims must stay byte-identical, so **every "chance" / "every Nth" / ramp uses a deterministic cadence counter, never RNG**. The engine's own crit is the template (`context.py:246-250`):

`crit_counter += 1; cadence = max(1, round(1/crit_chance))`; crit and reset when the counter reaches cadence. Kits follow suit -- `veldt_pronghorn` fires every 3rd auto via `% 3`; `glade_heron`'s poison uses percentage decay (`decay_fraction`, `status.py:60`) to reach a deterministic plateau rather than a random shed. A `SeededRng` exists on `ctx.rng` for cases that genuinely need draws, but it must be seeded from the combat seed. Prefer cadence.

3.11.8 Statuses (`src/game/status.py`)

Statuses are id-based `StatusDefs` in `STATUS_DEFS`; `secs(x)` converts seconds -> ticks (`SECS = 100`, tick = 10 ms). `StackBehaviour` is `REFRESH` (reapply resets duration -- most CC) or `STACK` (poison/slow/sudden_death/stone_charge/nerei_grudge). `StatusGate` flags what a piece can't do: `BLOCKS_ACTION` (stun/frozen/fear), `BLOCKS_CAST` (silence), `BLOCKS_ATTACK` (disarm), `BLOCKS_MOVEMENT` (root/frozen), `HEXPROOF` (excluded from single-target acquisition; AoE still hits, V.40). Shipped statuses: `stun, silence, disarm, root, burn, poison, slow, charged, focus_fire, grievous, hexproof, taunt, soaked, frozen, fear, sudden_death, grief, stone_charge, soul_charged, nerei_grudge`. DOTs run on a free-running clock (`dot_interval_ticks`, default 100 = 1 s) that is **not** reset by reapply spam; `potency` overrides per-DOT-tick damage.

3.11.9 The description layer (T.34, V.38/V.46) -- `AbilityMeta` + `Magnitude`

The tooltip/combat-log presentation is a *parallel* registry `ABILITY_META` keyed by the same ability ids. The rule (source-of-truth B): a handler and its tooltip read the **same** `Magnitude` object, so their numbers can never drift. Author the headline number as a module-level `Magnitude`, call `.eval(actor)` in the handler, and pass the same object into the meta's `terms`.

The **closed** `Magnitude` family (`registries.py`, all pure + RNG-free, all self-rendering):

- `ScalingTerm(label, base, scaling="", note="")` -- linear `base [+ stat*coeff ...]`; `eval` reuses `_eval_scaling`. The workhorse (e.g. `ScalingTerm("damage", 100.0, "strength*1.5")`).
- `PctResource(label, pct, of="self"|"target", resource="max_hp", note="")` -- %-of-a-resource; reads the attribute **directly** (used precisely because `stat("max_hp")` is 0).
- `MaxOfTerm(label, coeff, stats=("strength","intelligence"), base=0.0)` -- `base + max(stats)*coeff` (the non-linear `max()` `ScalingTerm` can't express).
- `SetByCaller(label, base=0.0, coeff=1.0, key="stacks")` -- a runtime value the handler injects via `eval(caller={key: n})`; renders the *rate*.

`AbilityMeta(name, kind, blurb, terms=(), clauses=(), tags=())` -- `blurb` prose has `{label}` slots filled by the matching term; `clauses` are extra sentences (conditionals, cadences, status durations) that may carry their own `terms` via a `Clause(text=...)` or `Clause(template="...{token}...", terms=(...,))`. `tags` here are **UI-iconography** labels owned by this layer -- not the trait/role vocabulary.

```
DAWNWISP_HEAL = ScalingTerm("heal", 40.0, "strength*3.6")

@register_active("champ_dawnwisp.active")
def dawnwisp_active(ctx, actor, targets):
    ally = lowest_hp_ally(actor, ctx)
    if not ally:
        return
    ctx.heal(actor, ally, DAWNWISP_HEAL.eval(actor))   # handler reads the term
```

```
ABILITY_META["champ_dawnwisp.active"] = AbilityMeta(
    name="Knit Wound", kind="active",
    blurb="Mend the lowest-HP ally for {heal}.",          # tooltip reads the SAME term
    terms=(DAWNWISP_HEAL,), tags=("heal",),
)
```

3.11.10 Enemy kits

Enemies register with the same decorators; they are usually simpler (one active, one passive) and carry no items (`piece_from_enemy`, `loadout.py:193`). A summon uses a `SummonSpec` for the statline and `ctx.spawn` to place a full `Piece` (G6 -- summons are real pieces flagged `summon=True`, expiring via `expire_summon -> on_despawn`, never `on_death`):

```
_STEAM_TURRET = SummonSpec(stats={
    "max_hp": PctResource("max_hp", 0.25),
    "intelligence": ScalingTerm("intelligence", 0.0, "intelligence*0.79"),
    "armor": 20, "resistance": 20, "attack_speed": 80, "attack_range": 3,
    # ...flat literals pass through verbatim...
})

@register_active("enemy_steam_engineer.active")
def steam_engineer_active(ctx, actor, targets):
    from src.game.piece import Piece
    turret = Piece(id=f"{actor.id}_turret_{ctx.current_tick}",
                   base_stats=_STEAM_TURRET.eval(actor), affinity=actor.affinity,
                   is_enemy=actor.is_enemy, summon=True, summon_owner_id=actor.id,
                   summon_expires_tick=ctx.current_tick + 1200)
    turret.hp = turret.max_hp = turret.base_stats["max_hp"]
    ctx.spawn(turret, actor.position_q + 1, actor.position_r)
```

3.11.11 Boss kits -- phase hooks + map effects

Bosses add two things on top of a normal kit. A **phase hook** is a passive whose hook watches `on_damage_taken` (guarded by a `state["triggered"]` flag + `ONCE`-style scope), swaps the boss's `actives`, installs a phase-2 passive with `ctx.register_bundle`, and announces the transition with `ctx.fire("on_phase_change", PhaseEvent(...))`:

```
@register_passive("holloway.phase_hook")
def holloway_phase_hook(owner):
    state = {"triggered": False}
    def hook(ctx, event):
        if event.target is not owner:
            return
        if not state["triggered"] and owner.hp_pct <= 0.5:
            state["triggered"] = True
            owner.actives = [ActiveSlot(ability_id="holloway.magma_heave", mana_cost=...)]
            ctx.register_bundle(owner, holloway_cinder_husk(owner))       # phase-2 passive
            ctx.fire("on_phase_change", PhaseEvent(piece=owner, new_phase=2))
    return EffectBundle(hooks=[Hook("on_damage_taken", hook, scope=HookScope.PER_HIT)])
```

The **map effect** is attached separately (not a bundle): `attach_map_effect(effect_id, ctx, seed)` after building the context, before `engine.run(ctx)` (`loadout.py:228`; wiring in `tools/playtest/_common.py resolve_boss_combat`). It subscribes its own hooks to `ctx.bus` and writes `ctx.board_state`. On-death bursts are just a passive hooking `on_death` (`holloway.boiler_burst`).

3.12 Part B -- Faults to watch

Each rule below was paid for by a real misfit caught mid-design. Read before authoring or re-axising a kit.

3.12.1 Identity & role

1. **The Calling fixes the *playstyle*; the stat stays flexible.** Cast-Callings (Channeler / Mystic / Multicaster / Warden / Mender) -> `ability`. Auto-Callings (Hunter / Skirmisher / Stalker / Bruiser) -> `auto`. Guardian = tank, *leans* cast but soft. **Never put a Channeler in an `auto` cell or a Hunter in an `ability` cell** -- the kit reads wrong even when the numbers are fine. (T.36 caught this in 3 of 6 kings + 4 of 12 distribution pieces; the original plan draft had them backwards.) When a target cell needs a playstyle the Calling fights, the cheap fix is a **minimal, lore-natural Calling tweak** (add an auto-Calling), not a forced playstyle.

2. **Hold `intent` -> preserve the role. Flex `stat` + `playstyle` freely.** `classify_role(stat, reach, durability, playstyle, speed, intent)` -- `intent` (with durability) is the dominant role lever. Changing stat/playstyle reshapes *how* a piece fights; changing intent silently changes *what role it is*. A Hunter at `intent=utility` is still a **support** that happens to auto-attack -- not all auto-Callings are dealers. Verify `build_role_code` before/after any re-axis; if the role moved and you didn't mean it to, you changed intent by accident.

3. **Don't erase a piece's soul when you re-axis it -- relocate it.** dusk_bat (debuff-support) flipped to `str/auto` kept its blind by moving it *onto the autos*. phantom_lynx kept its pen/ghost identity by moving "ignore defense" onto a true-damage empowered auto. The mechanic moves to the new playstyle; the fantasy stays.

3.12.2 Coefficients & scaling

4. `str/ability`**: the ability is the main value; the STR coeff sits *below* the INT baseline.** The free auto-attack tagalong already pays STR (autos are `1.0-STR + 0.25-INT`, `context.py:422`). Parity vs the INT coeff it replaces: `coeff_str ~= coeff_int - 0.667-(autos_per_cast)` -> a ranged caster lands ~1.1-1.7; AoE+CC sits at the low end (the CC is the payoff). **Not** the old "ability empowers autos" steroid. (Mournhollow's naive `STR-2.7` was ~2x over.) **The free-auto subsidy is universal -- it applies to casters and supports too, not just carries.** Any STR- or hybrid-*stat* piece has live autos (`1.0-STR` base) that out-chip an INT peer's dead `0.25-STR` autos *for free*, even at a caster's low attack_speed (measured: a hybrid-stat support out-chips an INT support by ~5-18 DPS from the stat alone). So a STR/hybrid-stat **support's** ability coeff must also be discounted vs its INT peers, or it out-budgets them (T.36b: `dawnwisp` flipped int->str support with its heal discounted `INT-4.55->STR-3.6`, ~0.8x for the subsidy -- see the `DAWNWISP_HEAL` snippet above). The naive `int/ability -> str/ability` stat-swap that keeps the old INT coeff (mirewarden, hollow_elk had `INT-3.3-3.9`) is the same landmine -- drop the coeff hard on the swap.

4b. **When a re-axis produces a role that *misrepresents* the piece, suspect a taxonomy hole -- don't force a misfit.** A ranged playstyle-`hybrid` damage dealer (casts *and* autos) classified

as `marksman` because `classify_role` lumps hybrid-playstyle with auto. That was a real hole -- the ranged analog of `spellblade` (which is *stat*-hybrid). Fix = a new role (`Spellslinger`), not a contorted axis. Check `build_role_code/classify_role` on every re-axis; an off-reading role is a signal.

5. `*/auto` **(str/auto, int/auto, hybrid/auto): the autos must carry -- fold the active's payoff *into* an auto.** Patterns: on-hit proc (`int/auto` = on-hit-INT, no STR -- see `mirewarden_toad.passive` routing `intelligence*0.7` onto its autos), empower-next-auto (Yorick-style, `mirage_caracal/phantom_lynx` via `soul_charged`), discharge-on-auto (`granite_gorilla`). If the piece "doesn't care about its autos," it isn't really an auto piece. Autos default to `1.0-STR + 0.25-INT` -- an int/auto piece with no INT-on-auto routing ships with **dead autos** (the real mirewarden bug).

6. **Beware the hidden multiplier: per-event scaling x event count.** `charge += STR-k` *per blow* over `N` blows = an effective coeff of `k-N`, not `k`. A tank eats many hits -> `N` is large -> a normal-looking `k` becomes a hypercarry coeff, and stat items then scale it. **Fix:** keep `k` low *and* hard-cap the accumulator (`charge <= STR-1.5`) so the `N` multiplier can't run -- leaving linear stat scaling. Same family as the Aurion bug (`+1 primary/tick` -> ~600% over a fight, fixed by a cast-driven ramp capped at `_AURION_STACK_CAP = 8`, `champions.py`); both are unbounded per-tick/per-event ramps. Always bound a ramp by stacks or a cap.

7. **Size penetration against the actual resistance ceiling.** Penetration is a **global attacker stat** (`penetration` / `penetration_pct`, `context.py:269-270`), *not* per-hit -- you cannot scope it to one damage instance. Flat pen subtracts from mitigation, so it **zeroes any target whose res < pen**. Max resistance in the roster ~= **359** (T7 tanky_arm), typical big tank ~286, `MITIGATION_CONSTANT=100` (`context.py:49`). Size flat pen so it shreds tanks *partially* and never blankets the midfield (phantom's `INT-0.12` peaks ~49 ~= 14% of max res; `INT-0.3` zeroed everything under 123).

8. **True damage is premium -> lower its coeff than a magic hit.** `SourceTag.TRUE` / `damage_type="true"` bypasses **all** mitigation (`context.py:266`). Prefer it over penetration for an "ignore defense" finisher, but price it below a mitigated nuke.

3.12.3 Retaliation, sustain, persistence

9. **Never tie retaliation to a flat/% of *incoming* damage when HP pools are asymmetric.** A tank reflecting %-of-hit returns trivial damage to itself but *lethal* chip to a squishy -- worst case **a squishy dies faster attacking the tank than the tank dies**. Punishes the wrong axis. Use a banked-and-discharged model directed at the retaliator's *own* target, capped, scaled by the retaliator's own stat (not enemy damage). (granite_gorilla.) When you do reflect (`holloway.cinder_husk`), tag it `SourceTag.REFLECT` and early-return on an incoming `REFLECT` hit to prevent mutual-reflect recursion.

10. **Scope a diver's sustain to its own commitment, not a passive omni-stat.** A squishy auto-carry/swashbuckler gets lifesteal tied to its burst/true-strike (so it must commit and land to be rewarded), never free lifesteal on every hit. (phantom_lynx reaps only off the empowered true-damage auto; `riptide_caiman` heals a share of its Death Roll's dealt damage via the `deal_damage` return value.)

11. **Piece stat-stacking is in-combat only; cross-`Run` permastacking is augment-exclusive.** Combat is a pure function (V.2) -- all piece runtime state rebuilds per `resolve_combat`, so nothing persists across battles by construction. Use `Lifetime.COMBAT/TIMED`,

never `Lifetime.PERMANENT`, for a kit modifier. Write blurbs as "until end of battle," never "permanently" (which mislabels an in-combat ramp).

3.12.4 Process & verification

12. **The analytic DPS / HP-DPS proxy is a smoke-detector, not a scale.** Single-target DPS *understates* AoE+CC casters (no AoE multiply, no CC value) and auto-carries with uncounted procs/steroids. Use it to catch a piece sitting wildly off-budget (it caught Mournhollow at ~147k). The real gate is the V.33 ±10% HP-DPS proxy
    - `tools/simulation/stat_edge.py` teamfight sims.
13. **"power" is reserved.** `scaling.power(T,L) = 2^((T-1)/3 + triplings(L))` is the abstract power scalar. The survivabilityxdamage worth proxy is **HP-DPS** -- never call it "power."
14. **Determinism is non-negotiable (V.2/V.14).** Every "every Nth" / "chance" / ramp uses a deterministic cadence counter (like `crit_counter`), never RNG -- sims must stay byte-identical. See Part A -> Determinism for the primitives.
15. **When reshuffling axis assignments, preserve the *destination cell multiset*.** The statxplaystyle grid marginals are the contract, not which piece sits where. Reassign freely to honor Callings as long as the multiset of target cells is unchanged -- the grid still lands exactly. (Both T.36a kings and T.36b distribution were corrected this way with zero grid impact.)

3.13 Rosters -- champions, enemies, bosses

Status: LIVING -- source of truth is `src/game/content.py` + `bosses/data.py`. Audited by `/check` (counts + axis marginals). **Scope:** the live roster invariants, the `def -> model` build, the six identity axes `compose_stats` turns into a stat block, and where the real data lives. **Reconciled:** 2026-07-01.

Stat blocks, names, and lore live in code + the frozen catalogs (`docs/design/content/champion_roster.md`, `enemy_roster.md`, `boss_roster.md`). Here = the invariants, the axis vocabulary, and the build path.

3.13.1 Counts (verified; `/check` re-checks against code)

- **60 champions** -- `len(CHAMPION_ROSTER)`; tiers 1-10 x 6 affinities (`clear/cloudy/mist/rain/snow/thunder`), 10 per affinity (`_validate_rosters`, `content.py:715`).
- **60 enemies** -- `len(ENEMY_ROSTER)`; 30 `CLEAR` + 6 for each of the other 5 weathers (`content.py:720`).
- **6 bosses** -- `len(BOSS_DEFS)` (`bosses/data.py`).

3.13.2 Source of truth (code, not this doc)

- `content.py` -- authored tuples `_CHAMPION_DEFS: tuple[ChampionDef, ...]` and `_ENEMY_DEFS: tuple[EnemyDef, ...]` are the design data. They're built into the runtime model dicts:
    - `CHAMPION_ROSTER: dict[str, Champion]`, `ENEMY_ROSTER: dict[str, Enemy]`, `ENEMY_DEF_BY_ID` / `CHAMPION_DEF_BY_ID: dict[str, <Def>]` (formation/encounter read the defs).

  - Accessors `get_champion(id)` / `get_enemy(id)`; level rebuilds `build_champion_at_level(id, level)` / `build_enemy_at_level(id, level)` for L1-L3 (`content.py:693`).
- `bosses/data.py` -- `BOSS_DEFS: dict[int, BossDef]` (keyed by stage index 1-6; kit, map effect, spawn).

3.13.3 Def -> model build path

Each def is a **compact identity declaration** -- six axis tokens plus content metadata -- and the runtime `Champion`/`Enemy` is *derived*, not hand-authored (`_build_champion` / `_build_enemy`, `content.py:383`):

1. `compose_stats(stat, reach, durability, playstyle, speed, intent, tier)` -> a full combat stat dict from the six axes at that tier (see next section).
2. `_assert_budget` -- `stat_overrides` may drift the five PRIMARY stats by at most **±15%** of their base budget (`content.py:365`).
3. `_apply_stat_overrides` -- flat per-stat nudges, applied **after** tier-scale, **before** level-scale; keys must be real stat keys (`ALL_STAT_KEYS`).
4. `level_scale_stats(base, tier, level)` -- scaling curve (no-op at L1).
5. `role` / `role_code` derived via `classify_role` / `build_role_code`; `active_abilities` resolved (explicit `abilities=` list, or auto-discovery -- see below); `passive_ability` = `"{id}.passive"`.

Ability discovery (T.29d, V.49): a def's `abilities=None` (the factory default) auto-discovers every registered `{id}.active`, `{id}.active2`, ... in sorted order (`discover_abilities`, `content.py:19`). An explicit list overrides (named kits, bosses, or `[]` for a deliberately ability-less stat-stick). Every resolved `active`/`passive` id must exist in its registry (CI-guarded -- see abilities.md).

Champion vs Enemy defs share the whole stat/ability/damage framework and differ only in operation and metadata:

	`ChampionDef`	`EnemyDef`
id prefix	`champ_...`	`enemy_...`
tagging	`traits: list[str]` from `ALL_TRAIT_TAGS` (Kinship U Calling), non-empty	`tags: frozenset[str]` from `ENEMY_TAGS` (`human/beast/corrupted/machine/spirit`)
operation	drafted / bought / levelled	spawned by encounter/formation
extra guard	tier-10 champions **must** carry `Primordial` (`content.py:661`)	--

The combat entity built from either is the runtime `Piece` (see ../systems/combat.md).

3.13.4 `compose_stats` -- the six identity axes (V.33)

`compose_stats` starts from `_BASE_STATS` and multiplies in each axis, then applies the intent stat-bias, then tier-scales. This is the whole numeric identity of a piece -- no per-stat hand-authoring beyond the ±15% `stat_overrides`.

Base stats (`_BASE_STATS`, `content.py:27`): `max_hp 600`, `strength 50`, `intelligence 50`, `armor 25`, `resistance 25`, `attack_speed 100`, `mana_regen 100`, `move_speed 100`, `threat 60`, `attack_range 2`, `crit_chance 0.0`, `penetration 0`, `penetration_pct 0.0`.

Apply order (`content.py:311`): primary-stat x reach x durability x playstyle multipliers -> reach sets a discrete `attack_range` -> **speed** -> **intent** bias (one fixed point) -> tier-scale.

Axis	Values	What it does (`content.py`)
stat	`str` - `int` - `hybrid`	Splits STR/INT. `str` = 1.8 STR / 0.2 INT; `int` = 0.2 / 1.8; `hybrid` = 1.0 / 1.0 (`_PRIMARY_STAT:43`).
reach	`melee` - `ranged`	`melee`: +HP/armor/res/AS, `attack_range = 1`. `ranged`: −armor/res/HP/AS, `attack_range = 3`. The one **required positional** axis (no `hybrid` value) (`_REACH:49`).
durability	`squishy` - `hybrid` - `tanky_hp` - `tanky_arm`	Trades HP/armor/res against STR/INT and `threat`. Tanks read STR/INT at **0.42** so a primary-stat tank no longer rivals an assassin's primary (T.35b, B.20) (`_DURABILITY:66`).
playstyle	`auto` - `hybrid` - `ability`	`auto`: x1.3 AS, x0.6 mana_regen. `ability`: x0.75 AS, x1.5 mana_regen, x0.85 threat (`_PLAYSTYLE:101`).
speed	7 tiers (below)	Tempo/throughput trade: faster => ↑AS + ↑move_speed, ↓primary_stat (softer per-hit/cast) (`_SPEED:113`).
intent	`damage` - `utility` - `hybrid`	Power-**neutral** re-flavour (V.33): `damage` biases toward STR/INT/AS at the cost of HP/armor/res/threat; `utility` the reverse; `hybrid` is identity (`_INTENT:130`).
tier	`1..10`	Tier-scale: PRIMARY stats on `sqrt(power)`, SECONDARY on a gentle curve; `attack_speed` stays float (see scaling / `scaling.py`).

3.13.4.1 Speed axis vocabulary (T.33b -- 7 levels, slow -> fast)

`_SPEED` (`content.py:113`). `hybrid` is the neutral centre (omitted from `role_code`); `classify_role` ignores speed. Finer granularity => distinct pieces rarely share an `attack_speed`.

Speed	attack_speed	primary_stat	move_speed
`moloch`	x0.70	x1.25	x0.70
`leaden`	x0.85	x1.12	x0.85
`heavy`	x0.92	x1.06	x0.92
`hybrid`	x1.00	x1.00	x1.00
`light`	x1.10	x0.95	x1.08
`swift`	x1.20	x0.90	x1.15
`blazing`	x1.35	x0.82	x1.25

> **Vocabulary note (LIVING):** these seven tokens (`moloch/leaden/heavy/hybrid/light/swift/blazing`) are the **current** speed vocabulary -- the axis was renamed. Always verify against `content.py::_SPEED`; any other speed name predates the rename and is dead.

Caster tempo special-case (content.py:325): for playstyle="ability" the speed's attack_speed deviation is **halved and applied to BOTH** attack_speed **and** mana_regen -- a faster caster acts and refills mana quicker (more, softer casts), at half the swing since it lands on two stats. The primary_stat trade applies to whichever of STR/INT the piece keys (str/int/hybrid).

3.13.4.2 Ability mana cost

Ability cost is **uniform** -- no longer a per-unit def field. It is a constant (DEFAULT_MANA_COST = 300_000, registries.py:48) authored per-ability on the ActiveSlot (V.35/V.48), not a Piece stat.

3.13.5 Role taxonomy

A piece's role/role_code are **derived**, not authored. classify_role(...) maps the six axes to one of **9 coarse role titles** (ROLE_TITLES, content.py:161): tank, bruiser, support, mage, marksman, assassin, swashbuckler, spellblade, spellslinger. build_role_code(...) joins the six axis tokens in fixed order, omitting every hybrid, into the fine descriptor (a non-positional tag-set, V.32; never empty because reach is never hybrid). Spellslinger (ranged, playstyle-hybrid, damage) is the newest role (T.36b). See SPEC §V.31-V.33 and the T.32 role/intent system.

CALLING_TAGS is the frozen Calling trait-tag vocabulary Champion.traits draws from, alongside the 6 KINSHIP_TAGS (see traits.md).

3.13.6 Stat-axis balance (T.35b, #42 Finding B / B.20)

The stat axis x durability jointly set a unit's primary. T.35b re-tuned two axis tables in content.py so a primary-stat **tank no longer rivals an assassin's primary** (was: 1.8-str-axis x 0.55-tanky ~= 0.99 ~= a bruiser):

- _DURABILITY tanky_hp / tanky_arm strength/intelligence 0.55 -> 0.42.
- _INTENT damage strength/intelligence 1.08 -> 1.14, utility 0.94 -> 0.87 (defensive stats compensate so the V.33 HP-DPS proxy stays in [0.90,1.10]: damage 1.075, utility 0.947). Example: Coral Colossus STR 92->65, Duskstep Marten INT 127->134.

Dead-INT carriers fixed: every int/hybrid unit now reads INT in its kit -- per-role INT coefficients added to ~14 carriers' ability outlets (authored as Magnitudes, T.35a). Enforced by **§V.47** (test_axis_scaling_alignment): stat="int" must reference INT via a meta Magnitude, hybrid references both, str is auto-satisfied by the auto-attack. enemy_steam_engineer is allowlisted (its INT sizes the turret SummonSpec, not a meta outlet).

3.13.7 Axis-distribution rebalance (T.36)

The roster axes were rebalanced to principled marginals (a unified solve -- role is a pure fn of axes, V.32, so the marginals + soft role floors are the target and the role distribution is derived). Live counts (recompute from _CHAMPION_DEFS / _ENEMY_DEFS; ±2 of target is in-band -- **verified 2026-07-01**):

axis	champions	enemies
stat	str 22 - int 22 - hybrid 16	str 22 - int 22 - hybrid 16
playstyle	auto 24 - ability 24 - hybrid 12	auto 22 - ability 22 - hybrid 16
reach	melee 28 - ranged 32	melee 30 - ranged 30

axis	champions	enemies
durability	thp 11 - tarm 6 - squishy 13 - hybrid 30	thp 11 - tarm 7 - squishy 13 - hybrid 29
intent	damage 28 - utility 20 - hybrid 12	damage 29 - utility 19 - hybrid 12

Emergent enemy roles (T.36c -- curated by name/lore, opaque tags V.22): tank 12 - support 11 - spellblade 6 - bruiser 6 - mage 6 - swashbuckler 6 - marksman 5 - assassin 4 - spellslinger 4. New `Spellslinger` role (V.32, T.36b). All roles >=4. Re-axised kits honor V.46/V.47 (every int/hybrid enemy reads its primary stat; hybrid ability-users read both). Combined champ-vs-enemy **sim balance-validation DONE** [2026-06-17] (`tools/simulation/stat_edge.py`, team sims 2-5v5 x all weathers, `results/stat_edge_t36c.csv` n=8000 + iterate `_t36d.csv` n=1500): **zero champs over the** `|wr_delta|>0.10` **contract bar** after a 5-champ tune (mournhollow/veilfang_wolf/ember_salamander trimmed; aurion/will_o_fawn buffed -- `champions.py`). 44/60 inside ±0.05; the ±0.05 stretch on 3 stubborn over-performers (ember/mournhollow/veilfang ~+0.085, within n=1500 noise) is deferred to the full random-vs-random power sim. **D.25 reframe:** the residual STR-ability edge (+0.035) is STR-ability *over*-budget (free auto-tagalong `1-STR+0.25-INT`), not INT-ability under -- no further global INT bump warranted.

3.13.8 Invariants

- Counts + per-affinity splits above hold (`_validate_rosters`, `content.py:711`); `/check` recomputes them.
- Every `active_ability/passive_ability` id on a def resolves in its registry -- see abilities.md (CI-guarded).
- Champion traits subset of `ALL_TRAIT_TAGS`, non-empty; enemy tags subset of `ENEMY_TAGS`, non-empty (build-time asserts).
- Tier-10 champions carry `Primordial`.
- `stat_overrides` drift <= ±15% of PRIMARY budget (`_assert_budget`).
- §V.47 axis<->scaling: `int/hybrid` units reference INT in their kit (no dead INT).

3.13.9 File map

Concern	Symbol
Champion design data -> models	`content.py` (`_CHAMPION_DEFS`, `ChampionDef`, `_champion_def`, `CHAMPION_ROSTER`, `get_champion`)
Enemy design data -> models	`content.py` (`_ENEMY_DEFS`, `EnemyDef`, `_enemy_def`, `ENEMY_ROSTER`, `ENEMY_DEF_BY_ID`, `get_enemy`)
Axis -> stat block	`content.py` (`compose_stats`, `_BASE_STATS`, `_PRIMARY_STAT`, `_REACH`, `_DURABILITY`, `_PLAYSTYLE`, `_SPEED`, `_INTENT`)
Role derivation	`content.py` (`classify_role`, `build_role_code`, `ROLE_TITLES`, `CALLING_TAGS`, `KINSHIP_TAGS`)
Ability discovery	`content.py` (`discover_abilities`)
Tier / level scaling	`scaling.py` (`compose_stats` calls `stat_multiplier` / `level_scale_stats`)
Bosses	`bosses/data.py` (`BossDef`, `BOSS_DEFS`)

3.14 Abilities & passives -- id resolution, registration, hooks

Status: **LIVING** -- must match `src/game/abilities/` + `registries.py` + `effects.py/events.py`. Audited by `/check`. **Scope:** how a roster ability/passive id becomes a runtime handler, how a handler is registered, and the event/hook model handlers plug into. **Reconciled:** 2026-07-01.

Citations by symbol, not line. The *kits* (what each ability does, by lore) live in the frozen `docs/design/content/ABILITY_CATALOG_CHAMPIONS.md` / `ABILITY_CATALOG_ENEMIES.md`. The effect framework they plug into is ../systems/effects.md; the authoring/balance rules are ../systems/kit_design_conventions.md.

3.14.1 Id convention

A `Champion/Enemy` carries `active_ability` / `passive_ability` **string ids**, authored as prefixed ids in `content.py` (`active_ability = f"{id}.active"`, `passive_ability = f"{id}.passive"`, e.g. `champ_torrent_heron.active`). Handlers register under those exact ids -- the T.30 fix was aligning the two (previously short ids vs prefixed ids meant 0/240 resolved and every piece ran the generic fallback).

Multi-slot discovery (T.29d, V.49). A piece's active slots are auto-discovered by convention: `content.discover_abilities(piece_id)` regex-matches every registered `{piece_id}.active`, `{piece_id}.active2`, ... in sorted order (`.active` < `.active2` < ...). A champion with two registered actives (e.g. `champ_ember_salamander.active` + `.active2`) fields **two** `ActiveSlots` -- this is the Multicaster archetype (pieces with 2 active slots, each its own mana pool). There are **9** `.active2` secondaries in the roster (6 champion + 3 enemy). A def may override discovery with an explicit `abilities=` kwarg (named kits, bosses, null stat-sticks).

Bosses use named ids, not the `.active` convention. Each boss def lists its abilities explicitly (`abilities=[...]`) as `{boss}.{ability_name}` ids -- `holloway.pressure_vent`, `holloway.magma_heave`, `vance.sunflare_pounce`, plus a `{boss}.phase_hook` passive that drives the 2-phase transition (see "Boss phase hooks" below).

3.14.2 Registration

Handlers live in `abilities/champions.py`, `abilities/enemies.py`, `abilities/bosses.py` (+ `abilities/reference.py` for the shared/example kits that validate the pipeline) and register **at import time** via decorators from `registries.py`:

- `@register_active(ability_id, *, mana_cost=None, max_mana=0, start_mana=0, priority=1)` -> `ABILITY_REGISTRY[id] = handler`. The handler signature is `handler(ctx, actor, targets) -> None`. The optional mana kwargs author the ability's `AbilityMana` statline in the same call (see "Mana on the ability def").
- `@register_passive(passive_id)` -> `PASSIVE_REGISTRY[id] = factory`. The passive is a **factory** `factory(owner) -> EffectBundle` -- it returns a bundle of `Modifiers` + `Hooks` bound to `owner`, applied at loadout.
- `register_active_simple(id, SimpleActive(...))` -- declarative single-target / simple abilities with **no hand-written handler**. `SimpleActive(target, damage, scaling, tag, heal_amount, heal_scaling, heal_target)` synthesises a handler that resolves targets, deals `_eval_scaling(damage, scaling, actor)`, and optionally heals. `scaling` is an `_eval_scaling`

expression like "strength*1.5" or "intelligence*2.0+100" (stat aliases: str->strength, int->intelligence, atk->attack_speed, spd->move_speed, mr->mana_regen, arm/res/pen).

compile_loadout imports the abilities package (abilities/__init__.py imports reference, champions, enemies, bosses), which triggers every decorator, populating the registries before any combat. content.py imports the package the same way so the roster build can discover slots.

3.14.3 Mana on the ability def (T.29c, V.48)

Mana is **per** ActiveSlot, not a Piece stat. Each ability's mana statline lives in ABILITY_MANA: dict[str, AbilityMana], keyed by the same ids as ABILITY_REGISTRY:

- AbilityMana(mana_cost=300_000, max_mana=0, start_mana=0, priority=1) -- mana_cost is the cast threshold **and** the amount deducted per cast; max_mana=0 normalizes to 2 * mana_cost (overload headroom); start_mana seeds the pool at combat start; priority is the unified rank (charge cycle + cast pick, >=1).
- DEFAULT_MANA_COST = 300_000 is the baseline (former per-piece ability_cost, V.35). An ability with no entry uses the dataclass defaults.
- Author it inline on the decorator (@register_active("...", mana_cost=150_000)) or explicitly via register_ability_mana(id, mana_cost=..., priority=...).
- ability_mana(id) resolves the statline (defaults if unregistered); loadout seeds each ActiveSlot from it.

3.14.4 The hook model (event bus)

A passive/trait/item bundle carries Hooks that subscribe to combat events. The engine fires events on the EventBus; a Hook(event, handler, priority=0, scope=HookScope.PER_HIT) runs handler(ctx, event). HookScope dedups repeated fires: PER_HIT (every fire), ONCE_PER_CAST, ONCE_PER_TARGET, ONCE_PER_COMBAT.

The events the engine emits and their typed payloads (game/events.py):

Hook event	Payload	Key fields
on_combat_start / on_combat_end	CombatStartEvent / CombatEndEvent	--
on_tick	TickEvent	tick
on_cast / on_cast_complete	CastEvent	caster, ability_id, cast_id, slot_idx, mana_cost, mana_after
on_attack_start / on_attack_landed	AttackEvent	attacker, target, amount
on_damage_pre / on_damage_dealt / on_damage_taken / on_ability_damage	DamageEvent	attacker, target, amount, tag, is_crit, damage_type, is_dot
on_heal	HealEvent	source, target, amount
on_status_applied / on_status_expired	StatusEvent	target, status_id, duration_ticks, stacks
on_kill	KillEvent	killer, victim
on_death	DeathEvent	victim, killer
on_phase_change	PhaseEvent	piece, new_phase

Hook event	Payload	Key fields
on_spawn / on_despawn / on_footprint	SpawnEvent / DespawnEvent / FootprintEvent	--

on_damage_pre is a **reducing** hook (bus.fire_reducing) -- its handlers return a modified damage number (dodge, burst-reduction). The other damage events are observational. A passive typically guards on identity -- `if event.attacker is not owner: return` -- then acts via a ctx mutator (ctx.deal_damage, ctx.heal, ctx.apply_status, ctx.apply_modifier, ctx.grant_barrier, ...).

SourceTag (the event.tag / damage classification, effects.py): BASIC_ATTACK, ABILITY, ITEM_PROC, DOT, STATUS, REFLECT, TRUE. ITEM_PROC is the "follow-up hit" tag that does **not** re-fire on_attack_landed/on_ability_damage, so proc/secondary damage can't recurse; TRUE bypasses all mitigation.

3.14.4.1 Grounded example -- Ember Salamander (champ_ember_salamander)

- .active "Kindling Light" -- register_active_simple-style single-target nuke + applies burn.
- .active2 "Magma Burst" (mana_cost=150_000) -- a hand-written handler: picks primary_target, evals EMBER_MAGMA_BURST (a ScalingTerm), deals it to every enemy in radius 2 via enemies_in_radius.
- .passive "Smoldering Strikes" -- @register_passive returning an EffectBundle(hooks=[Hook("on_attack_landed", hook, scope=PER_HIT)]); the hook guards event.attacker is owner + event.target.has_status("burn"), then deals a ScalingTerm bonus.

3.14.4.2 Boss phase hooks

A boss's {boss}.phase_hook is a @register_passive returning a Hook("on_damage_taken", ..., scope=ONCE_PER_COMBAT): when the owner drops below the phase threshold it mutates the piece (grants abilities via owner.actives.append, applies buffs) and fires on_phase_change with a PhaseEvent(new_phase=2). The reference kit phase_hook_test in reference.py is the minimal template.

3.14.5 Counts (verified; /check re-checks)

- len(ABILITY_REGISTRY) = **144**, len(PASSIVE_REGISTRY) = **147** (shared handlers serve multiple roster ids; the guarantee below is per-roster-id, not per-handler).

3.14.6 The resolution guarantee (CI-guarded)

Every ability/passive id referenced by the roster must resolve to a registered handler -- enforced by guard tests (tests/game/test_ability_catalog.py: test_all_champion_abilities_resolve etc.; SPEC §V.15/§V.17). An unregistered id is a build failure, not a silent generic-fallback. The engine's unregistered-ability path (engine._resolve_action, which casts on full mana and deals damage **without** spending mana or firing on_cast) is only a defensive fallback, not the norm -- and cast-triggered trait/passive riders only fire for **registered** abilities.

3.14.7 Ability descriptions (T.34)

A parallel registry `ABILITY_META: dict[str, AbilityMeta]` (in `registries.py`) gives each roster ability id a tooltip. `AbilityMeta(name, kind, blurb, terms, clauses, tags)` is presentation metadata; `ability_text.render(meta, source) -> RenderedAbility(name, text, formula, tags)` is **pure** (no Flet/I-O, extends V.1) and reads live numbers via `source.stat(name)`. The same call serves both contexts -- a base `Champion`/`Enemy` (roster sheet, via the `.stat()` field adapters) and a live `Piece` (combat, with modifiers; **bosses always via the compiled** `Piece`). `render_for(id, source)` is the dict-lookup convenience.

Each scaling term renders its coefficients as **percentages of the source stat** so players read what a number scales from: `formula` lines read `550 = 100 + 150% STR + 150% INT  (STR 1.5x150, INT 1.5x150)` -- the trailing note shows each term's contribution math (`coeffxstat`), not the bare stat value, so the headline number is fully traceable -- and `text` carries a compact inline suffix beside the rendered total (`...550 magic damage. (100 +150% STR +150% INT)`). Pure-flat / no-scaling terms (buffs) add no suffix.

Source-of-truth B (V.38 + V.46): every stat-scaled outlet a handler computes lives **once** in a `Magnitude` the handler reads via `term.eval(...)` -- the tooltip renders the same object, so tooltip and combat numbers cannot drift. T.35a promoted `ScalingTerm` into a **closed** `Magnitude` **family** (modeled on Unreal GAS's `EGameplayEffectModifierMagnitude`), so there is **no** free inline handler math anymore -- the old "Tier-B stays inline + prose" carve-out is gone. The four kinds (all in `registries.py`, all pure/RNG-free, all self-describing via `eval`/`render_formula`/`render_inline`/`render_token`):

kind	shape	absorbs
`ScalingTerm`	`base + Σ source.stat-coeff` (delegates to `_eval_scaling`)	linear damage/heal/buff/shred
`PctResource`	`getattr(obj, resource)-pct`, `of="self"|"target"`	%-of-max-HP heals (reads `.max_hp` **directly** -- `Piece.stat("max_hp")` is 0, see `effects.compute_stat`)
`MaxOfTerm`	`base + max(source.stat(s)...)-coeff`	`max(STR,INT)` outlets (non-linear)
`SetByCaller`	`base + caller[key]-coeff` (handler injects the runtime value)	per-stack / runtime-count outlets

`Clause` carries an optional `{token} template` + its own `terms`, so a Tier-B scaler's prose number is filled from the same `Magnitude` the handler reads (A1) -- e.g. Hierarch's `Grants Armor ({armor})....` `SummonSpec` holds a summon's statline as `Magnitude` fractions + flat literals (`eval(owner) -> base_stats`), so summon stats are introspectable, not inline. **A2 guard (`test_no_orphan_stat_reads`, V.46):** an AST walk fails the build if any handler reads `.stat()`/`.max_hp` outside a `Magnitude` on its meta -- except ids on `_PROSE_ALLOWLIST` (flat `max_hp +=` growth: `champ_snowpelt_cub.passive`, `champ_glacierback_mammoth.passive`, `enemy_levyman.passive`). The conversion was **byte-identical** (sim digest unmoved, V.2/V.14); only the rendered `formula`/`text` of the 15 converted Tier-B abilities gained their scaling lines.

Tick -> seconds (V.39): `TICKS_PER_SECOND = 100` in `ability_text.py` is the single source for the `100 ticks = 1 second` display convention (matches `status.SECS = 100`). Mechanics stay ticks-only; only `ability_text.render` (and `ui/`, which imports the constant from here) converts. A coverage

guard + golden formula snapshot (`tests/game/ability_formulas.snapshot.json`) pin every rendered id; regenerate with `UPDATE_ABILITY_SNAPSHOT=1 uv run pytest tests/game/test_ability_text.py`.

Meta coverage spans **all 285 roster ability ids**: 120 champion (60 `.active` T.34a + 60 `.passive`) + 120 enemy (60 `.active` T.34b + 60 `.passive`) + 36 boss named-id T.34c + **9** multicaster `.active2` secondaries T.29d = 285 -- verified `len(ABILITY_META) == 285`. (Suffix histogram: `.active` 120, `.passive` 120, `.active2` 9, boss-named 36.) `tools/export_roster.py` serializes the champion/enemy (and optional boss) rosters with rendered descriptions to JSON.

3.14.8 File map

Concern	Symbol
Champion kits	`abilities/champions.py`
Enemy kits	`abilities/enemies.py`
Boss kits (2-phase, named ids + `.phase_hook`)	`abilities/bosses.py`
Shared/declarative templates	`abilities/reference.py`, `registries.register_active_simple`
Registries + decorators	`registries.py` (`ABILITY_REGISTRY`, `PASSIVE_REGISTRY`, `@register_active`/`@register_passive`, `register_active_simple`)
Mana on the ability def	`registries.py` (`ABILITY_MANA`, `AbilityMana`, `ability_mana`, `register_ability_mana`, `DEFAULT_MANA_COST`)
Hooks + events	`effects.py` (`Hook`, `HookScope`, `EventBus`, `SourceTag`, `EffectBundle`), `events.py` (typed payloads)
Slot discovery	`content.py` (`discover_abilities`), `piece.py` (`ActiveSlot`, `Piece.actives`)
Ability descriptions (T.34/T.35a)	`registries.py` (`ABILITY_META`, `Magnitude` family: `ScalingTerm`/`PctResource`/`MaxOfTerm`/`SetByCaller`, `Clause` w/ `template+terms`, `SummonSpec`, `AbilityMeta`), `ability_text.py` (`render` pure per-kind dispatch, `render_for`, `TICKS_PER_SECOND`)
id source on the model	`content.py` (`active_ability`/`passive_ability`)

3.15 Traits -- kinships, callings, breakpoints

Status: LIVING -- must match `src/game/traits/` + `content.py` trait vocab. Audited by `/check`. **Reconciled:** 2026-06-16 (T.28d + T.29d Multicaster).

[stub] **PARTIAL** -- T.28a/b/c/d built the framework + stat packs + the full reachable combat surface. T.28b combat primitives (second-wind, tidal HoT, enrage, time-ramp, dodge, hexproof opener; kiting, backline, taunt, revive). **T.28c (done)** added the rider batch over existing `ctx`: Hunter bonus-auto/empowered/cleave/team-aura, Mystic `ability_can_crit`+splash, Guardian start+periodic shields, Bruiser lifesteal, Skirmisher @8 team ramp, Stalker hi-HP-bonus/mana-on-kill/hexproof-after-kill, Channeler free-cast/first-cast-twice, Warden cast-shield+team-opener, Trickster slow+taunt-on-cast+mana-aura, Mender heal-splash/overheal-shield, Spirit echo, Swarm on-death

chitin spawn. **T.28d (done):** renamed `untargetable->hexproof` + fixed single-target acquisition to honor the gate (AoE still hits; `pierces_hexproof` bypass -- V.40, B.15); 5 affinity @10 riders (Galvanized crit-arc, Frostbound chill-attackers, Stormfed mana-haste [stat], Shrouded longer hexproof-opener, Overcast burst-reduction) + Sunlit premium-stat @10 pack; Scaled @5 hard-CC immunity + @8 favorable weather override; Spirit @8 reduced-potency echo (0.6x) + mana-haste + hexproof pierce; fold-ins Beast @4/@6 strength-ramp, Skyborn @3 kite-reward, Tidekin @3 ally-heal. **Deferred to T.31 (D.20):** 6 Primordial @1 signature mechanics (legendaries already carry full T.30 kits; signatures unreachable until the unlock augments)

- Primordial @3 tier-up. Primordial @1 ships as its stat pack only. Design (frozen): `docs/design/content/trait_catalog.md` v2.1.

3.15.1 Where it lives

- `game/traits/types.py` -- `TraitScope` (`PER_TRAIT_PIECE`/`TEAM_WIDE`), `TraitBreakpoint(count, scope, bundle_factory)`, `DynamicThreshold` (`callable(team, board_cap) -> int`).
- `game/traits/_packs.py` -- `stat_pack_bundle` (mul/add `Modifiers`; an `attack_speed` mul moves tie-order on its own now that AS is a float, V.34 / T.29-pre -- no `milli_AS` rider) + `define_trait` shorthand (registers a trait from `(count, scope, muls, adds)` rungs). `define_trait` also records each rung's raw `(label, muls, adds)` in `TRAIT_STAT_PACKS` so the description layer can derive a rung's stat line from the same numbers the bundle applies (T.41b, V.79).
- `game/traits/{affinities,kinships,callings}.py` -- the 25 trait factories (declarative stat-pack rungs). Imported by `traits/__init__`, which registers them into `TRAIT_REGISTRY` as a side effect.
- `game/traits/__init__.py` -- `affinity_trait` (affinity -> derived tag), `_resolve_traits`, `resolve_and_apply_traits`.
- `game/registries.py` -- `TRAIT_REGISTRY` + `@register_trait`.
- `content.py` -- `KINSHIP_TAGS` (6) / `CALLING_TAGS` (13) / `ALL_TRAIT_TAGS`; `Champion.traits`.

3.15.2 The 25 traits -- three families

A *trait* is a synergy tag whose breakpoints grant `EffectBundles` when enough tag-sharing **unique champions** are fielded. `len(TRAIT_REGISTRY) == 25`, split into three families:

- **6 Kinships** (`KINSHIP_TAGS`, authored on `Champion.traits`): Beast, Spirit, Skyborn, Scaled, Tidekin, Swarm. Each champion carries **exactly one** Kinship (the "what animal" axis). One **Tier-10 anchor per kinship** (see Roster below).
- **6 Affinities** (derived from `affinity`, **never stored** on `traits`): Sunlit/Overcast/Shrouded/Stormfed/Frostbound/Galvanized = Clear/Cloudy/Mist/Rain/Snow/Thunder. `traits/__init__.affinity_trait(affinity)` maps the piece's single `affinity` field (V.6) to its synthetic tag at resolution; it never reads node weather. Because affinity is the same axis the weather triangle uses, the affinity trait ladder is weather-independent.
- **13 Callings** (`CALLING_TAGS`, authored on `Champion.traits`): Hunter, Guardian, Mystic, Warden, Stalker, Bruiser, Skirmisher, Channeler, Mender, Trickster, Packmate, Primordial, **Multicaster**. Callings are the "how it fights" axis; a champion carries one or more.

`ALL_TRAIT_TAGS` (19) = Kinships U Callings -- the **authorable** vocabulary (affinities are excluded

because they're derived, not authored). content.py self-asserts every Champion.traits tag in ALL_TRAIT_TAGS and resolves in TRAIT_REGISTRY (V.22).

Multicaster (T.29d) is the quick-caster Calling on **multi-slot champs** (pieces with 2 active slots / a registered .active2; ~6-carrier pool). Rungs @2/3/4 are all PER_TRAIT_PIECE; each cast stacks cast_momentum (attack_speed

- mana_regen muls, capped) -- no team apex (apex = min(pool, cap), V.37). **Primordial** is the six Tier-10 legendaries' Calling; it's **augment-gated** (V.37) and ships with its stat pack live but its @1 signature mechanic + @3 tier-up deferred (D.20).

3.15.3 Rung authoring -- define_trait (_packs.py)

Every trait is a stack of TraitBreakpoint(count, scope, bundle_factory) rungs. define_trait(trait_id, *rungs) takes rung tuples (count, scope, muls[, adds[, hook_builders]]):

- count -- the min unique-carrier count to clear the rung. An int, **or** a DynamicThreshold callable(team, board_cap) -> int (Packmate's @full-board apex = lambda team, cap: cap).
- scope -- TraitScope.PER_TRAIT_PIECE (carriers only) or TEAM_WIDE (whole player team). _PER / _TEAM shorthands in the factory files.
- muls -- {stat: fraction}, 0.08 => x1.08; adds -- {stat: amount} flat. Built into Modifiers by stat_pack_bundle (mul->1+pct, add->flat, both Lifetime.COMBAT, source_id = "trait:<id>@<label>").
- hook_builders -- optional 5th element: a list of builders (owner, source_id) -> list[Hook] (the mechanic riders from mechanics.py), appended to the rung's bundle.

define_trait also records each rung's raw (label, muls, adds) in TRAIT_STAT_PACKS so the description layer derives a rung's stat line from the **same numbers** the bundle applies (T.41b, V.79 -- can't drift). Registration is a side effect: register_trait(trait_id)(lambda: breakpoints).

RNG-free cadence rule (V.2/V.14/V.37). Every "every Nth cast", "chance", or ramp in a rider uses a **deterministic cadence counter** or an HP/geometry threshold -- never RNG -- so sims stay byte-identical. Trait resolution itself is pure and counted once at loadout (below).

3.15.4 Resolution (in compile_loadout, step 3)

1. After weather (step 2), before passives (step 7).
2. _resolve_traits(team, board_cap) counts **unique champion ids** per tag (V.21), incl. each piece's **derived affinity tag**; finds the highest cleared rung (dynamic thresholds resolved against board_cap == len(team)).
3. Applies each cleared rung's bundle to its targets (PER_TRAIT_PIECE -> carriers; TEAM_WIDE -> whole team). **Player team only -- enemies never light up** (V.22).
4. **HP re-sync:** the engine reads piece.max_hp/hp as cached fields, so after applying hp-mul modifiers the function recomputes max_hp = hp = piece.stat("hp") (pieces start each combat at full HP).
5. Returns trait_activations: [(trait_id, count, threshold), ...] (sorted), surfaced through compile_loadout's 3-tuple return -> BattleResult.

Pure + RNG-free -> replay-stable (V.21). Counting is at loadout; mid-combat spawns/revives never raise a count.

3.15.4.1 Virtual carriers -- emblems & augments (V.21)

A tag's carrier count can be raised without a native carrier, deterministically:

- **Emblems (T.29b items):** an item's `granted_traits` adds a tag to the wearer. Item bundles apply in `compile_loadout` step 2.5, **before** trait resolution (step 3), so an emblem wearer counts toward the tag's Kinship/Calling count like a native carrier (it's a real piece with an extra tag).
- **Augment Crest/Crown/Worldroot (T.31):** injected as `bonus_counts: dict[str, int]` -- **virtual** carriers added to a tag's count in `_resolve_traits` *before* breakpoint selection. `bonus_counts` can light up a tag with **zero** fielded carriers (`carriers.setdefault(tag, set())`). Both `_resolve_traits` and the UI `preview_team_traits` accept `bonus_counts` so the preview matches what combat clears.

3.15.5 Breakpoint shapes (apex = `min(pool, cap)`, V.37)

Ladders are single-step-leaning with `@1` entries on supports/casters/kiters; see the catalog for per-trait rungs. **T.28a implements the stat-pack portion of every rung**; mechanic riders layer onto the same trait ids in b/c. Packmate's apex is a **dynamic** `@full-board` threshold (== fielded board size).

3.15.5.1 Cumulative rungs (load-bearing) -- V.41

Resolution applies **only the highest cleared rung's bundle** (one `TraitBreakpoint`, not a union). Stat magnitudes are authored as the **total at that rung** (they replace, not stack); mechanic riders persist **only if each higher rung re-lists them**. **V.41** guards this: `test_trait_rungs_are_cumulative_for_mechanics` asserts every mechanic fingerprint at rung N reappears at rungs > N -- sole exception, carrier-movement (kiting/backline) at a `TEAM_WIDE` apex. So each rung **re-includes every mechanic it should still grant** -- e.g. Skirmisher carries `time_ramp` on @2/@3/@4/@5/@8 and `dodge` on @4/@5/@8. Forgetting to re-include silently drops a lower mechanic at the higher count. **Apex exception:** carrier-*movement* mechanics (kiting, backline) are NOT re-applied at a TEAM apex -- applying them team-wide would make every ally kite/seek; apex trades the few-carrier identity for a team buff. Caster/role-gated riders (echo, splash, lifesteal, shields, on-death spawn) ARE team-safe (no-op for the wrong role) and stay; the Swarm on-death spawn is explicitly `trait="Swarm"`-guarded so a TEAM apply only spawns for Swarm pieces.

3.15.6 Combat primitives (T.28b) -- `game/traits/mechanics.py` + engine

All deterministic (cadence counters / HP thresholds / geometry, never RNG -- V.2/V.14/V.37). A rung's optional 5th tuple element is a list of builders `(owner, source_id) -> list[Hook]` appended to its bundle (`_packs.define_trait`).

Hook riders (ride the event bus, no engine edits):

- `second_wind` (Primordial @2) -- `on_damage_taken`; first drop below 60% HP -> `grant_barrier(0.4-max_hp, 1200t)` once. Reuses V.28 barriers (decay = expiry).
- `tidal_hot` (Tidekin @5/@8) -- `on_tick` cadence `ctx.heal` of the carrier.
- `enrage` (Beast @8) -- `on_damage_taken`; one-shot AS+STR burst below 25% HP.
- `time_ramp` (Skirmisher @2) -- `on_tick` stacking AS `mul` to a cap.

- `dodge` (Skirmisher @4) -- `on_damage_pre` reducing; every Nth basic -> 0 (engine floors final to 1, so a near-total mitigation).
- `hexproof_opener` (Spirit @5, Shrouded @10) -- `on_combat_start` applies `hexproof`.

Engine-behaviour arms (set a flag/status the engine reads):

- `kiting` (Skyborn @2) -- `on_combat_start` sets `Piece.is_kiter`; melee (base range <=1) also get +1 `attack_range` (no double vs @5's flat +1). Engine `_kite_step` (movement): retreat one hex from a **single** adjacent melee threat that strictly increases distance while keeping >=1 enemy attackable; plant when swarmed (>=2 melee), cornered, or no target.
- `backline_seeker` (Stalker @2) -- sets `Piece.seeks_backline`; engine paths to the deepest enemy column (`_backline_subset`) and `_select_target` prefers it. No teleport.
- `revive_first_ally` (Mender @6, TEAM_WIDE) -- `on_death`; the first ally death each combat -> `ctx.revive(victim, 0.3)`. Once-per-combat shared across all carriers via a flag on `ctx`. The one true revive (V.37); mid-combat revives never raise a trait count.
- **taunt** -- `StatusGate`-free `taunt` StatusDef; `StatusInstance.source_id` = taunter. Engine `_taunt_target` forces the taunter as target (overrides current/backline) and as the movement goal. Capability only in T.28b -- wired to Trickster casts in T.28c.

`StatusGate.HEXPROOF` (status `hexproof`, renamed from `untargetable` in T.28d) excludes a piece from **single-target acquisition** -- the engine `_opponents` auto-attack scan **and** every single-target helper in `targeting.py` (`primary_target`/`lowest_hp_enemy`/`highest_ap_enemy`/`random_enemy`/`furthest_enemy`/`_closest_enemy`). AoE/untargeted effects (`enemies_in_radius`, `line_targets`, a cast iterating `ctx.enemies_of`) **still hit** -- MTG-hexproof. A piece with `Piece.pierces_hexproof` (Spirit @8) ignores the exclusion. The piece still acts. (V.40, B.15)

3.15.7 Mechanic + apex riders (T.28c) -- `game/traits/mechanics.py`

All hook idioms over the existing `ctx` mutators (no new engine subsystems). Secondary/proc damage uses `SourceTag.ITEM_PROC` -- the existing "follow-up hit" tag that does NOT re-fire `on_attack_landed`/`on_ability_damage`, so no recursion. Recast/heal-splash guard re-entry with `ctx._in_recast` / `ctx._in_heal_splash`.

- Hunter: `bonus_auto_damage` + `empowered_shot` (cadence) + `cleave` (`on_attack_landed`); @8 TEAM aura.
- Mystic: `ability_crit` (flips `Piece.ability_can_crit`) + `ability_splash` (`on_ability_damage` -> adjacent enemies).
- Guardian/Warden: `start_shield` (`on_combat_start`), `periodic_shield` (`on_tick` self + adjacent allies), `cast_shield_lowest` (`on_cast`).
- Bruiser/Beast: `attack_lifesteal` (`on_damage_dealt`, basic only).
- Stalker: `high_hp_bonus` (`on_damage_dealt` vs >60%-HP target), `mana_on_kill` + `hexproof_after_kill` (`on_kill`).
- Channeler/Spirit: `free_cast` (refund every Nth), `recast_first`, `echo_cadence` (re-run via `ctx.cast_ability`, `_in_recast`-guarded).
- Trickster: `slow_on_cast`, `taunt_on_cast` (reuses the T.28b taunt status), `mana_denial_aura` (`on_tick` drains adjacent enemy slot mana).
- Mender: `heal_splash` (`on_heal` -> lowest-HP other ally), `overheal_shield` (heal to a near-full ally banked as a barrier).
- Swarm: `on_death_spawn` -- a dying Swarm leaves a stat-fraction chitin via the existing summon pattern (`Piece`+summon flags+`ctx.spawn`); `trait`-guarded.

- Packmate `@full-board`: the flat stat pack IS the apex payoff (no rider).
- Multicaster (T.29d): `cast_momentum` (`on_cast_complete` -> each cast stacks `+per attack_speed` mul + `+mr_per mana_regen` mul, capped) -- the quick-caster snowball for multi-slot champs.

Cast-triggered riders only fire for **registered** abilities (the unregistered fast-path in `_resolve_action` deals damage without firing `on_cast`).

3.15.8 Apex riders + arms (T.28d) -- `game/traits/mechanics.py`

All RNG-free. Affinity `@10` rungs (PER scope -- at @10 the board is mono-affinity):

- Galvanized: `crit_arc` (`on_damage_dealt` + `is_crit` -> arc to a neighbour, ITEM_PROC).
- Frostbound: `chill_attackers` (`on_damage_taken` -> slow the attacker).
- Overcast: `burst_reduction` (reducing `on_damage_pre`, cap a hit at 25% max-HP).
- Shrouded: `hexproof_opener(300)` (longer team opener).
- Stormfed: mana-haste = a fatter `mana_regen` stat in the @10 pack (no hook).
- Sunlit: **stat-only** @10 -- broadened to premium stats (`crit_chance`/`penetration_pct` flat adds + `mana_regen` mul), deliberately small.

Kinship arms + fold-ins:

- Scaled: `cc_immunity` (@5/@8 -- sets `Piece.cc_immune`; `ctx.apply_status` skips hard-CC, i.e. any `BLOCKS_*`-gated status; slow/DoTs still land). **Signature -> carrier-guarded at the @8 TEAM rung** (`cc_immunity(trait="Scaled")`) so the team-wide armor/res pack still blankets the squad but CC-immunity stays a Scaled perk. @8 also marks `Piece.weather_favored` via the **pre-weather pass** `mark_weather_overrides` (loadout step 2-pre, carrier-guarded) -> `_apply_weather_to_piece` uses `WEATHER_BUFF_BASE[weather]` regardless of affinity.
- Spirit @8 (TEAM): `echo_cadence(3, potency=0.6)` goes team-wide (caster-gated, the apex buff) -- reduced-potency echo via transient `ctx._echo_potency`, read in `deal_damage` (damage-only). The `hexproof_opener` (re-included, cumulative) + `pierce_hexproof` are **Spirit signatures -> carrier-guarded** (`trait="Spirit"`). `mana_regen` haste (stat) is team-wide.
- Beast @4/@6/@8: `time_ramp(stat="strength")` (re-included up the ladder with enrage @8).
- Skyborn: `kiting` re-included on the @3/@5 PER rungs (cumulative -- was dropped past @2, **B.16**); **not** re-applied at the @8 TEAM apex (movement exception). `kite_reward` (`on_damage_dealt`, bonus vs a target whose `attack_range` < gap) on @3/@5/@8 (team-wide damage buff at the apex).
- Tidekin @3/@5/@8: `ally_tidal` (cadence-heal the lowest-HP ally; distinct from the carrier `tidal_hot` at @5/@8).

Carrier-scope principle (T.28d): at a `TEAM_WIDE` apex, stat packs + event/role-gated riders (echo, lifesteal, shields, ramp, splash, sustain) apply team-wide -- that IS the apex trade. **Signature/identity effects** (CC-immunity, hexproof opener/pierce) and **carrier-movement** (kiting/backline) stay carrier-only -- trait-guarded (`cc_immunity(trait=...)`, `pierce_hexproof(trait=...)`, `hexproof_opener(trait=...)`) or omitted from the apex, mirroring the `on_death_spawn` `trait`-guard.

Deferred to T.31 (D.20): Primordial @1 signature mechanics + @3 tier-up.

3.15.9 Roster (post-T.28a rebalance, B.9)

- Kinship pools (sum 60): Beast 14, Spirit 11, Skyborn 9, Scaled 9, Tidekin 9, Swarm 8. One **Tier-10 anchor per kinship**: Mournhollow->Beast, Aurion->Spirit, Aerion->Skyborn, Umbra->Scaled, Nerei->Tidekin, Borealis->Swarm.
- Calling pools (sum 96): Bruiser 10; Guardian 9; Hunter/Mystic/Skirmisher/Packmate/Stalker 8; Channeler 7; Warden/Trickster/Mender/Primordial/Multicaster 6.
- `CALLING_TAGS` dropped the 4 dead T.5 tags (Bulwark/Drifter/Harbinger/Emissary) and added **Packmate** (8 cheap T1-3 secondary carriers). Hunter spread toward lower tiers (e.g. Dusk Bat T2). Primordial shop access is augment-gated (T.31); trait factories ship ready-but-dormant.

3.15.10 Descriptions (T.41b) -- `game/traits/meta.py` + `game/describe.py`

Player-facing trait text flows through the shared description render-layer:

- `traits/meta.py::TRAIT_META: dict[str, TraitMeta]` -- authored **name + blurb
  - per-breakpoint `text`** for all **25** traits (transcribed from `trait_catalog.md`, reconciled to the **code's** counts -- a few drifted during T.28b/c balance: **Bruiser** apex @10 not @8, **Stalker** apex @8 not @7). Each trait's `rungs` keys **== its `factory()` breakpoint counts** (V.79, test-guarded).
- `describe.render_trait(id) -> RenderedTrait(name, blurb, rungs)` -- each `RenderedRung(count, text, stat_line)` pairs the authored effect text with a **stat line derived from** `_packs.TRAIT_STAT_PACKS` (the rung's `muls/adds`, never re-typed -> can't drift, V.79). Pure, no Flet/mutation (V.80).
- **Consumer:** `ui/components/trait_synergies.py::trait_synergies_panel` tooltips (Prep **and** Combat) -- trait blurb + every breakpoint (`* cleared / o`) with its **scope** (`[carriers]` = tag-sharers only / `[team]` = whole team, from the rung's `TraitScope`), stat line, and effect text.

3.15.11 Invariants

- V.21 (unique-id count, RNG-free, replay-stable), V.22 (every `Champion.traits` tag resolves in `TRAIT_REGISTRY`; >=1 Kinship + >=1 Calling; Primordial at T10), V.37 (apex/dynamic-threshold + primitive determinism), **V.40 (hexproof targeting -- single-target acquisition excludes `HEXPROOF`; AoE still hits; `pierces_hexproof` bypass; B.15)**, **V.41 (cumulative rungs -- a higher rung re-includes every lower rung's mechanic; sole exception carrier-movement at a TEAM apex; B.16)**, **V.79 (every trait has `TRAIT_META` with one description per `factory()` breakpoint; stat line derived from `TRAIT_STAT_PACKS`; T.41b)**. Guarded by `tests/game/test_traits.py` + `tests/game/test_describe.py` + `tests/game/test_hexproof.py` + `content.py` self-asserts.

3.16 Items

Status: LIVING -- must match `src/game/items/`. Audited by `/check`. **Scope:** content pointer -- the item **engine** lives in the system doc; this doc holds the roster shape + counts + the description layer. **Reconciled:** 2026-07-01.

[done] **SHIPPED (T.29a-d)** -- the item engine is built. `ITEM_REGISTRY` holds **50** combat items; `RUN_ACTION_REGISTRY` holds **5** special run-actions. Mana primitive (V.48)

+ multi-slot (V.49) landed in T.29c/d. Descriptions (T.41a) flow through the shared render-layer.

3.16.1 Roster shape + counts (verified 2026-07-01 against the registries)

`ITEM_REGISTRY` = **50** combat items (`id -> Callable[[Piece], EffectBundle]`):

Group	Count	File	IDs
Base components	8	`base.py` (`BASE_COMPONENTS`) + factories in `combined.py`	`fang`, `talon`, `heartseed`, `springtear`, `old_hide`, `stoneplate`, `wardpelt`, `keen_claw`
Same-component combines	8	`combined.py`	e.g. `apex_fang` (fang+fang), `deepwell` (springtear+springtear)
Cross-component combines	28	`combined.py`	e.g. `bloodthorn_briar` (fang+heartseed), `splitwind_talons` (talon+wardpelt)
Kinship emblems	6	`emblems.py`	`beast/skyborn/scaled/tidekin/swarm/spirit_emblem`

So `combined.py` registers **44** (8 raw + 16 core-cut + 20 T.29b combined) and `emblems.py` adds **6** -> 50. The 36 same+cross combines are keyed by `RECIPE_MAP` (`recipes.py`); the 6 emblems are crafted via `combine(spirit_gem, c)`.

`RUN_ACTION_REGISTRY` = **5** special run-actions (`special.py`) that never enter combat (V.24): `reforger`, `unbinding_totem`, `echo_acorn`, `glimmerdust`, `reclaimers_cache`. The 6th "special", **Spirit Gem** (`SPIRIT_GEM = "spirit_gem"`), is the emblem-maker handled inline by `combine()` -- crafting, not a run-action.

This is a content pointer; the real material lives in:

- **How the engine works** -> `docs/live/systems/items.md` (package layout, components, recipes, equip, reward drops, mana items, the modifier+hook factory pattern).
- **The data** -> `src/game/items/` (`base.py`, `combined.py`, `emblems.py`, `special.py`, `recipes.py`); ids/counts are `/check`-verified against `ITEM_REGISTRY` / `RUN_ACTION_REGISTRY`.
- **As-designed lore + intent** (frozen) -> `docs/design/content/item_catalog.md`.
- **Build rationale** (frozen) -> `docs/design/tasks/t29_item_engine_plan.md`.

3.16.2 Descriptions (T.41a) -- `game/items/meta.py` + `game/describe.py`

Player-facing item text flows through the shared description render-layer:

- `items/meta.py::ITEM_META: dict[str, ItemMeta]` -- authored **name + blurb** for **all 50** `ITEM_REGISTRY` ids (transcribed from the frozen `item_catalog.md`). `set(ITEM_META) == set(ITEM_REGISTRY)` is test-guarded (V.78). The blurb is *effect/flavor prose only* -- no stat numbers.
- `describe.render_item(id) -> RenderedEntry(name, text, stat_line)` -- the **stat line is derived by introspecting the item's** `EffectBundle` (`_bundle_stat_line`: the registered factory built with a **null owner**; `Modifier mul -> +12% STR`, `add` -> flat, crit/pen% ->

percentage). The number shown is exactly the number combat applies and cannot drift (V.78); rendering has no side effect (V.80). Hook-only riders carry no stat delta, so their effect reads from the blurb.

- **Consumer:** the Prep item chips (`ui/views/prep.py::_item_chip`) show the rendered name + a tooltip (name - stat line -- blurb - kind/action hint), replacing the old Title-case `_item_label` stopgap. (Trait descriptions: T.41b.)

3.17 Augments -- run-long modifiers

Status: **LIVING** -- audited by `/check`. **Reconciled:** 2026-07-01 (T.31 + T.42a reroll). **Scope:** augment model, registry, offer/reroll, quality curve, the `RunModifiers` combat seam, quest trackers. Code: src/game/augments.py. Design: `docs/design/tasks/t31_augment_system_plan.md`, `docs/design/content/augment_catalog.md`.

3.17.1 Model

`Augment(id, name, scope, quality, handler, piece_filter, quest_tracker, blurb)` -- frozen dataclass, registered via `@register_augment(...)` into `AUGMENT_REGISTRY` (`registries.py`). Importing `src.game.augments` populates it (mirrors abilities).

- `AugmentScope` = `TEAM` | `PIECE` | `RUN` -- selects the **handler signature**:
  - `TEAM`: `handler(team: list[Piece], state: dict) -> EffectBundle` -- modifiers/statuses applied to **each** team piece; hooks subscribed **once** (global, close over `team`).
  - `PIECE`: `handler(piece: Piece, state: dict) -> EffectBundle` -- applied to each team piece passing `piece_filter`; hooks close over that piece.
  - `RUN`: `handler(run: Run) -> None` -- mutates `Run` once at **pick time**, no bundle.
  - `state` is `RunModifiers.augment_state` -- read by run-scaling augments (`the_uprising`) and Crest bonuses.
- `AugmentQuality` = `COMMON` | `RARE` | `EPIC` | `PRISMATIC`.
- **54 registered** = 51 catalog augments + 3 Primordial-unlock RUN augments (`unlock_verdant`/`unlock_tempest`/`unlock_stoneveil`, EPIC -- gate the T10 late shop, T.28a/V.37). By quality: **COMMON 13 - RARE 13 - EPIC 16 - PRISMATIC 12** (the 3 unlocks fall in the 16 EPIC). By scope: **TEAM 30 - RUN 19 - PIECE 5**.

`Modifier.source_id` is `augment:<id>` (V.45). All handlers RNG-free (V.2/V.14). Stat magnitudes are **MVP `mul` values** (catalog ships concepts only) -- a tuning surface (D.11).

3.17.2 Combat seam (V.2 amendment, V.18)

`resolve_combat(..., run_mods=None)` / `resolve_boss_combat(..., run_mods=None)` thread a `RunModifiers(augments, augment_state, run=None)`. `None` => every non-augment caller (all balance sims) is **byte-for-byte identical** -- guarded by `test_augments.py::test_none_run_mods_byte_identical`.

`compile_loadout(..., run_mods=None)` applies augments in the documented order (loadout.py):

- **step 3** -- trait resolution reads `augment_state["trait_bonus"]` (Crest/Crown/Worldroot inject virtual carriers into `_resolve_traits` before breakpoint selection).
- **step 3.5** -- flags `piece._has_active_synergy` (the `built_different` filter).
- **step 6** -- `apply_run_augments` builds TEAM/PIECE bundles fresh (V.18: never persisted).

- **step 9** -- `wire_quest_trackers` subscribes quest trackers (no-op unless `run_mods.run` set).

3.17.3 Offers, reroll, quality curve

- `generate_augment_offer(run_seed, node_index, stage_index, *, reroll_count=0, exclude=())` -- deterministic 1-of-3 via `augment_seed(run_seed, node_index, reroll_count)` (encounter.py, V.84). For each of 3 slots: roll a quality by the stage curve (`_weighted_quality`), then pop a uniform augment from that quality's pool. No dups within the offer; `exclude` (already-active ids) filtered out; qualities with weight 0 skipped. `reroll_count` selects the draw: `0` -> `CH_AUGMENT` (fresh node offer), `1` -> `CH_REROLL` (first reroll -- **both byte-identical to the pre-reroll channels**), `>=2` -> `CH_REROLL` strided by `AUGMENT_REROLL_STRIDE` for awarded/banked rerolls (T.42a) -- replaces the old `rerolled: bool` arg.
- `rerolls_available(run, reroll_count)` / `reroll_augment_offer(run, node_index, stage_index, reroll_count)` -- game-side reroll bookkeeping (**1 base free reroll per node visit** + `augment_state["banked_rerolls"]`; only `component_stipend` banks one). `rerolls_available = (1 if reroll_count == 0 else 0) + banked`. `reroll_augment_offer` spends the free reroll first, then decrements `banked_rerolls`, and returns `(new_offer, new_count, left)` at `reroll_count + 1` -- or `None` when exhausted (view disables the button). Keeps the view Flet-free of game logic per V.63.
- **Prismatic gated to stage >= 2** (D3).
- `quality_weights_for_stage(i)` -- per-stage Common->Prismatic weights (`_STAGE_WEIGHTS`, §5 curve; tuning surface). Prismatic 0 at stage 1, non-decreasing after.
- `apply_augment(run, augment)` -- appends id to `run.active_augments`; **RUN** handlers fire immediately (mutate Amber/items/Tempest/state); TEAM/PIECE just record the id (their bundle rebuilds fresh each combat, V.18); a `quest_tracker` seeds its `augment_state` slot.
- **Node resolution.** Picking an augment is the AUGMENT node's *pick* -- the view calls `apply_augment`, then `economy.resolve_nonfight_node(run)` marks the node CLEARED + advances (V.83): **no income, tempest, Hearts, or** `battle_log` (no fight occurred).

3.17.4 Quest trackers (§9.3)

`QUEST_TRACKER_REGISTRY` + `QUEST_TRACKER_EVENTS` (`register_quest_tracker`). Run-level bus subscribers that survive across combats, mutating persistent `Run` state. MVP quests: `scouts_pay`, `prospector`, `stormbound_trail`, `bloodless_victory`, `the_uprising`, `the_long_hunt`.

3.17.5 Known simplifications (flagged, follow-ups)

- **Living World** (`living_world`) -- **redesigned weather-driven** (away from the boss-only catalog concept, which was inert on 44/50 nodes). Each live weather grants a bespoke team boon echoing its @10 affinity: CLEAR=regen+STR/INT, CLOUDY=−18% incoming, MIST=`hexproof` opener, RAIN=mana-regen+lifesteal, SNOW=2 `slow` stacks on enemies, THUNDER=AS+lightning-on-hit. Works every fight. (SNOW dogfoods the B.25/V.53 fix -- `slow` now throttles meter advancement.)
- **The Long Hunt** keys on **boss-id kills**, not a `boss_phase2` victim tag (none exists yet) (D5).
- **Primordial Bond** grants a stat tier + once-per-combat barrier; the `@2`-breakpoint-for-free and `@3` fixpoint tier-up (D.20) are **deferred** (D8).
- **Trail Rations / Adrenal Glands / Heart of the Storm** -- modelled as steady stat/regen buffs rather than exact catalog mechanics (D1).

- `salvage_rights` sets `augment_state["salvage_bonus"]`; the sell path (economy.py, T.22) must read it.

Chapter 4

AI-Agent Collaboration

Tempest Fauna Trail was built as a deliberate **vibe-coding case study**: almost all of the code was written by an AI coding agent (Claude Code) driven by a human operator, under a process designed to keep that collaboration honest. The project treats the *how we worked with the agent* as a first-class artifact -- every non-trivial milestone carries a mandatory **"Process notes (AI collaboration)"** section and a **prompting-strategy reflection** in its journal entry, precisely because that signal is invisible in the diff and unrecoverable later.

This chapter synthesizes those ~30 journaled reflections into the durable lessons. It is the part of the documentation that no amount of reading the source can reconstruct: where the agent drifted, what the process caught, and how the way of driving the agent evolved over the eight-week build.

4.1 The working model

The collaboration settled into a repeatable shape early and held:

> **The human steers design at the decision forks; the agent runs the mechanical build.** The operator resolves genuine forks up front, then lets the agent execute against a plan -- "resolve the design forks, then let the agent run" is described across the later journals as *the* dominant pattern on this project.

Concretely this is enforced by a spec-driven workflow (detailed in the next chapter): the agent writes a plan doc, the human approves it, the spec is amended, then the build is a comparatively low-entropy transcription of an already-settled design. The recurring refrain -- *"spend the tokens in planning where a mistake is cheap to fix, and the build stops being where surprises live"* -- is the thesis of the whole method, and the journals repeatedly record it paying off.

4.2 The recurring lessons

4.2.1 1. Design docs lie -- verify every primitive against the code

The single most-repeated lesson. Design documents and even the spec contain **illustrative-but-wrong** examples, and the agent will confidently emit spec-consistent, code-inconsistent prose. A partial catalogue of what verification-against-code caught before it shipped:

- `ability_power` / `mana_max` / `attack_damage` -- stat keys that **do not exist** (the real key is `intelligence`; mana is per-`ActiveSlot`, not a `Piece` stat).
- `resolve_combat(..., run_mods=None)` documented as current -- but `run_mods` was an **unbuilt future task** (T.31); the real signature had no such argument.
- Hook event names guessed (`on_attack`) rather than grepped (`on_attack_landed`).
- `CombatOutcome.TEAM_WIN` / `BattleResult.winner` in test assertions -- the real values were `CombatOutcome.WIN` and `BattleResult.outcome`.
- `ft.BarChart` mandated by the spec, but **Flet 0.85 removed the chart widgets from core** -- the framework's own API had drifted.

The generalized rule the project adopted: **writing and verifying are different jobs.** For any doc or handler that *describes* or *reads from* code, a verification-against-code pass must be its own explicit, budgeted step -- never assumed to have happened during authoring. The lesson was eventually extended even to *test assertion targets* (enum values, dataclass field names), which are easy to skip because they "obviously" match the interface you just wrote.

4.2.2 2. Self-review is a separate, load-bearing step -- green tests are not "done"

Repeatedly, an explicit adversarial review pass -- prompted by the operator as *"review the plan for weak spots"* or *"review the changes so far"* -- caught real defects that a **passing test suite did not**:

- A combat-view HP bar that would freeze through ability bursts (caught at *plan* time, before implementation -- described as the single highest-leverage moment of that session).
- A `milli_AS` value that shipped desynced under weather (weather scaled `attack_speed` but not its tiebreak twin).
- A save-load path that would leak a raw `ValueError` past the typed-error contract.

The distilled principle: *"tests passing is not 'done'; an explicit adversarial self-review is a separate step and it keeps earning its keep."*

4.2.3 3. Distrust the measurement -- a metric reading zero is a claim about the metric

Two of the sharpest bugs were **instrumentation lies**, not system bugs:

- A roster sim reported **0 casts** in a 12,000-tick fight. The first instinct ("pre-existing, out of scope") was wrong; the operator pushed *"question your sim."* Instrumenting the actual code path showed casts *were* firing -- the recorder's `_on_cast` was a silent no-op, so every cast was invisible.
- Identical mirror-match cells reading != 50% win rate exposed a genuine engine **side-bias** in the tiebreaker, affecting every prior combat result.

Rule adopted: when a metric shows a suspicious extreme (0 / all / exactly 50%), **wrap the function and instrument the path -- do not grep the rendered output.** A number that "won't move" under a lever is often a *mixed* metric hiding two problems; slice it one dimension finer before blaming the lever.

4.2.4 4. Every bug becomes an invariant -- the backprop reflex

The project's most distinctive discipline: a bug fix is not complete until the team asks *"what invariant would have caught this?"* and, where possible, adds one that turns the failure into a **red test**. This "backprop" reflex converted one-off patches into durable guards throughout -- a NaN-weather averaging bug became an invariant; a dead-stat balance hole became a CI axis<->scaling guard that *immediately* flagged two more dead-stat units a hand-audit had missed; a mana-per-slot duplication became a slot-count-invariance guard. The invariants double as **agent guardrails**: they encode, in an executable form, the exact drifts a future agent would otherwise re-introduce.

4.2.5 5. The human catches the hacks and redirects the method

The operator's interventions were consistently high-leverage, and the journals are candid that the human was often right against the agent's in-flight approach:

- *"This seems more like a hack... one logic that handles single/null/multi"* -- forced a materially cleaner multi-slot architecture than the agent's planned `secondary=` kwarg.
- *"Blast is big, blast is big"* -- vetoed the agent's twice-proposed scope shortcuts in favour of the full, correct refactor.
- *"Just run normal sims, compare win-rates across roles"* -- beat the agent's synthetic-harness instinct; the right experiment design emerged from the operator's domain corrections, not the agent's first plan.
- *"If the issue is developing blind, can you change that?"* -- flipped a self-imposed constraint (see below).

The emergent rule: **when the user smells a hack, stop and re-derive the seam -- do not defend the in-flight approach.** The recommendations the agent surfaced existed largely so they could be *cheaply overridden*; the operator overrode roughly half, which is the point of making forks legible.

4.2.6 6. Ask after a cheap investigation, not instead of one

`AskUserQuestion` forks worked best when the agent **verified a premise first**, then posed the decision with a recommendation and the concrete trade-off. Twice, feeding the user a code fact *mid-question* flipped their answer (a "does this already work?" turned into a real bug fix once the agent showed the helper didn't filter the gate; a "build it now" reversed to "defer" once the agent showed the content already existed). The pattern: **verify -> state the fact -> re-pose the fork.** For the rote decisions (dozens of invented stat magnitudes), a **decide-and-flag** approach kept momentum -- default the value, log it as a numbered flag for later review, rather than blocking on each one.

4.2.7 7. Headless UI: the operator is the visual oracle -- until it isn't

The agent cannot see rendered Flet output, so early UI work treated the human's screenshots as the only test oracle. Mitigations that worked: a **fake `Page` harness** to catch import/attribute/exception errors, and leaning on pure, testable playback models for the parts that *are* testable. But the most important lesson was to **not bake a constraint into the recommendation before checking it is real** -- when pushed, the agent found it *could* self-verify via `flet run -w` plus Playwright screenshots, which flipped the entire interaction model. The subtler traps were environmental: **web != desktop** for background-thread->UI safety (a repaint

that renders in the web build silently no-ops on desktop), and a stale **service-worker** frame that nearly sent the agent chasing a phantom bug.

4.3 How the collaboration evolved

The journals show a clear trajectory across the eight weeks:

- **Early entries** are build- and bug-post-mortems written *after* the work.
- **Mid-project**, the discipline of writing process notes for a *plan* (before any code) began surfacing drifts -- an axis-count discrepancy, a dead-field audit -- as first-class findings rather than incidental discoveries during a later build. Plan-time journaling proved worth keeping.
- **Later**, the dominant mode became **conversation-as-design-tool, then skill-chain to execute**: entire subsystems emerged from a handful of "what if X worked like Y?" turns, were locked into the plan and spec, and only then built. The skill chain (`/plan` -> `/spec` -> `/build` -> `/check`, re-entering `/spec` to correct a falsified claim) held its seams -- routing even mid-build spec corrections through the sole mutator kept the encoding and numbering discipline intact and left an honest trail.

Two organizational lessons round it out. **Scope is escalated, not half-built**: when a task's real size exceeded its estimate, it was split along a real seam (traits became T.28 a/b/c/d, each shipping green) rather than left perpetually "80% done." And **concurrency demands blast-radius discipline**: a two-agent parallel experiment succeeded only because the *shared test baseline* -- not just file ownership -- was made explicit, and a careless `git add -A` in a shared worktree once swept another stream's spec edits into the wrong commit (producing a duplicate invariant), yielding the standing rule to **stage your own paths explicitly, never `-A`, in a shared tree.**

The through-line: the process exists to compensate for the agent's specific failure modes -- confident-but-unverified prose, trust in green tests, trust in its own measurements -- by making verification, self-review, and human design authority into explicit, load-bearing steps rather than assumed ones.

Chapter 5

How the Project Was Built

Tempest Fauna Trail is a FH Technikum Wien project -- two students, eight weeks -- graded on UI, data loading, visualization, code structure, and documentation. What follows is how it was actually built: the **spec-driven method** that governed every change, and the **implementation arc** that method produced, from data models to a playable run loop.

5.1 Spec-Driven Development

The organizing idea is that a single **contract** -- SPEC.md -- drives the work, and every other document has a defined and different relationship to the truth. Nothing lands without passing through the contract.

5.1.1 The document layers and the LIVING vs FROZEN rule

The rule that prevents documentation drift is the distinction between **LIVING** and **FROZEN** docs:

Layer	Doc	Role	Currency
Contract	`SPEC.md`	the full contract (§G/§C/§I/§V/§T/§B/§D)	LIVING
Map	`ARCHITECTURE.md`	where things live and how they interact	LIVING
Reference	`docs/live/**`	how each subsystem works *now*	LIVING
Record	`docs/design/**`	how a task was *planned/built*	FROZEN
History	`docs/journal/**`	*why*, chronologically	FROZEN

LIVING docs describe how things work *now* and **must match the code** -- drift is treated as a bug, the same as a failing test. **FROZEN** docs are point-in-time records that are *never* retro-edited; a frozen task plan is read as a dated snapshot, not current truth. The recurring, expensive failure the project kept paying down was reading a frozen plan as if it were live -- so the discipline is: when a change lands, reconcile the matching LIVING doc in the same commit, and leave the frozen plan alone.

5.1.2 The skill chain

The workflow is mechanized as a chain of skills, each with a single, bounded job:

- `/plan` -- writes a `docs/design/tasks/tN_*_plan.md` before any build; runs a required-reading pass and verifies every primitive against the code; ends in the exact spec deltas needed. Writes only the plan doc.
- `/spec` -- the **sole mutator** of SPEC.md. New rows, invariants, bug backprop. Keeping one mutator preserves the numbering and encoding discipline.
- `/build` -- plan-then-execute against a §T task; flips task status; auto-runs backprop on a test failure.
- `/check` -- a read-only drift audit: every backticked code reference in the LIVING docs must resolve, every invariant must hold, content counts must match. It reports; it never edits.

The seams held in practice: each stage's output is the next stage's contract, so a mistake gets caught at the cheapest layer. When a build *falsified* a plan's claim (for instance, a change that was assumed byte-identical turned out to shift tick timing), the correction was routed back through `/spec` rather than buried -- the build is allowed to falsify the plan, and the contract is corrected in the open.

5.1.3 Invariants as executable guardrails

The `§V` invariants are the backbone. They are not aspirational prose -- most are **CI-guarded**, and each new one is typically born from a bug via *backprop* (the "what invariant would have caught this?" reflex from the previous chapter). A sampling of the load-bearing ones:

- **V.1** -- `game/` has zero Flet imports (pure logic).
- **V.2** -- `resolve_combat` is a pure function: identical inputs yield byte-identical output. No RNG, no clock, no globals.
- **V.14 / V.19** -- the simulation layer imports only pure game logic; all procedural generation derives from `(run_seed, node_index, channel)`.
- **V.3 / V.4** -- the API key is never logged; all HTTP runs on a worker thread.
- Content and balance guards (id resolution, axis<->scaling coverage, weather-metric NaN handling) that turn "silently wrong content" into red tests.

5.2 The implementation arc

Development proceeded **engine-first, UI-last** -- a direct consequence of the purity invariants. Because `game/` is Flet-free and I/O-free, the entire game is testable and simulatable without a UI or a network, so the whole engine, content, and balance layers could be built and validated through the playtest CLI and the power simulation long before any view existed. The tasks (`§T`) fall into a few waves:

5.2.1 Foundation (the deterministic core)

`T.1` data models -> `T.2` the two decoupled weather systems (Weather Favor + Affinity Clash) -> `T.3` the tick-based combat engine -> `T.4` the 50-city / 6-stage route -> `T.5` the champion/enemy rosters -> `T.6`/`T.7` the OpenWeather client, cache, and 3-stream refresher -> `T.18` the power/scaling formula. This wave established the determinism doctrine that everything downstream relies on.

5.2.2 Framework and tooling

T.19 seed-deterministic encounter generation -> T.20 the ability / passive / status framework (typed event bus, hooks, modifiers, registries) -- the declarative substrate all content plugs into -> T.21 challenge and 2-phase boss encounters with map effects -> T.24 role-aware enemy formation -> T.26 unification of combat onto a single tick loop (the old parallel engine was deleted). Alongside, T.25 the power simulation (matchup sweeps, win-rate/power metrics) and T.27 the playtest CLI gave the project its two ways to **drive the game headlessly** -- the tools that made engine-first development trustworthy.

5.2.3 Content deepening and balance

The largest wave, and the one where the spec-driven method earned the most. Traits were built in a deliberate split -- T.28a (framework + declarative stat packs) -> T.28b (engine primitives) -> T.28c (mechanic/apex riders via hook idioms) -> T.28d (hexproof correctness + fold-ins). Items followed the same shape: T.29-pre reworked the combat **stat substrate** (weather-as-modifiers, attack_speed to float), then T.29a-T.29d delivered the item engine, the full catalog, the **mana primitive**, and multi-slot pieces. T.30 implemented all 120 roster ability handlers plus 6 boss kits; T.31 the augment system; T.32 the role/intent axis rework; T.33a the scaling classes and a fair total order (T.33b the speed-axis diversity spread 3->7); T.34a-c the per-roster ability metadata and T.35a the ability-description/tooltip layer (T.35b a dead-stat balance pass); and T.36a-c the roster axis-distribution rebalance, closed with power sweeps. Much of this wave's value was in **design conversation and balance analysis** rather than code -- for example, the INT-ability damage coefficient was tuned empirically *and* cross-checked against a closed-form DPS-parity equilibrium ($\sim= 3.7$), so the balance pass ended with math and simulation agreeing, and produced a reusable coefficient rule for future kits.

5.2.4 The player-facing UI (built last, over the finished engine)

With a complete engine, the view layer was built last: T.8 theme/components -> T.9 menu -> T.10 run-start -> T.11 Trail view + Canvas route map -> T.12a-d the combat view (a forward replay stepper, animations, boss support, autoplay) -> T.13 the Canvas run-summary chart -> T.14 save/load -> T.15a/b the reward step and full routing -> T.23a/b the Prep view (economy, shop, board placement, item equip) -> T.37a-c the replay backend that combat playback rides on -> T.38 node-type rewards and survivable "Hearts" -> T.39 persistent live node weather -> T.40 the Prep UX overhaul -> T.41a/b the description render layer -> T.42a/b the augment- and supply-node UIs plus the non-fight run-loop dispatch (and the viz/affinity_clash_heatmap Canvas). With T.42 the menu->...->summary loop is complete end to end. This wave depended on a hard constraint -- the agent cannot see rendered output -- which shaped the collaboration lessons of the previous chapter (visual gating, self-screenshotting, web-vs-desktop threading traps).

5.3 Determinism as the spine

If one property holds the whole project together, it is **determinism**. Because resolve_combat is pure and all generation is seeded, the same seed yields the same route, squads, shop, economy, and byte-identical combat -- which is exactly what makes the simulation layer a trustworthy balance oracle and the combat replay possible (state for a view is *recomputed* from the seed, not recorded). Every engine or content change was argued for byte-identity or an explicit, sanctioned

re-baseline, and the distinction between ”byte-identical” and ”deterministic but re-baselined” was tracked carefully in the spec. This is the invariant that lets a two-person, eight-week, agent-built project stay coherent: the contract says what must hold, the invariants make it checkable, and the seed makes it reproducible.